

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
ПО ПРОГРАММЕ БАКАЛАВРИАТА**

09.03.04 Программная инженерия

(код, наименование ОПОП ВО: направление подготовки, направленность (профиль))

«Разработка программно-информационных систем»

Разработка компонентно-ориентированной программной системы

для построения интерактивных приложений на Java

(название темы)

Дипломный проект

(вид ВКР: дипломная работа или дипломный проект)

Автор ВКР

(подпись, дата)

К. Р. Алешин

(инициалы, фамилия)

Группа ПО-226

Руководитель ВКР

(подпись, дата)

А. Ф. Рубанов

(инициалы, фамилия)

Нормоконтроль

(подпись, дата)

А. А. Чаплыгин

(инициалы, фамилия)

ВКР допущена к защите:

Заведующий кафедрой

(подпись, дата)

А. В. Малышев

(инициалы, фамилия)

Курск 2026 г.

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

УТВЕРЖДАЮ:

Заведующий кафедрой

(подпись, инициалы, фамилия)

« ____ » _____ 20 ____ г.

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ ПО ПРОГРАММЕ БАКАЛАВРИАТА

Студента Алешина К.Р., шифр 22-06-0044, группа ПО-226

1. Тема «Разработка компонентно-ориентированной программной системы для построения интерактивных приложений на Java» утверждена приказом ректора ЮЗГУ от «03» апреля 2026 г. № 1585-с.

2. Срок предоставления работы к защите «9» июня 2026 г.

3. Исходные данные для создания программной системы:

3.1. Перечень решаемых задач:

1) провести анализ предметной области и существующих программных средств построения интерактивных приложений;

2) сформулировать требования к компонентно-ориентированной программной системе и определить порядок ее использования разработчиком;

3) спроектировать архитектуру программной системы, включающую ядро компонентной модели, управление приложением и сценами, графическую подсистему, обработку ввода, скрипты и базовую физику;

4) реализовать основные классы, компоненты, системы обработки и демонстрационные сцены;

5) выполнить модульное, регрессионное и системное тестирование разработанной программной системы.

3.2. Входные данные и требуемые результаты для программы:

1) Входными данными для программной системы являются: исходный код приложения на языке Java, описывающий сцены, сущности, компоненты, системы обработки и пользовательские скрипты; параметры запуска приложения; графические ресурсы, включая шейдеры, текстуры и геометрические данные; события пользовательского ввода с клавиатуры и мыши; параметры объектов сцены, камер, источников света, физических тел и коллайдеров.

2) Требуемыми результатами работы программы являются: сформированное настольное интерактивное приложение на языке Java; отображаемые кадры активной сцены; обновленное состояние сущностей и компонентов сцены; результаты обработки пользовательского ввода; результаты выполнения пользовательских скриптов; результаты базовой обработки физических взаимодействий; диагностические сообщения и отладочная визуализация; демонстрационные сцены, подтверждающие работоспособность основных подсистем.

4. Содержание работы (по разделам):

4.1. Введение.

4.2. Анализ предметной области: интерактивные приложения как класс программных систем, программные средства построения интерактивных приложений, компонентно-ориентированный подход к организации объектов сцены, разработка интерактивных приложений на Java, анализ существующих решений.

4.3. Техническое задание: основание для разработки, цель и назначение разработки, требования к программной системе, требования к архитектурной модели, управлению приложением и сценами, графической подсистеме, пользовательскому вводу, скриптам, базовой физике, демонстрационным сценами, программному интерфейсу, нефункциональным характеристикам и оформлению документации.

4.4. Технический проект: общие сведения о программной системе, обоснование выбора технологий, внутреннее устройство программной системы, структура пакетов, жизненный цикл приложения и сцены, проектирование

ядра компонентной модели, графической подсистемы, подсистемы ввода, скриптов, ресурсов, базовой физики и демонстрационных сцен.

4.5. Рабочий проект: спецификация реализованных компонентов и классов программной системы, описание математических, графических, ресурсных и физических классов, систем обработки и демонстрационных сцен, модульное, регрессионное и системное тестирование программной системы.

4.6. Заключение.

4.7. Список использованных источников.

5. Перечень графического материала:

Лист 1. Сведения о ВКРБ

Лист 2. Цель и задачи разработки

Лист 3. Архитектура программной системы

Лист 4. Компонентная модель программной системы

Лист 5. Жизненный цикл кадра приложения

Лист 6. Результаты разработки и тестирования

Руководитель ВКР

(подпись, дата)

А. Ф. Рубанов

(инициалы, фамилия)

Задание принял к исполнению

(подпись, дата)

К. Р. Алешин

(инициалы, фамилия)

РЕФЕРАТ

Объем работы равен 162 страницам. Работа содержит 9 иллюстраций, 18 таблиц, 55 библиографических источников и 6 листов графического материала. Количество приложений – 2. Графический материал представлен в приложении А. Фрагменты исходного кода представлены в приложении Б.

Перечень ключевых слов: интерактивное приложение, компонентный подход, компонентно-ориентированная система, Java, LWJGL, OpenGL, GLSL, сцена, сущность, компонент, система обработки, рендеринг, физическая подсистема.

Объектом разработки является компонентно-ориентированная программная система для построения интерактивных приложений на языке Java.

Целью выпускной квалификационной работы является разработка программной системы, предоставляющей Java-разработчику основные средства создания интерактивных приложений: управление приложением и сценами, компонентную модель объектов, обработку пользовательского ввода, графический вывод, выполнение пользовательских скриптов и базовую физическую обработку.

В процессе выполнения работы проведен анализ предметной области и существующих программных средств построения интерактивных приложений. Сформулированы требования к программной системе, спроектирована архитектура приложения, сцены, сущностей, компонентов и систем обработки. В качестве основных технологий использованы язык Java, библиотека LWJGL, графический API OpenGL, язык шейдеров GLSL, а также библиотеки GLFW и STB.

В результате работы реализованы ядро компонентной модели, управление жизненным циклом приложения и сцен, графическая подсистема, подсистема ресурсов, обработка пользовательского ввода, система пользовательских скриптов, отладочная визуализация и базовая физическая подсистема. Для проверки работоспособности разработаны демонстрационные сце-

ны, показывающие использование камеры, освещения, текстур, материалов, скриптов, пользовательского ввода, коллайдеров и физических тел.

Разработанная программная система может использоваться как основа для создания учебных, демонстрационных и исследовательских интерактивных приложений на Java, а также для изучения внутреннего устройства компонентной архитектуры, графической подсистемы и базовой физической обработки.

ABSTRACT

The volume of work is 162 pages. The work contains 9 illustrations, 18 tables, 55 bibliographic sources and 6 sheets of graphic material. The number of applications is 2. The graphic material is presented in annex A. Source code fragments are presented in annex B.

List of keywords: interactive application, component-based approach, component-oriented system, Java, LWJGL, OpenGL, GLSL, scene, entity, component, processing system, rendering, physics subsystem.

The object of the development is a component-oriented software system for building interactive applications in Java.

The purpose of the final qualification work is to develop a software system that provides a Java developer with the basic tools for creating interactive applications: application and scene management, a component-based object model, user input processing, graphical output, user script execution and basic physics processing.

During the work, the subject area and existing software tools for building interactive applications were analyzed. The requirements for the software system were formulated, and the architecture of the application, scene, entities, components and processing systems was designed. The main technologies used in the development are the Java programming language, the LWJGL library, the OpenGL graphics API, the GLSL shading language, as well as the GLFW and STB libraries.

As a result of the work, the core of the component model, the life cycle management of the application and scenes, the graphics subsystem, the resource subsystem, user input processing, the user script system, debug visualization and the basic physics subsystem were implemented. Demonstration scenes were developed to verify the functionality of the system and show the use of camera, lighting, textures, materials, scripts, user input, colliders and physical bodies.

The developed software system can be used as a basis for creating educational, demonstration and research interactive applications in Java, as well as for

studying the internal structure of component architecture, the graphics subsystem and basic physics processing.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	15
1 Анализ предметной области	19
1.1 Интерактивные приложения как класс программных систем	19
1.2 Программные средства построения интерактивных приложений	21
1.3 Компонентно-ориентированный подход к построению интерактивных приложений	23
1.4 Ключевые сущности предметной области	25
1.5 Особенности разработки интерактивных приложений на Java	27
1.6 Анализ существующих программных решений	28
1.7 Выводы по анализу предметной области	31
2 Техническое задание	33
2.1 Основание для разработки	33
2.2 Цель и назначение разработки	33
2.3 Требования к программной системе	35
2.3.1 Требования к архитектурной модели	35
2.3.2 Требования к управлению приложением и сценами	36
2.3.3 Требования к графической подсистеме	37
2.3.4 Требования к пользовательскому вводу	37
2.3.5 Требования к системе пользовательских скриптов	38
2.3.6 Требования к базовой физической подсистеме	38
2.3.7 Требования к демонстрационным сценам	39
2.4 Руководство по использованию программной системы	40
2.4.1 Подготовка окружения	40
2.4.2 Создание приложения	42
2.4.3 Создание сцены и добавление систем	43
2.4.4 Добавление источника света	44
2.4.5 Создание камеры	44
2.4.6 Создание статического объекта	45
2.4.7 Создание динамического физического тела	45

2.4.8	Создание кинематического объекта	46
2.4.9	Создание пользовательского скрипта	47
2.4.10	Использование пользовательского ввода	48
2.4.11	Создание отладочной геометрии	48
2.4.12	Общий порядок создания сцены	49
2.5	Требования к программному интерфейсу	49
2.6	Нефункциональные требования	50
2.6.1	Требования к программному обеспечению	50
2.6.2	Требования к аппаратному обеспечению	51
2.6.3	Требования к надежности	51
2.6.4	Требования к сопровождаемости	52
2.6.5	Требования к безопасности	52
2.7	Требования к оформлению документации	53
3	Технический проект	54
3.1	Общие сведения о программной системе	54
3.2	Обоснование выбора технологий	56
3.2.1	Язык программирования Java	57
3.2.2	Библиотека LWJGL	58
3.2.3	OpenGL и GLSL	58
3.2.4	GLFW и STB	59
3.3	Внутреннее устройство программной системы	59
3.3.1	Структура пакетов проекта	59
3.3.2	Основные элементы внутреннего устройства	60
3.3.3	Устройство приложения и контекста выполнения	63
3.3.4	Устройство сцены	63
3.3.5	Сущности, компоненты и системы обработки	64
3.3.6	Жизненный цикл приложения и сцены	65
3.4	Проектирование основных подсистем	67
3.4.1	Подсистема управления сценами	67
3.4.2	Графическая подсистема	67
3.4.3	Подсистема ресурсов	70

3.4.4	Подсистема пользовательского ввода	70
3.4.5	Подсистема пользовательских скриптов	71
3.4.6	Подсистема отладочной визуализации	71
3.5	Проектирование физической подсистемы	71
3.5.1	Общая структура физической подсистемы	72
3.5.2	Математическая модель движения	73
3.5.3	Метод численного интегрирования	74
3.5.4	Обнаружение столкновений	75
3.5.5	Разрешение столкновений	76
3.5.6	Архитектурные ограничения физической подсистемы	77
3.6	Проектирование демонстрационных сцен	78
3.7	Выводы по разделу	80
4	Рабочий проект	81
4.1	Спецификация компонентов программной системы	81
4.1.1	Вспомогательные математические классы	82
4.1.2	Классы ядра компонентной модели	84
4.1.3	Классы управления приложением и сценами	85
4.1.4	Компоненты сцены	87
4.1.5	Графические и ресурсные классы	89
4.1.6	Системы обработки	91
4.1.7	Демонстрационные сцены	94
4.2	Модульное и регрессионное тестирование программной системы	97
4.2.1	Тестирование ядра компонентной модели	97
4.2.2	Тестирование подсистемы ввода	99
4.2.3	Тестирование менеджера сцен	100
4.2.4	Тестирование физической подсистемы	101
4.3	Системное тестирование программной системы	103
4.3.1	Проверка сцены ShowcaseScene	104
4.3.2	Проверка сцены Interactive2DScene	105
4.3.3	Проверка сцены TexturedLightingScene	107
4.3.4	Проверка сцены PhysicsScene	109

ЗАКЛЮЧЕНИЕ	113
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	115
ПРИЛОЖЕНИЕ А Представление графического материала	121
ПРИЛОЖЕНИЕ Б Фрагменты исходного кода программы	128
На отдельных листах (CD-RW в прикрепленном конверте)	162
Сведения о ВКРБ (Графический материал / Сведения о ВКРБ.png)	Лист 1
Цель и задачи разработки (Графический материал / Цель и задачи разработки.png)	Лист 2
Архитектура программной системы (Графический материал / Архитектура программной системы.png)	Лист 3
Компонентная модель программной системы (Графический материал / Компонентная модель программной системы.png)	Лист 4
Жизненный цикл кадра приложения (Графический материал / Жизненный цикл кадра приложения.png)	Лист 5
Результаты разработки и тестирования (Графический материал / Результаты разработки и тестирования.png)	Лист 6

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

ВКР – выпускная квалификационная работа.

ОПОП ВО – основная профессиональная образовательная программа высшего образования.

ПО – программное обеспечение.

ЮЗГУ – Юго-Западный государственный университет.

AABB (Axis-Aligned Bounding Box) – ограничивающий параллелепипед, выровненный по осям координат.

API (Application Programming Interface) – программный интерфейс приложения.

EBO (Element Buffer Object) – объект буфера индексов OpenGL.

ECS (Entity-Component-System) – архитектурный подход к построению программных систем, основанный на сущностях, компонентах и системах обработки.

GLFW – библиотека, используемая для создания окон, управления контекстом OpenGL и обработки пользовательского ввода.

GLSL (OpenGL Shading Language) – язык программирования шейдеров OpenGL.

GPU (Graphics Processing Unit) – графический процессор.

JAR (Java Archive) – архивный формат, используемый для распространения Java-классов и связанных с ними ресурсов.

JDK (Java Development Kit) – комплект средств разработки для платформы Java.

JVM (Java Virtual Machine) – виртуальная машина Java.

LWJGL (Lightweight Java Game Library) – библиотека, предоставляющая Java-приложениям доступ к низкоуровневым графическим и системным API.

OpenGL (Open Graphics Library) – кроссплатформенный графический API для вывода двухмерной и трехмерной графики.

SAT (Separating Axis Theorem) – теорема разделяющей оси, используемая для проверки пересечения выпуклых тел.

SDK (Software Development Kit) – комплект средств разработки.

STB – набор вспомогательных библиотек, применяемых в том числе для загрузки изображений и других мультимедийных ресурсов.

UML (Unified Modeling Language) – язык графического описания для моделирования программных систем.

VAO (Vertex Array Object) – объект массива вершин OpenGL.

VBO (Vertex Buffer Object) – объект буфера вершин OpenGL.

ВВЕДЕНИЕ

Интерактивные приложения занимают важное место среди современных программных систем. К ним относятся учебные и инженерные симуляторы, демонстрационные трехмерные сцены, визуализаторы, игровые прототипы, виртуальные лаборатории и другие программы, в которых результат работы зависит от постоянного взаимодействия пользователя с объектами сцены. Такие приложения должны не только выводить изображение на экран, но и регулярно обновлять внутреннее состояние, обрабатывать ввод с клавиатуры и мыши, управлять временем кадра, изменять положение объектов, выполнять пользовательские сценарии поведения и формировать новое изображение с малой задержкой.

Разработка подобных программ связана с повторяющимся набором технических задач. Разработчику необходимо создать окно приложения, организовать основной цикл, хранить объекты сцены, задавать их свойства, подключать графические ресурсы, описывать поведение объектов, обрабатывать пользовательский ввод и, при необходимости, рассчитывать простые физические взаимодействия. Если все эти механизмы реализуются заново для каждого отдельного приложения, значительная часть времени тратится не на прикладную логику, а на повторное создание базовой программной инфраструктуры.

Для решения этой проблемы используются программные основы и движки, предоставляющие готовые механизмы работы со сценами, объектами, ресурсами и графикой. Однако существующие решения не всегда удобны для небольших учебных и исследовательских проектов. Полноценные игровые движки и крупные фреймворки обладают большим количеством возможностей, но могут быть избыточными, сложными в изучении и не всегда позволяют подробно рассмотреть внутреннее устройство системы. Низкоуровневые библиотеки, напротив, дают доступ к графическому API и средствам ввода, но оставляют разработчику необходимость самостоятельно проектировать архитектуру приложения. Например, LWJGL предоставляет Java-доступ

к низкоуровневым библиотекам, в том числе OpenGL и GLFW, но не задает готовую объектную модель приложения [37]. OpenGL, в свою очередь, решает задачу вывода графики, но не определяет способ организации сцен, компонентов и систем обработки [39].

В связи с этим актуальной является разработка компонентно-ориентированной программной системы, которая занимает промежуточное положение между низкоуровневой графической библиотекой и крупным готовым движком. Такая система должна предоставлять Java-разработчику основные средства построения интерактивного приложения, но при этом оставаться достаточно простой и открытой для изучения, модификации и расширения.

Основой разрабатываемой системы является компонентный подход к организации объектов сцены. В рамках этого подхода объект не описывается глубокой иерархией наследования. Вместо этого он представляется сущностью, к которой добавляются независимые компоненты: трансформация, графическое представление, камера, источник света, физическое тело, коллайдер или пользовательский скрипт. Обработка таких объектов выполняется системами, каждая из которых отвечает за отдельную часть поведения приложения. Подобная организация позволяет переиспользовать компоненты, добавлять новую функциональность без изменения базового класса объекта и разделять разные аспекты работы сцены.

Разрабатываемая программная система ориентирована на создание настольных интерактивных приложений на языке Java. Ее пользователем является не конечный пользователь готового приложения, а разработчик, который создает классы сцен, добавляет сущности и компоненты, подключает системы обработки и описывает поведение объектов средствами программного кода. В составе системы рассматриваются ядро компонентной модели, управление приложением и сценами, графическая подсистема, подсистема ресурсов, обработка пользовательского ввода, скрипты, отладочная визуализация и базовая физическая подсистема.

Цель настоящей работы – разработка компонентно-ориентированной программной системы для построения интерактивных приложений на Java.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области и существующих программных средств построения интерактивных приложений;
- сформулировать требования к компонентно-ориентированной программной системе и определить порядок ее использования разработчиком;
- спроектировать архитектуру системы, включающую ядро компонентной модели, управление сценами, графическую подсистему, обработку ввода, скрипты и базовую физику;
- реализовать основные классы, компоненты, системы обработки и демонстрационные сцены;
- выполнить модульное, регрессионное и системное тестирование разработанной программной системы.

Структура работы. Выпускная квалификационная работа состоит из введения, четырех разделов основной части, заключения, списка использованных источников и приложений.

Во введении обоснована актуальность выбранной темы, сформулированы цель и задачи разработки, а также приведена структура выпускной квалификационной работы.

В первом разделе выполняется анализ предметной области. Рассматриваются интерактивные приложения как класс программных систем, особенности их внутренней организации, компонентно-ориентированный подход, ключевые сущности предметной области, особенности разработки на языке Java и существующие программные решения.

Во втором разделе приводится техническое задание. В нем указано основание для разработки, определены цель и назначение программной системы, сформулированы требования к архитектурной модели, управлению приложением и сценами, графической подсистеме, пользовательскому вводу, скриптам, базовой физике, демонстрационным сценами, программному

интерфейсу, нефункциональным характеристикам и оформлению документации.

В третьем разделе представлен технический проект программной системы. В разделе обоснован выбор технологий, описаны Java, LWJGL, OpenGL, GLSL, GLFW и STB, рассмотрено внутреннее устройство системы, структура пакетов, жизненный цикл приложения и сцены, а также проектирование основных подсистем и демонстрационных сцен.

В четвертом разделе приведен рабочий проект. В нем представлена спецификация реализованных классов и компонентов программной системы, описаны математические, графические, ресурсные и физические классы, системы обработки и демонстрационные сцены. Также в разделе приведены результаты модульного, регрессионного и системного тестирования.

В заключении излагаются основные результаты, полученные при выполнении выпускной квалификационной работы.

В приложении А представлен графический материал, подготовленный для защиты выпускной квалификационной работы. В приложении Б приведены фрагменты исходного кода разработанной программной системы.

1 Анализ предметной области

1.1 Интерактивные приложения как класс программных систем

Интерактивным приложением называют программную систему, результат работы которой зависит не только от заранее заданных данных, но и от постоянного взаимодействия с пользователем. Такое приложение получает события от клавиатуры, мыши или другого устройства ввода, изменяет внутреннее состояние и выводит обновленное изображение с достаточно малой задержкой, чтобы пользователь воспринимал происходящее как непрерывный процесс.

К интерактивным приложениям относятся компьютерные игры, учебные симуляторы, демонстрационные 3D-сцены, инженерные визуализаторы, виртуальные лаборатории, средства прототипирования интерфейсов и интерактивные художественные инсталляции. Эти программы различаются по назначению, но имеют общую техническую основу. Для них характерны периодическое обновление состояния, отрисовка кадра, обработка ввода и управление набором объектов, присутствующих в сцене.

Обычное прикладное приложение часто строится вокруг запроса пользователя: пользователь нажимает кнопку, программа выполняет действие и возвращается в состояние ожидания. Интерактивное приложение работает иначе. В нем присутствует непрерывный цикл: программа опрашивает устройства ввода, рассчитывает изменения за прошедший промежуток времени, обновляет положение и состояние объектов, после чего формирует новый кадр. Такой цикл принято называть главным циклом приложения или игровым циклом, даже если разрабатываемая программа не является игрой [1, 2].

В простейшем виде работа интерактивного приложения может быть представлена схемой, приведенной на рисунке 1.1.



Рисунок 1.1 – Обобщенная схема работы интерактивного приложения

На практике каждый из показанных этапов распадается на несколько подзадач. Обработка ввода включает регистрацию нажатий клавиш, перемещения мыши, изменения состояния кнопок и колесика прокрутки. Обновление состояния объектов связано с вычислением трансформаций, анимаций, перемещений, пользовательских сценариев поведения. Обработка столкновений требует определения пересечений между объектами, реакции на контакт и, при необходимости, изменения скорости или положения объектов. Отрисовка кадра предполагает подготовку данных для графического процессора, выбор камеры, материалов, источников света, шейдеров и очередности вывода объектов. Общие принципы графического конвейера, преобразований, освещения и вывода изображений в реальном времени рассматриваются в работах по компьютерной графике и рендерингу [16, 19, 20].

Особенность таких приложений состоит в том, что их внутренняя структура быстро усложняется с ростом числа объектов. Один объект может иметь положение в пространстве, графическое представление, коллайдер, физическое тело, пользовательский сценарий поведения, источник света или камеру. При этом разные объекты используют разные сочетания этих возможностей. Например, камера не должна иметь графическую модель, а декоративный объект может не участвовать в столкновениях. Если каждую разновидность объекта описывать отдельным классом, то количество клас-

сов и связей между ними начинает расти быстрее, чем сама предметная область. Именно поэтому при разработке интерактивных приложений особенно важен выбор архитектурного подхода.

1.2 Программные средства построения интерактивных приложений

Разработка интерактивного приложения с нуля требует решения большого числа технических задач, не связанных напрямую с логикой конкретного проекта. Разработчику нужно создать окно, получить графический контекст, организовать цикл обновления, загрузить ресурсы, отрисовать геометрию, обработать ввод, хранить состояние сцены, описать поведение объектов и обеспечить освобождение ресурсов. Поэтому на практике редко используют только стандартную библиотеку языка программирования. Обычно выбирают один из классов программных средств: низкоуровневую графическую библиотеку, фреймворк, игровой движок или специализированную среду прототипирования.

Низкоуровневая графическая библиотека предоставляет доступ к аппаратно-ускоренной графике, но почти не навязывает архитектуру приложения. Примером является OpenGL. Khronos Group описывает OpenGL как кроссплатформенный API для отрисовки 2D- и 3D-графики [39]. Такой API решает задачу взаимодействия с графическим процессором, но не предоставляет готовой модели сцены, системы сущностей, иерархии объектов или компонентной архитектуры. Программист получает контроль над рендерингом, но вынужден самостоятельно проектировать остальную инфраструктуру приложения.

Для языка Java важную роль играет LWJGL. Официальный сайт характеризует LWJGL как библиотеку, предоставляющую Java-приложениям кроссплатформенный доступ к нативным API, включая OpenGL, OpenAL и OpenCL [37]. Фактически LWJGL является мостом между Java и низкоуровневыми средствами работы с графикой, звуком и окном. Это делает библиотеку удобной основой для собственных движков и графических инструментов,

но она не является готовой системой построения сцен. Разработчику необходимо самостоятельно реализовывать структуру объектов, управление ресурсами, обновление состояния и пользовательские абстракции.

Фреймворк находится на более высоком уровне. Он не только предоставляет доступ к графике и вводу, но и задает часть структуры приложения. Одним из наиболее известных Java-фреймворков является libGDX. В официальном описании libGDX определяется как кроссплатформенный Java-фреймворк для разработки игр на основе OpenGL ES, поддерживающий Windows, Linux, macOS, Android, браузер и iOS [44]. Его сильная сторона заключается в переносимости и широком наборе готовых средств. Однако libGDX оставляет разработчику свободу в выборе архитектуры предметных объектов. Для небольших проектов это удобно, но при построении системы из множества объектов разработчик все равно должен заранее решить, как именно будут описываться сущности, компоненты и связи между ними.

Игровой движок поднимает уровень абстракции еще выше. Он обычно содержит модель сцены, подсистемы рендеринга, ввода, ресурсов, анимации, физики и инструменты для разработки. Для Java таким решением является jMonkeyEngine. На официальном сайте он описан как современный игровой движок, написанный преимущественно на Java и ориентированный на разработчиков, которым нужен контроль над кодом и возможность расширять движок под собственный процесс разработки [45]. Это полноценный инструмент, но вместе с полнотой возможностей появляется и более высокий порог входа. Пользователь должен принять архитектурные решения движка, изучить его жизненный цикл, систему ресурсов, пространственную модель и набор классов.

Отдельную группу составляют среды прототипирования и визуального программирования. Processing, согласно официальному описанию, является гибким программным «скетчбуком» и языком для обучения программированию, который с 2001 года используется в визуальных искусствах, образовании и быстром прототипировании [46]. Такие инструменты снижают барьер входа, но их модель обычно ориентирована на небольшие эскизы, визуаль-

ные эксперименты и учебные проекты. Для разработки расширяемой компонентной системы они подходят хуже, чем библиотека или фреймворк, поскольку скрывают часть инфраструктуры и задают собственный стиль организации программы.

Таким образом, между низкоуровневой библиотекой и полноценным игровым движком существует промежуточная ниша. Java-разработчику может быть нужна не готовая тяжелая среда с редактором и большим набором подсистем, а легковесная программная основа, позволяющая строить интерактивные сцены из сущностей, компонентов и систем. Такая основа должна закрывать типовые задачи цикла приложения, сцены, рендеринга, ввода и базовой обработки столкновений, но при этом оставаться понятной и расширяемой на уровне исходного кода.

1.3 Компонентно-ориентированный подход к построению интерактивных приложений

Одним из главных вопросов при проектировании интерактивного приложения является способ описания объектов сцены. В традиционном объектно-ориентированном подходе разработчик часто начинает с иерархии классов: базовый класс объекта, затем классы персонажей, снарядов, декораций, источников света, камер, физических объектов. На раннем этапе такая схема кажется удобной. Например, можно создать базовый класс `GameObject`, от него унаследовать `RenderableObject`, затем `PhysicalRenderableObject` и так далее.

Проблема появляется, когда свойства начинают комбинироваться. Объект может быть видимым, но не иметь физического тела. Другой объект может участвовать в столкновениях, но не отображаться на экране. Третий объект может иметь скрипт поведения и источник света. При наследовании такие сочетания приводят либо к очень глубокой иерархии, либо к дублированию кода в разных ветвях. Изменение одного свойства становится изменением класса, а не добавлением новой возможности к объекту.

Компонентный подход решает эту проблему иначе. Объект сцены рассматривается не как экземпляр специализированного класса, а как контейнер

для набора независимых компонентов. Компонент описывает одну характеристику или одну сторону поведения объекта: положение в пространстве, камеру, отрисовку сетки, источник света, физическое тело, коллайдер, пользовательский скрипт. Чтобы изменить объект, разработчик добавляет или удаляет компоненты, не создавая новый класс для каждого сочетания возможностей.

Идея отказа от жесткой иерархии наследования в пользу композиции объектов рассматривается в практических материалах по компонентным объектным системам и ECS-фреймворкам [28, 29, 30, 31, 32, 33].

Преимущество компонентной модели хорошо видно на простом примере. Объект, имеющий компоненты Transform и MeshRenderer, может быть отрисован как обычная модель. Если к нему добавить компонент Rigidbody, он начинает участвовать в физической обработке. Если добавить компонент Script, объект получает пользовательское поведение. При этом базовая сущность остается той же самой, а набор возможностей определяется составом компонентов.

В популярных инструментах разработки интерактивных приложений компонентная идея встречается достаточно часто. Например, в Unity функциональность объекта сцены определяется компонентами, прикрепленными к GameObject [47]. Unity также развивает отдельное направление ECS, называя его дата-ориентированным фреймворком для высокопроизводительной обработки сущностей [48]. Однако Unity не является Java-решением и представляет собой крупную экосистему с собственным редактором, форматом проектов и моделью разработки. Для Java-разработчика, желающего изучить или использовать компонентную модель в небольшом приложении, такой инструмент может оказаться избыточным.

Компонентно-ориентированную архитектуру необходимо отличать от канонического Entity-Component-System. В строгом варианте ECS сущность обычно является только идентификатором, компонент хранит данные, а вся логика выполняется системами, которые выбирают сущности по набору компонентов. Такой подход хорошо подходит для высокопроизводительных

сцен, поскольку данные можно располагать в памяти компактно и обрабатывать пакетами. Но он требует более строгой организации данных и усложняет разработку для небольших проектов.

Разрабатываемая программная система ближе к компонентно-ориентированной модели с элементами ECS, чем к полностью каноническому дата-ориентированному ECS. В ней есть сущности, компоненты, системы и запросы к сущностям по составу компонентов. При этом компоненты являются объектами Java и могут иметь собственный жизненный цикл. Такой вариант сохраняет основную идею композиции объектов из компонентов, но остается более привычным для Java-разработчика, работающего с объектами, классами и методами.

Для ВКР выбран именно этот подход, так как он позволяет совместить простоту объектной модели Java с гибкостью компонентной структуры сцены. Разработчик получает возможность описывать интерактивные объекты через состав компонентов, не создавая сложную иерархию наследования и не переходя к полностью дата-ориентированной архитектуре.

1.4 Ключевые сущности предметной области

Несмотря на различие конкретных библиотек и движков, большинство средств построения интерактивных приложений оперирует схожими понятиями. Ключевой сущностью является сцена. Она задает контекст выполнения: содержит набор объектов, управляет их жизненным циклом, обновляет системы и определяет, какие объекты участвуют в текущем кадре. В играх сцена может соответствовать уровню, меню или отдельному режиму. В учебном симуляторе сцена может представлять лабораторную установку, эксперимент или демонстрационный пример.

Сущность представляет отдельный объект сцены. Сама по себе сущность может не содержать логики и данных предметной области. Ее назначение состоит в том, чтобы объединять компоненты и выступать единицей идентификации. Такой подход позволяет рассматривать любой объект сце-

ны одинаково: камеру, куб, источник света, физическое тело или служебный объект управления.

Компонент описывает отдельную характеристику сущности. Наиболее универсальным компонентом является трансформация, задающая положение, поворот и масштаб объекта. Без нее невозможно определить, где находится модель, камера или коллайдер. Другие компоненты добавляют графическое представление, камеру, освещение, физическое тело, обработчик столкновений или пользовательское поведение.

Система выполняет обработку сущностей, имеющих определенный набор компонентов. Например, система рендеринга выбирает сущности с трансформацией и графическим компонентом, система скриптов вызывает пользовательское поведение, а физическая система работает с физическими телами и коллайдерами. Благодаря этому логика обработки не размазывается по всем объектам, а сосредотачивается в подсистемах, отвечающих за конкретную область.

Ресурсы приложения включают шейдеры, текстуры, материалы, меши и другие данные, используемые несколькими объектами. В интерактивных приложениях важно не только загрузить ресурс, но и контролировать срок его жизни. Графические ресурсы занимают память не только в оперативной памяти приложения, но и на стороне графического процессора, поэтому при смене сцены или завершении приложения их необходимо корректно освободить.

Камера определяет способ наблюдения сцены. В 3D-приложении она хранит параметры проекции, область видимости, соотношение сторон и положение в пространстве. Источники света влияют на расчет освещенности объектов. Материал и шейдер задают способ визуализации поверхности. Ввод пользователя связывает внешние действия с изменениями состояния сцены. Математическая основа работы камеры, трансформаций и ориентации объектов связана с использованием векторов, матриц и кватернионов [17, 23].

Связь этих сущностей можно описать так: приложение содержит активную сцену, сцена содержит сущности и системы, сущности состоят из компонентов, а системы обрабатывают подходящие сущности и используют ресурсы для формирования результата. Такая модель достаточно проста для понимания, но при этом позволяет строить разнообразные интерактивные приложения без создания отдельного класса для каждого вида объекта.

1.5 Особенности разработки интерактивных приложений на Java

Java традиционно используется для разработки серверных приложений, корпоративных систем, инструментов автоматизации, учебных программ и кроссплатформенных настольных приложений. Для интерактивной графики Java применяется реже, чем C++ или C#, однако у нее есть ряд свойств, полезных для рассматриваемой задачи. Язык имеет строгую типизацию, развитую стандартную библиотеку, автоматическое управление памятью и широкий набор средств разработки. Официальная документация Java SE 25 включает спецификации платформы, API и инструменты JDK, что делает платформу достаточно полно документированной для учебного и прикладного использования [34, 35, 36]. Практические рекомендации по проектированию Java-кода и использованию стандартных возможностей языка рассматриваются в работах Дж. Блоха, К. Хорстманна и Г. Шилдта [6, 7, 8].

Главное преимущество Java в контексте данной работы заключается в понятной объектной модели. Сущности, компоненты, системы, материалы, меши, камеры и ресурсы естественно выражаются через классы и интерфейсы. Это упрощает проектирование программной системы, ориентированной на разработчика, который уже знаком с Java и хочет создавать интерактивные сцены без перехода на другой язык и без изучения крупной игровой среды.

При этом Java не предоставляет встроенного доступа к OpenGL на уровне стандартной библиотеки. Для работы с аппаратно-ускоренной графикой нужны дополнительные библиотеки, такие как LWJGL. Использование LWJGL позволяет сохранить разработку на Java, но работать с графическим API, близким к нативному уровню. Такой подход требует большей аккуратности

сти: разработчик должен учитывать создание окна, контекста OpenGL, буферов, шейдеров, загрузку данных на графический процессор и освобождение ресурсов.

Еще одна особенность связана с производительностью. Интерактивное приложение должно обновлять состояние и отрисовывать кадры много раз в секунду. Поэтому неудачные архитектурные решения быстро становятся заметными: лишние выделения памяти, частый поиск объектов, неоптимальная сортировка команд рендеринга или избыточная работа с графическими ресурсами приводят к задержкам. В Java часть низкоуровневых деталей скрыта виртуальной машиной, но это не отменяет необходимости аккуратно строить цикл обновления и ограничивать лишние операции внутри него.

Для рассматриваемой программной системы важно, что математика трехмерной сцены реализуется в самом проекте. Векторные, матричные и кватернионные операции используются для трансформаций, камеры, ориентации объектов и расчета положения в пространстве. Такой подход увеличивает объем реализации, но соответствует базовым принципам построения трехмерной графики и преобразований объектов сцены [17, 19]. Такой подход делает систему более самостоятельной и позволяет подробно раскрыть проектные решения в техническом проекте и рабочем проекте.

Отсутствие готового сборщика на раннем этапе разработки не влияет на анализ предметной области, но отражает текущий уровень зрелости программной системы. Для дальнейшего развития проекта целесообразно добавить Maven или Gradle, чтобы упростить подключение зависимостей, запуск тестов и воспроизводимую сборку. В рамках первой главы этот вопрос важен как подтверждение общей проблемы: Java-разработчику нужны не только низкоуровневые библиотеки, но и удобная, воспроизводимая инфраструктура создания интерактивных приложений.

1.6 Анализ существующих программных решений

Для сравнения выбраны решения, близкие к теме работы по разным признакам. LWJGL представляет низкоуровневый доступ к графическим и

системным API из Java. libGDX является Java-фреймворком для кроссплатформенной разработки. jMonkeyEngine представляет полноценный Java-движок. Processing отражает подход быстрого визуального прототипирования на Java-подобной основе. Unity включен в анализ не как Java-решение, а как распространенный пример компонентной модели и практики построения интерактивных сцен.

Краткое сравнение программных средств построения интерактивных приложений приведено в таблице 1.1.

Таблица 1.1 – Сравнение программных средств построения интерактивных приложений

Средство	Тип и платформа	Возможности	Особенности для Java-разработчика
LWJGL	Низкоуровневая Java-библиотека, использующая нативные привязки	Доступ к OpenGL, OpenAL и другим системным API; построение 2D- и 3D-графики возможно через OpenGL; собственного редактора нет	Дает высокий уровень контроля над графикой и окружением выполнения, но не предоставляет готовую сцену, компонентную модель, управление ресурсами и жизненный цикл объектов.
libGDX	Java-фреймворк для разработки игр и графических приложений	Поддерживает 2D- и 3D-графику, работу с ресурсами, вводом и несколькими целевыми платформами; обязательного редактора нет	Удобен для кроссплатформенной разработки, однако архитектура игровых объектов и способ организации логики во многом остаются ответственностью разработчика.

Продолжение таблицы 1.1

Средство	Тип и платформа	Возможности	Особенности для Java-разработчика
jMonkeyEngine	Java-движок для создания трехмерных приложений	Ориентирован преимущественно на 3D-графику, содержит готовые подсистемы сцены, материалов, освещения, физики и ресурсов; имеются дополнительные инструменты разработки	Близок к полноценному решению на Java, но требует изучения крупного фреймворка и принятия его проектной модели, что может быть избыточно для небольших учебных и исследовательских проектов.
Processing	Среда и библиотека для визуального программирования на Java-подобной основе	Позволяет быстро создавать 2D- и 3D-скетчи, визуализации и учебные интерактивные примеры; имеет собственную среду разработки	Имеет низкий порог входа, но больше ориентирован на визуальные эксперименты и прототипирование, чем на построение расширяемой компонентной архитектуры.
Unity	Крупный игровой движок и редактор, использующий язык C#	Поддерживает 2D- и 3D-графику, физику, визуальный редактор сцен, компонентную модель объектов и развитую экосистему инструментов	Показывает преимущества компонентного подхода, но не относится к Java-экосистеме и может быть слишком тяжелым решением для небольших проектов, где требуется простая программная основа.

Из сравнения видно, что существующие решения закрывают разные части задачи. LWJGL удобен как низкоуровневая основа, но не решает архитектурные вопросы. libGDX снижает объем технической работы и дает готовый фреймворк, но не ставит компонентную модель в центр архитектуры. jMonkeyEngine наиболее близок к полноценному движку на Java, однако его использование предполагает освоение более крупной системы. Processing удобен для быстрых визуальных экспериментов, но не ориентирован на построение расширяемой сценной архитектуры. Unity показывает преимущества компонентного подхода, но относится к другой технологической экосистеме.

Для небольшого Java-проекта возникает разрыв между двумя крайностями. С одной стороны, можно взять низкоуровневую библиотеку и написать всю архитектуру самостоятельно. С другой стороны, можно использовать полноценный движок, приняв его сложность и набор инструментов. В рамках ВКР интерес представляет промежуточное решение: программная система, которая предоставляет компонентную организацию сцены, базовые системы рендеринга, ввода, скриптов и обработки столкновений, но остается достаточно легкой для изучения и модификации.

1.7 Выводы по анализу предметной области

Интерактивные приложения отличаются от обычных прикладных программ наличием постоянного цикла обработки ввода, обновления состояния и вывода кадра. Для их разработки требуется не только реализовать предметную логику, но и создать инфраструктуру сцены, ресурсов, объектов, компонентов, рендеринга и пользовательского ввода.

Анализ существующих решений показал, что Java-разработчик может использовать низкоуровневые библиотеки, фреймворки или полноценные движки. Однако низкоуровневые библиотеки требуют самостоятельной разработки архитектуры, а крупные движки часто оказываются избыточными для небольших учебных, демонстрационных и исследовательских проектов. Это создает потребность в промежуточной программной системе, которая за-

крывает основные инфраструктурные задачи, но остается простой для изучения и расширения.

Компонентно-ориентированный подход позволяет отказаться от сложных иерархий наследования и описывать объекты сцены как набор независимых компонентов. Для выбранной темы это особенно важно, поскольку объекты интерактивной сцены отличаются не типом класса, а набором возможностей: одни отображаются, другие участвуют в столкновениях, третьи выполняют пользовательскую логику или задают камеру.

Разрабатываемая программная система должна быть ориентирована на Java-разработчика и предоставлять базовый набор средств для построения интерактивных сцен: управление сценой, сущностями, компонентами и системами, рендеринг, обработку ввода, скрипты поведения и базовую обработку столкновений. Такой результат соответствует утвержденной теме ВКР и создает основу для дальнейшего развития системы до более полноценного игрового или интерактивного движка.

2 Техническое задание

2.1 Основание для разработки

Полное наименование системы: «Разработка компонентно-ориентированной программной системы для построения интерактивных приложений на Java».

Основанием для разработки программы является задание на выпускную квалификационную работу бакалавра по направлению подготовки 09.03.04 «Программная инженерия». Разрабатываемая программная система представляет собой программную основу для создания интерактивных приложений на языке Java. В дальнейшем по мере расширения подсистем, средств разработки и пользовательской документации данная система может быть развита до более полного игрового движка, однако в рамках настоящей работы основной акцент сделан на ядре, компонентной модели, сценах, системах обработки, визуализации, вводе и базовой физике.

Разработка ведется с учетом требований к стадиям создания программной документации. В разделе технического задания необходимо определить назначение разработки, сформулировать задачи, описать требования к программной системе и показать порядок ее использования разработчиком интерактивного приложения.

2.2 Цель и назначение разработки

Целью разработки является создание компонентно-ориентированной программной системы, позволяющей Java-разработчику строить простые интерактивные приложения и учебные игровые прототипы без самостоятельной реализации полного набора низкоуровневых механизмов: создания окна, основного цикла приложения, управления сценой, хранения объектов, обработки пользовательского ввода, визуализации трехмерных объектов и базовой обработки физических тел.

Функциональное назначение системы состоит в предоставлении разработчику набора классов и подсистем, с помощью которых можно описывать

интерактивную сцену как совокупность сущностей, компонентов и систем обработки. Сущность используется как контейнер компонентов, компонент хранит состояние или поведение объекта, а система выполняет обработку групп сущностей, удовлетворяющих заданным условиям. Такой подход позволяет отделить общие механизмы работы приложения от прикладной логики конкретной сцены.

Эксплуатационное назначение системы заключается в использовании ее Java-разработчиком при создании настольных интерактивных приложений. Пользователь разрабатываемой системы не является конечным пользователем готовой игры или демонстрации. В данном случае пользователем выступает программист, который подключает исходные коды движка, создает класс приложения, описывает сцену, добавляет объекты, настраивает компоненты и запускает приложение в среде разработки или из командной строки.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Разработать ядро компонентно-ориентированной модели, включающее сущности, компоненты, системы обработки, запросы сущностей и жизненный цикл объектов сцены.
2. Спроектировать и реализовать подсистему управления приложением и сценами, обеспечивающую создание окна, выполнение основного цикла, обработку времени кадра и безопасное изменение состава сцены во время работы программы.
3. Реализовать графическую подсистему на основе OpenGL, обеспечивающую отображение объектов сцены с использованием мешей, материалов, текстур, камер и направленного освещения.
4. Реализовать подсистемы пользовательского ввода, скриптов и базовой физики, позволяющие обрабатывать действия пользователя, задавать поведение объектов средствами Java-кода и демонстрировать взаимодействие физических тел и коллайдеров.
5. Подготовить набор демонстрационных сцен и описание порядка использования программной системы с примерами кода, показывающими со-

здание приложения, сцены, сущностей, компонентов, систем обработки и пользовательских скриптов.

2.3 Требования к программной системе

Разрабатываемая программная система должна предоставлять Java-разработчику средства для создания интерактивного приложения программным способом. В системе не предусматривается визуальный редактор сцен, поэтому создание объектов, настройка компонентов, подключение систем и запуск сцены выполняются в исходном коде приложения.

Основным сценарием использования системы является разработка интерактивной сцены. Разработчик создает объект приложения, задает параметры окружения, формирует сцену, добавляет в нее системы обработки, затем создает сущности с компонентами и запускает основной цикл выполнения. После запуска система самостоятельно выполняет обновление активной сцены, обработку ввода, вызов пользовательских скриптов, физическое обновление, рендеринг и освобождение ресурсов при завершении работы.

2.3.1 Требования к архитектурной модели

Архитектурная модель системы должна основываться на компонентно-ориентированном подходе. В системе должны использоваться следующие основные понятия:

- сущность – объект сцены, имеющий уникальный идентификатор и набор компонентов;
- компонент – часть состояния или поведения сущности;
- система – объект, выполняющий обработку сущностей, имеющих требуемый набор компонентов;
- сцена – контейнер сущностей и систем, обновляемый в основном цикле приложения;
- контекст движка – объект, объединяющий окно, ввод, менеджер ресурсов, менеджер сцен, конфигурацию и время выполнения.

Компонентная модель должна позволять добавлять к сущности несколько независимых возможностей. Например, объект сцены может одновременно иметь компонент трансформации, компонент отображения, физическое тело и коллайдер. При этом прикладная логика не должна требовать создания отдельного класса-наследника для каждого нового типа игрового объекта.

Система должна поддерживать жизненный цикл компонентов. После присоединения компонента к сущности должен вызываться этап инициализации связи с сущностью, при запуске сцены – этап начала работы, при остановке – этап остановки, при удалении – этап уничтожения. Это необходимо для корректной регистрации внутренних состояний, освобождения ресурсов и прекращения работы скриптов.

2.3.2 Требования к управлению приложением и сценами

Программная система должна обеспечивать создание окна приложения, настройку базовых параметров запуска и выполнение основного цикла. К базовым параметрам относятся цвет очистки окна, масштаб времени, фиксированный шаг обновления, автоматическое обновление соотношения сторон камеры и режим отладочной проверки ошибок OpenGL.

Менеджер сцен должен обеспечивать загрузку активной сцены и замену текущей сцены во время выполнения. Смена сцены должна выполняться безопасно, без нарушения текущего цикла обновления. Это важно для интерактивных приложений, в которых сцена может быть перезапущена по действию пользователя, например при нажатии клавиши сброса демонстрации.

Сцена должна поддерживать добавление и удаление сущностей, добавление и удаление систем обработки, выполнение цикла обновления и освобождение ресурсов. Изменение структуры сцены во время обновления должно откладываться до безопасного момента, чтобы добавление или удаление объектов из пользовательского скрипта не приводило к ошибкам обхода коллекций.

2.3.3 Требования к графической подсистеме

Графическая подсистема должна использовать OpenGL и обеспечивать отображение трехмерных объектов в окне приложения. Для типового объекта сцены необходимо поддерживать компонент трансформации, компонент отображения меша и материал. Меш должен задавать геометрию объекта, материал – параметры отображения, а трансформация – положение, поворот и масштаб в сцене.

В составе системы должны быть предусмотрены стандартные геометрические примитивы, необходимые для демонстрационных сцен. К таким примитивам относятся куб, плоскость или четырехугольник, а также простые формы, позволяющие быстро собрать тестовую сцену без загрузки модели из внешнего файла. Загрузка мешей из внешних файлов в рамках текущей версии системы не является обязательной возможностью.

Подсистема ресурсов должна обеспечивать загрузку и повторное использование шейдеров и текстур. Текстуры должны загружаться из файлов ресурсов с использованием средств LWJGL STB. Повторная загрузка уже использованного ресурса должна избегаться за счет кэширования, так как в интерактивных приложениях один и тот же шейдер или одна и та же текстура могут использоваться многими объектами.

Для освещения должна быть предусмотрена поддержка базового направленного источника света. Такая модель освещения достаточна для демонстрации объемных объектов, положения камеры и работы материалов, но не рассматривается как полноценная система освещения с разными типами источников, тенями и постобработкой.

2.3.4 Требования к пользовательскому вводу

Программная система должна обеспечивать получение состояния клавиатуры и мыши. Разработчик должен иметь возможность проверять факт удержания клавиши, однократное нажатие клавиши, положение курсора и изменение положения курсора между кадрами. Эти данные должны быть до-

ступны из пользовательских скриптов, поскольку именно скрипты отвечают за прикладное поведение объектов сцены.

Для трехмерных демонстрационных сцен должно быть предусмотрено управление камерой. Перемещение камеры должно выполняться с помощью клавиш W, A, S, D, SPACE и LEFT_SHIFT; поворот камеры должен выполняться по смещению мыши. Такой способ управления является привычным для интерактивных трехмерных приложений и позволяет свободно осматривать сцену. Для двумерной интерактивной сцены должно быть предусмотрено получение положения курсора, преобразование координат мыши в координаты мира и обработка нажатий кнопок мыши.

2.3.5 Требования к системе пользовательских скриптов

Система должна предоставлять базовый класс сценария поведения. Пользовательский скрипт должен подключаться к сущности как обычный компонент. При запуске сцены система скриптов должна находить такие компоненты и вызывать методы их жизненного цикла.

В пользовательском скрипте разработчик должен иметь возможность получать компоненты текущей сущности, читать пользовательский ввод, изменять трансформацию объекта, прикладывать силы или импульсы к физическому телу, а также обращаться к контексту движка для выполнения действий уровня приложения. Например, в одной из демонстрационных сцен скрипт может менять направление движения кинематической платформы или перезапускать сцену при нажатии клавиши.

2.3.6 Требования к базовой физической подсистеме

Физическая подсистема должна обеспечивать базовую обработку физических тел и столкновений. В системе должны поддерживаться динамические, статические и кинематические тела. Динамические тела должны реагировать на гравитацию, скорость, силы и импульсы. Статические тела должны использоваться как неподвижное окружение. Кинематические тела должны

перемещаться заданным разработчиком способом и участвовать во взаимодействии с динамическими объектами.

Для проверки столкновений должен использоваться компонент коллайдера. В текущей версии системы основным типом коллайдера является коробочный коллайдер. Он подходит для демонстрации падения кубов, взаимодействия с полом, стенами и движущейся платформой. Требование к физической подсистеме не включает моделирование сложных форм, мягких тел, суставов, транспортных средств и других возможностей полноценных физических движков.

Физическая система должна выполняться после пользовательских скриптов и до рендеринга. Такой порядок позволяет сначала обработать ввод и пользовательскую логику, затем обновить физическое состояние объектов, а после этого отобразить уже измененную сцену.

2.3.7 Требования к демонстрационным сценам

В составе программной системы должен быть подготовлен набор демонстрационных сцен, показывающих основные возможности движка в нескольких вариантах использования. Каждая демонстрационная сцена должна запускаться как самостоятельный Java-класс с методом `main` и использовать публичный программный интерфейс движка для создания приложения, сцены, сущностей, компонентов и систем обработки.

Набор демонстрационных сцен должен включать:

- `ShowcaseScene` – сцену для проверки базового трехмерного рендеринга, работы камеры, направленного освещения, иерархии трансформаций, слоев отображения, отсечения невидимых объектов и отладочных направляющих;

- `Interactive2DScene` – сцену для проверки ортографической камеры, обработки положения мыши, интерактивных объектов, изменения визуального состояния объектов и пользовательских скриптов двумерного интерфейсного типа;

- `TexturedLightingScene` – сцену для проверки материалов с текстурами, процедурного формирования текстур, передачи текстурных данных в OpenGL, направленного освещения и изменения параметров света во время выполнения;

- `PhysicsScene` – сцену для проверки физической подсистемы, свободной камеры, статических, динамических и кинематических тел, коробочных коллайдеров, отладочной визуализации коллайдеров и перезапуска сцены по клавише.

Основным демонстрационным примером для проверки физической подсистемы должна выступать сцена `PhysicsScene`. Она должна содержать статические объекты окружения, динамические физические объекты, падающие под действием гравитации, кинематическую платформу, отладочную сетку и визуализацию коллайдеров. Остальные демонстрационные сцены должны показывать, что архитектура системы не ограничивается физической симуляцией и может использоваться для разных типов интерактивных приложений.

2.4 Руководство по использованию программной системы

Данный подраздел описывает порядок работы с разрабатываемой программной системой с точки зрения Java-разработчика. В отличие от конечного пользовательского приложения, движок не предоставляет готовый визуальный интерфейс для создания сцен. Все действия выполняются в исходном коде: разработчик создает приложение, описывает сцену, добавляет объекты и компоненты, после чего запускает программу.

2.4.1 Подготовка окружения

Для работы с программной системой необходимо установить JDK версии 25. В качестве внешней библиотеки используется LWJGL версии 3.3.6. В состав подключаемых модулей LWJGL должны входить:

- `lwjgl` – базовый модуль LWJGL;
- `lwjgl-glfw` – модуль для создания окна, OpenGL-контекста и обработки ввода;

- lwjgl-opengl – модуль для доступа к функциям OpenGL;
- lwjgl-stb – модуль для загрузки изображений и текстур.

Проект не использует Maven или Gradle. Поэтому разработчик может работать с ним двумя способами. Первый способ заключается в открытии проекта в IntelliJ IDEA и ручном добавлении JAR-файлов LWJGL в зависимости проекта. Второй способ заключается в ручной компиляции исходных файлов с помощью `javac` и запуске нужного класса через `java`.

При настройке проекта через IntelliJ IDEA необходимо выполнить следующие действия:

1. Открыть каталог проекта как обычный Java-проект.
2. Указать установленный JDK 25 в качестве SDK проекта.
3. Добавить JAR-файлы LWJGL 3.3.6 и JAR-файлы нативных библиотек для целевой операционной системы в раздел зависимостей проекта.
4. Убедиться, что рабочим каталогом запуска является корневой каталог проекта, в котором расположен каталог `res`.
5. Создать конфигурации запуска для одного или нескольких демонстрационных классов: `common.showcase.ShowcaseScene`, `common.showcase.Interactive2DScene`, `common.showcase.TexturedLightingScene`, `common.showcase.PhysicsScene`.

Для ручной компиляции можно использовать следующий общий порядок команд. Команды предполагают, что все JAR-файлы LWJGL помещены в каталог `lib`, а компиляция выполняется из корня проекта. В примерах ниже запускается сцена `PhysicsScene`; для запуска других демонстрационных сцен достаточно заменить имя запускаемого класса на `ShowcaseScene`, `Interactive2DScene` или `TexturedLightingScene` из пакета `common.showcase`.

Листинг 1 – Пример ручной компиляции проекта в Windows

```
1 dir /s /b src\*.java test\*.java > sources.txt
2 javac -encoding UTF-8 -d out -cp "lib/*" @sources.txt
3 java -cp "out;lib/*" common.showcase.PhysicsScene
```

Для Linux и macOS отличается только разделитель элементов `classpath`. Вместо точки с запятой используется двоеточие.

Листинг 2 – Пример ручной компиляции проекта в Linux и macOS

```
1 find src test -name "*.java" > sources.txt
2 javac -encoding UTF-8 -d out -cp "lib/*" @sources.txt
3 java -cp "out:lib/*" common.showcase.PhysicsScene
```

Если нативные библиотеки LWJGL не находятся автоматически через JAR-файлы, их необходимо указать через параметр `java.library.path`. Однако при использовании набора библиотек, полученного через конфигурацию LWJGL вместе с `native JAR` для соответствующей операционной системы, обычно достаточно добавить все JAR-файлы в `classpath`.

2.4.2 Создание приложения

Точкой входа в интерактивное приложение является обычный метод `main`. В нем создается конфигурация движка, экземпляр приложения, загружается стартовая сцена и запускается основной цикл. Пример минимальной структуры запуска приведен в листинге 2.3.

Листинг 3 – Создание приложения и запуск сцены

```
1 public static void main(String[] args) {
2     EngineConfig config = new EngineConfig()
3         .clearColor(0.05f, 0.07f, 0.08f, 1f)
4         .autoUpdateCameraAspect(true)
5         .debugOpenGLErrors(false);
6
7     Application app = new Application("interactive scene", config);
8     app.getWindow().setCursorDisabled();
9     app.context().sceneManager().load(createScene());
10    app.run();
11 }
```

В данном примере объект `EngineConfig` определяет параметры запуска. Метод `clearColor` задает цвет очистки окна перед отрисовкой кадра. Параметр `autoUpdateCameraAspect` включает автоматическое обновление соотношения сторон камеры при изменении размеров окна. Параметр `debugOpenGLErrors` позволяет отключить или включить дополнительные проверки ошибок OpenGL.

После создания приложения в трехмерных демонстрационных сценах может отключаться отображение курсора, так как они используют свободное управление камерой мышью. В двумерной интерактивной сцене курсор, на-

оборот, остается видимым для обработки кликов по объектам. Затем через менеджер сцен загружается сцена, созданная методом `createScene`, и вызывается метод `run`. После этого управление переходит основному циклу приложения.

2.4.3 Создание сцены и добавление систем

Сцена создается как экземпляр класса `Scene`. До добавления сущностей в нее необходимо подключить системы обработки. Порядок подключения систем важен, потому что каждая система выполняется в порядке добавления. В листинге ниже приведен пример настройки систем для физической демонстрационной сцены.

Листинг 4 – Создание сцены и подключение систем

```
1 private static Scene createScene() {  
2     Scene scene = new Scene();  
3  
4     scene.addSystem(new ScriptSystem());  
5  
6     scene.addSystem(new PhysicsSystem()  
7         .gravity(0f, -9.81f, 0f)  
8         .fixedTimeStep(1f / 60f)  
9         .maxSubSteps(5)  
10        .velocityIterations(12)  
11        .positionIterations(10)  
12        .debugDrawColliders(true));  
13  
14     scene.addSystem(new RenderSystem()  
15         .ambientColor(new Vec4(0.18f, 0.20f, 0.23f, 1f)));  
16  
17     scene.addSystem(new DebugRenderSystem());  
18  
19     return scene;  
20 }
```

Сначала добавляется `ScriptSystem`, поскольку пользовательские скрипты могут изменять состояние объектов, читать ввод, прикладывать силы или запрашивать смену сцены. После этого добавляется `PhysicsSystem`, которая обновляет физические тела и коллайдеры. Затем выполняется `RenderSystem`, получающая уже обновленные трансформации объектов. Последней добавляется `DebugRenderSystem`, отвечающая за вывод отладочной геометрии.

2.4.4 Добавление источника света

Для базового освещения сцены используется компонент DirectionalLight. Он задает направление света, цвет и интенсивность. Такой источник света не имеет положения в пространстве и имитирует удаленный источник, например солнце.

Листинг 5 – Создание направленного источника света

```
1 scene.addEntity(new Entity("sun")
2     .addComponent(new DirectionalLight()
3         .direction(-0.4f, -1f, -0.25f)
4         .color(new Vec4(1f, 0.94f, 0.82f, 1f))
5         .intensity(1.2f)));
```

Свет оформляется как обычная сущность с компонентом. Такой подход соответствует общей компонентной модели: даже служебные объекты сцены могут быть представлены через сущности и компоненты.

2.4.5 Создание камеры

Для отображения сцены необходима сущность с компонентом Camera. Чтобы камера имела положение и поворот, к сущности добавляется компонент Transform. Для управления камерой используется скрипт FreecamController.

Листинг 6 – Создание свободной камеры

```
1 scene.addEntity(new Entity("camera")
2     .addComponent(new Transform()
3         .position(0f, 4f, 9f)
4         .rotation(Quaternion.euler(-22f, 0f, 0f)))
5     .addComponent(new Camera()
6         .fov(60f)
7         .near(0.1f)
8         .far(80f)
9         .cullingMask(LAYER_WORLD | LAYER_PHYSICS)
10        .priority(10))
11    .addComponent(new FreecamController()
12        .speed(7f)
13        .sensitivity(18f)));
```

Компонент Camera задает угол обзора, ближнюю и дальнюю плоскости отсечения, маску видимости и приоритет. Приоритет нужен в случае, если в сцене присутствует несколько камер. Скрипт FreecamController считывает ввод пользователя и изменяет трансформацию камеры.

2.4.6 Создание статического объекта

Статический объект окружения можно создать без компонента Rigidbody. Для пола физической демонстрационной сцены достаточно задать трансформацию, коробочный коллайдер и компонент отображения меша.

Листинг 7 – Создание пола сцены

```
1 scene.addEntity(new Entity("floor")
2   .addComponent(new Transform()
3     .position(0f, -0.1f, 0f)
4     .scale(8f, 0.2f, 8f))
5   .addComponent(new BoxCollider()
6     .halfExtents(0.5f, 0.5f, 0.5f)
7     .friction(1f)
8     .restitution(0f))
9   .addComponent(new MeshRenderer()
10    .mesh(Meshes.cube())
11    .material(new Material()
12      .color(new Vec4(0.18f, 0.24f, 0.22f, 1f)))
13    .layer(LAYER_WORLD)
14    .boundsRadius(6f)));
```

В данном примере куб используется как масштабируемый параллелепипед. Компонент Transform растягивает его до размеров пола. Компонент BoxCollider задает область столкновения. Компонент MeshRenderer указывает геометрию, материал, слой отображения и радиус границ, используемый при обработке видимости.

2.4.7 Создание динамического физического тела

Динамический объект отличается от статического тем, что к нему добавляется компонент Rigidbody. Этот компонент хранит тип тела, массу, скорость, демпфирование, настройки гравитации и другие физические параметры.

Листинг 8 – Создание динамического куба

```
1 scene.addEntity(new Entity("falling-box")
2   .addComponent(new Transform()
3     .position(0f, 5f, 0f))
4   .addComponent(new Rigidbody()
5     .mass(1f)
6     .velocity(1f, 0f, 0f)
7     .linearDamping(0.05f)
8     .angularDamping(0.08f))
```

```

9      .addComponent(new BoxCollider()
10         .halfExtents(0.5f, 0.5f, 0.5f)
11         .friction(0.9f)
12         .restitution(0.1f))
13      .addComponent(new MeshRenderer()
14         .mesh(Meshes.cube())
15         .material(new Material()
16            .color(new Vec4(0.25f, 0.68f, 1f, 1f)))
17         .layer(LAYER_PHYSICS)
18         .boundsRadius(1f)));

```

После запуска сцены физическая система будет применять к такому объекту гравитацию, учитывать скорость, выполнять проверку столкновений с коллайдерами и изменять компонент Transform. Рендеринг будет использовать уже обновленное положение объекта.

2.4.8 Создание кинематического объекта

Кинематическое тело не падает под действием гравитации, но может перемещаться с заданной скоростью и взаимодействовать с динамическими телами. Такой объект удобно использовать для движущихся платформ и элементов окружения.

Листинг 9 – Создание кинематической платформы

```

1 scene.addEntity(new Entity("kinematic-platform")
2     .addComponent(new Transform()
3         .position(0f, 0.35f, 3.1f)
4         .scale(1.8f, 0.25f, 0.7f))
5     .addComponent(new Rigidbody()
6         .type(PhysicsBodyType.KINEMATIC)
7         .velocity(0.9f, 0f, 0f)
8         .useGravity(false))
9     .addComponent(new BoxCollider()
10        .halfExtents(0.5f, 0.5f, 0.5f)
11        .friction(0.9f))
12    .addComponent(new MeshRenderer()
13        .mesh(Meshes.cube())
14        .material(new Material()
15            .color(new Vec4(1f, 0.9f, 0.28f, 1f)))
16        .layer(LAYER_PHYSICS)
17        .boundsRadius(2f))
18    .addComponent(new PlatformBounceScript(-2.4f, 2.4f)));

```

В конце к сущности добавляется пользовательский скрипт PlatformBounceScript. Он следит за положением платформы и меняет направление скорости при достижении границ движения.

2.4.9 Создание пользовательского скрипта

Пользовательский скрипт создается наследованием от базового класса Script. В методе init обычно выполняется получение ссылок на компоненты сущности, а в методе loop описывается поведение, выполняемое каждый кадр.

Листинг 10 – Пример пользовательского скрипта движения платформы

```
1 private static final class PlatformBounceScript extends Script {
2     private final float minX;
3     private final float maxX;
4     private Rigidbody body;
5     private Transform transform;
6
7     private PlatformBounceScript(float minX, float maxX) {
8         this.minX = minX;
9         this.maxX = maxX;
10    }
11
12    @Override
13    public void init() {
14        body = entity().getComponent(Rigidbody.class);
15        transform = entity().getComponent(Transform.class);
16    }
17
18    @Override
19    public void loop(float dt) {
20        if (body == null || transform == null) return;
21
22        Vec3 position = transform.position();
23        Vec3 velocity = body.velocity();
24
25        if ((position.x <= minX && velocity.x < 0f) ||
26            (position.x >= maxX && velocity.x > 0f)) {
27            body.velocity(-velocity.x, velocity.y, velocity.z);
28        }
29    }
30 }
```

Данный пример показывает типовой способ расширения поведения объектов. Скрипт не изменяет устройство движка, не требует создания новой системы и работает как компонент конкретной сущности. Благодаря этому прикладная логика остается отделенной от ядра программной системы.

2.4.10 Использование пользовательского ввода

Для чтения пользовательского ввода из скриптов используется класс Input. Например, в физической демонстрационной сцене клавиша R может использоваться для перезапуска сцены.

Листинг 11 – Пример обработки нажатия клавиши

```
1 private static final class PhysicsSceneControlScript extends Script {  
2     @Override  
3     public void loop(float dt) {  
4         if (!Input.GetKeyDown(KeyCode.R)) return;  
5  
6         EngineContext context = EngineContext.current();  
7         context.window().setCursorDisabled();  
8         context.sceneManager().requestReplace(createScene());  
9     }  
10 }
```

В этом примере используется однократное нажатие клавиши. При нажатии R получается текущий контекст движка, скрывается курсор и запрашивается замена активной сцены. Замена выполняется не немедленно, а через менеджер сцен, что предотвращает изменение структуры приложения в небезопасный момент цикла обновления.

2.4.11 Создание отладочной геометрии

Для наглядной проверки работы сцены может использоваться отладочный рендеринг. Например, можно каждый кадр выводить сетку на плоскости, чтобы лучше видеть движение физических объектов.

Листинг 12 – Пример вывода отладочной сетки

```
1 private static final class PhysicsGridScript extends Script {  
2     @Override  
3     public void loop(float dt) {  
4         for (int i = -4; i <= 4; i++) {  
5             DebugRenderer.line(  
6                 new Vec3(i, 0f, -4f),  
7                 new Vec3(i, 0f, 4f),  
8                 new Vec4(0.18f, 0.25f, 0.28f, 1f));  
9  
10            DebugRenderer.line(  
11                new Vec3(-4f, 0f, i),  
12                new Vec3(4f, 0f, i),  
13                new Vec4(0.18f, 0.25f, 0.28f, 1f));  
14        }  
15    }
```


Отладочная геометрия не является частью постоянной модели сцены. Она создается на кадр и используется для проверки положения объектов, границ коллайдеров, ориентации сцены и поведения физической подсистемы.

2.4.12 Общий порядок создания сцены

С учетом приведенных примеров общий порядок разработки интерактивной сцены выглядит следующим образом:

1. Создается конфигурация движка и экземпляр приложения.
2. Создается объект сцены.
3. В сцену добавляются системы обработки в требуемом порядке.
4. При необходимости в сцену добавляется источник света.
5. В сцену добавляется камера: перспективная для трехмерной сцены или ортографическая для двумерной сцены.
6. Создаются сущности окружения, интерактивные объекты или физические тела в зависимости от назначения сцены.
7. При необходимости добавляются пользовательские скрипты поведения.
8. Сцена загружается через менеджер сцен.
9. Запускается основной цикл приложения.

Такой порядок хорошо подходит для учебных и демонстрационных проектов, потому что вся сцена описывается в одном или нескольких Java-классах. Разработчик сразу видит, какие системы включены в сцену, какие сущности созданы, какие компоненты назначены объектам и какой код отвечает за поведение.

2.5 Требования к программному интерфейсу

Программный интерфейс системы должен быть доступен из Java-кода и не должен требовать от разработчика прямой работы с низкоуровневыми вызовами OpenGL для типовых операций создания сцены. Разработчик должен

использовать готовые классы приложения, сцены, сущностей, компонентов, систем, материалов, мешей, текстур, ввода и времени.

Публичные классы должны поддерживать цепочечную настройку объектов, когда метод изменения параметра возвращает сам объект. Такой стиль используется при создании конфигурации, компонентов, материалов и систем. Он уменьшает объем вспомогательного кода и делает описание сцены более компактным.

Программный интерфейс должен позволять:

- создавать приложение с заданным заголовком окна и конфигурацией;
- загружать и заменять активную сцену;
- добавлять в сцену системы обработки;
- создавать сущности и назначать им компоненты;
- получать компоненты сущности по типу;
- создавать пользовательские скрипты поведения;
- получать состояние клавиатуры и мыши;
- загружать и использовать текстуры, материалы и шейдеры;
- создавать физические тела и коробочные коллайдеры;
- выводить отладочную геометрию при разработке сцены.

При этом интерфейс должен оставаться достаточно простым для учебного использования. Разработчик должен иметь возможность создать первую интерактивную сцену без предварительного изучения сложного визуального редактора, системы сборки, специализированного языка описания сцен или внешнего формата проекта.

2.6 Нефункциональные требования

2.6.1 Требования к программному обеспечению

Для разработки и запуска программной системы требуется JDK 25. В качестве внешней библиотеки используется LWJGL 3.3.6. Обязательными модулями LWJGL являются GLFW, OpenGL и STB. Система должна запускаться на настольных операционных системах Windows, Linux и macOS

при наличии соответствующих нативных библиотек LWJGL и графического драйвера.

Так как система использует OpenGL и шейдеры GLSL версии 330 core, целевая рабочая станция должна поддерживать OpenGL 3.3 Core Profile или более новую совместимую версию. При отсутствии подходящего графического драйвера корректная работа графической подсистемы не гарантируется.

2.6.2 Требования к аппаратному обеспечению

Аппаратные требования программной системы определяются требованиями JDK 25, LWJGL, используемых нативных библиотек и графического драйвера. Для работы демонстрационных сцен необходим персональный компьютер или ноутбук с поддержкой OpenGL 3.3 Core Profile, клавиатурой и мышью. Отдельные численные требования к объему оперативной памяти, производительности процессора и видеосистемы в рамках технического задания не задаются, поскольку демонстрационные сцены не ориентированы на предельную нагрузку и используются для проверки архитектурных возможностей движка.

2.6.3 Требования к надежности

Программная система должна предсказуемо обрабатывать некорректное использование основных объектов. При передаче недопустимых значений в конфигурацию, компоненты или системы должны возникать диагностируемые исключения. Например, не допускается создание сущности с пустым идентификатором, добавление пустого компонента, установка отрицательного фиксированного шага времени или установка неположительной массы физического тела.

Система должна корректно освобождать ресурсы при завершении приложения. После выхода из основного цикла должны быть уничтожены активные сцены, компоненты, системы, окно, графические ресурсы и контекст

выполнения. Это требование особенно важно для приложений, использующих нативные ресурсы OpenGL и LWJGL.

Изменение состава сцены во время обновления должно выполняться безопасно. Если сущность, компонент или система добавляются или удаляются из пользовательского скрипта, операция должна быть отложена до момента, когда изменение структуры сцены не нарушит обход систем и сущностей.

2.6.4 Требования к сопровождаемости

Исходный код программной системы должен иметь модульную структуру. Классы ядра компонентной модели, компоненты, графическая подсистема, скрипты, физическая подсистема и демонстрационные сцены должны быть разделены по пакетам. Такое разделение необходимо для того, чтобы разработчик мог изучать и изменять отдельные части движка без просмотра всего проекта сразу.

Демонстрационные сцены должны использовать публичный программный интерфейс движка и выступать как практические примеры его применения. Это позволяет использовать их не только для проверки работоспособности, но и как основу пользовательской документации.

2.6.5 Требования к безопасности

Разрабатываемая программная система является локальной программной основой и не предусматривает сетевого взаимодействия, хранения учетных записей, обработки персональных данных и разграничения пользовательских ролей. Поэтому требования информационной безопасности ограничиваются корректной обработкой внешних файлов ресурсов и предотвращением аварийного завершения при ошибках загрузки.

При работе с ресурсами должны быть диагностируемыми ошибки чтения файлов, компиляции шейдеров, загрузки текстур и создания графических объектов. Разработчик должен иметь возможность понять причину ошибки по сообщению исключения или отладочному выводу.

2.7 Требования к оформлению документации

Программная документация должна включать материалы, необходимые для понимания назначения, архитектуры, порядка сборки, запуска и использования программной системы. В составе пояснительной записки должны быть представлены анализ предметной области, техническое задание, технический проект и рабочий проект.

В техническом задании должны быть описаны назначение разработки, задачи, требования к программной системе, функциональные сценарии и порядок использования движка разработчиком. В техническом проекте должны быть раскрыты выбранные средства разработки, архитектура, структура пакетов, основные подсистемы и проектные решения. В рабочем проекте должны быть приведены спецификации основных классов, описание тестирования и результаты проверки работоспособности программной системы.

Текстовые материалы, рисунки, таблицы, листинги и графический материал должны оформляться в соответствии с требованиями, применяемыми к выпускным квалификационным работам по направлению подготовки 09.03.04 «Программная инженерия».

3 Технический проект

3.1 Общие сведения о программной системе

Разрабатываемая компонентно-ориентированная программная система предназначена для построения интерактивных приложений на языке Java. Система не является прикладной программой с фиксированным набором экранов и заранее заданной предметной областью. Ее основное назначение состоит в том, чтобы предоставить Java-разработчику программную основу для создания собственных сцен, объектов, компонентов, систем обработки и пользовательских сценариев поведения.

В рамках технического проекта программная система рассматривается как легковесный движок, ориентированный на программное создание интерактивных сцен. В составе разработки отсутствует визуальный редактор, поэтому основным интерфейсом взаимодействия с системой является Java API. Разработчик создает экземпляр приложения, задает конфигурацию, формирует сцену, регистрирует системы обработки и добавляет в сцену сущности с наборами компонентов. После запуска приложения основной цикл выполняет обновление активной сцены, обработку пользовательского ввода, выполнение скриптов, расчет базовой физики и отрисовку кадра.

Ключевая идея проектирования заключается в разделении объекта сцены и его функциональных признаков. Объект сцены представлен сущностью, а его состояние и поведение задаются компонентами. Например, объект камеры содержит компонент Transform, компонент Camera и скрипт свободного перемещения, а физический визуальный объект содержит Transform, Rigidbody, BoxCollider и MeshRenderer. Обработка сущностей выполняется специализированными системами: скриптовой, физической, графической и отладочной.

Такое устройство позволяет отказаться от глубокой иерархии классов для разных типов объектов. Вместо классов вида `extttPlayer`, `extttWall`, `extttPlatform` и `extttLightObject` используется сборка сущности из небольших независимых компонентов. В результате один и тот же компонент может переиспользоваться в разных объектах сцены, а новая функциональ-

ность добавляется без изменения базового класса сущности. Такое решение соответствует принципам разделения ответственности, слабой связанности, поддерживаемости и независимого развития частей программной системы [4, 5, 12, 14].

Общая архитектура разрабатываемой программной системы представлена на рисунке 3.1. При оформлении проектных диаграмм учитывались рекомендации по практическому применению UML [10, 11, 9].

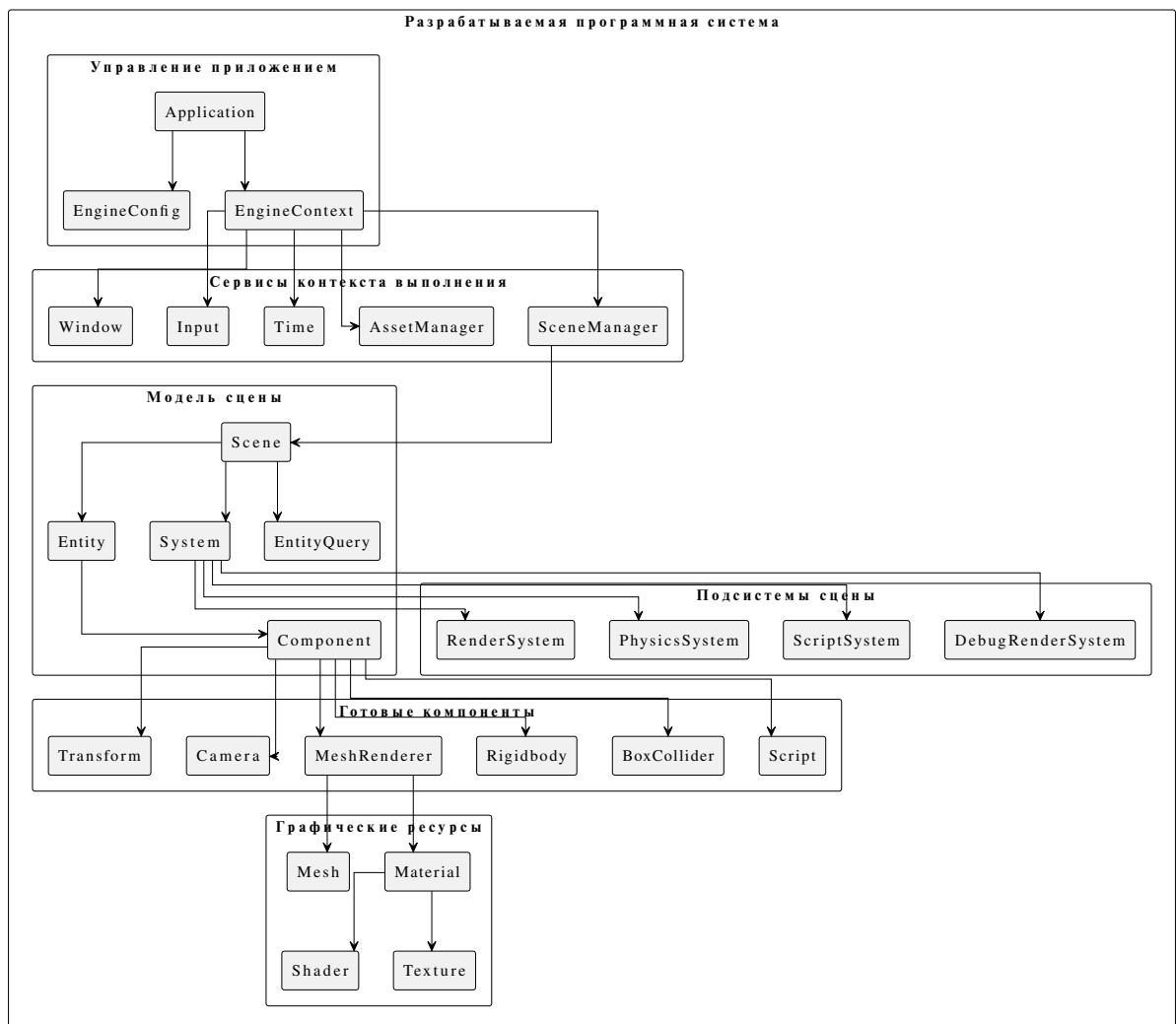


Рисунок 3.1 – Общая архитектура программной системы

В дальнейшем технический проект раскрывает не проект базы данных, так как база данных в системе не используется, а внутреннее устройство программной системы: структуру пакетов, принципы работы сцены, жизненный цикл приложения, устройство основных компонентов, графическую подси-

стему, подсистему ресурсов, скрипты, отладочную визуализацию и самостоятельно реализованную физическую подсистему.

3.2 Обоснование выбора технологий

Для реализации системы выбран набор технологий, который позволяет сохранить доступ к низкоуровневым возможностям графической подсистемы и при этом организовать архитектуру в привычной для Java-разработчика объектной форме. Основные технологии и их назначение приведены в таблице 3.1.

Таблица 3.1 – Используемые технологии и их назначение

Технология	Назначение	Причина использования
Java 25	основной язык реализации программной системы	обеспечивает объектно-ориентированную модель, строгую типизацию, кроссплатформенность на уровне JVM и удобство построения расширяемого API.
LWJGL 3.3.6	доступ из Java к нативным библиотекам	позволяет использовать OpenGL, GLFW и STB без перехода на C/C++, при этом не навязывает собственную архитектуру игрового движка.
OpenGL	графический API для вывода сцены	используется для создания GPU-ресурсов, передачи вершинных данных, настройки буферов и выполнения команд отрисовки.
GLSL	язык шейдерных программ	используется для описания вершинного и фрагментного шейдеров, расчета преобразований, текстурирования и базового освещения объектов.

Продолжение таблицы 3.1

Технология	Назначение	Причина использования
GLFW	создание окна, OpenGL-контекста и обработка событий ввода	позволяет работать с настольным окном приложения, клавиатурой, мышью, событиями изменения размера окна и состоянием курсора.
STB	загрузка изображений с диска	используется для загрузки текстур, применяемых материалами графической подсистемы.

3.2.1 Язык программирования Java

Java выбрана в качестве основного языка реализации, так как архитектура системы строится вокруг объектов, классов и интерфейсов, а сама платформа имеет подробные спецификации языка и виртуальной машины [34, 35, 36]. Архитектура системы опирается на классы, наследование, инкапсуляцию и обобщенные типы. Компоненты сцены, системы обработки, менеджеры ресурсов и пользовательские скрипты представлены обычными Java-классами. Это делает программный интерфейс понятным для разработчика, знакомого с объектно-ориентированным программированием.

Дополнительным преимуществом является наличие виртуальной машины JVM. Приложение может запускаться на разных операционных системах при наличии установленного JDK, нативных библиотек LWJGL для целевой платформы и графического драйвера с поддержкой требуемой версии OpenGL. Для разрабатываемой системы это важно, поскольку она предназначена не для одной демонстрационной сцены, а для создания разных интерактивных приложений на общей программной основе.

3.2.2 Библиотека LWJGL

LWJGL используется как низкоуровневый слой доступа к нативным библиотекам [37, 51, 52]. В рамках проекта задействованы GLFW, OpenGL и STB. При этом LWJGL не определяет структуру сцены, систему компонентов, жизненный цикл приложения, графическую очередь или физическую модель. Эти части реализуются в самой разрабатываемой системе.

Такое разделение удобно для проектирования. С одной стороны, система получает доступ к реальному графическому API и нативному окну. С другой стороны, прикладной разработчик не обязан напрямую работать с повторяющимся низкоуровневым кодом: создание окна, управление ресурсами, обновление сцены, отрисовка и обработка ввода вынесены в отдельные классы движка.

3.2.3 OpenGL и GLSL

Проектирование графической подсистемы выполняется с учетом модели OpenGL, спецификации версии 3.3 Core Profile, справочных материалов OpenGL Wiki и спецификации GLSL 3.30 [39, 40, 41, 42].

OpenGL применяется для вывода объектов сцены [39, 40, 41, 53, 54]. На уровне графической подсистемы создаются вершинные массивы, буферы вершин, индексные буферы и выполняются команды отрисовки. Геометрия объекта хранится в классе Mesh, а ее GPU-представление создается и кэшируется при первом использовании в рендеринге.

Шейдеры написаны на языке GLSL. Вершинный шейдер получает данные вершины и матрицы модели, вида и проекции. Фрагментный шейдер использует параметры материала, текстуру, цвет и данные направленного источника света. Такой набор достаточен для демонстрации трехмерной сцены с материалами, текстурами, камерой и базовым направленным освещением.

3.2.4 GLFW и STB

Подсистема окна и ввода опирается на GLFW, а загрузка изображений выполняется через STB [38, 43].

GLFW используется для создания окна приложения, настройки OpenGL-контекста, обработки событий клавиатуры, мыши, прокрутки и изменения размеров окна. Состояние ввода сохраняется в объекте, доступном через фасад Input, поэтому пользовательские скрипты могут работать с клавишами и мышью без прямого обращения к callback-функциям GLFW.

STB применяется для загрузки изображений с диска [43]. В текущей версии системы поддержана загрузка текстур, которые затем используются материалами. Загрузка трехмерных моделей из файлов не реализуется: геометрия задается программно, а для типовых случаев используются встроенные примитивы.

3.3 Внутреннее устройство программной системы

3.3.1 Структура пакетов проекта

Для описания структуры проекта и зависимостей между частями используются UML-диаграммы, подготовленные средствами PlantUML [50, 49].

Исходный код системы разделен на пакеты, каждый из которых отвечает за отдельную часть архитектуры. Такое разделение позволяет отделить ядро компонентной модели от графики, физики, управления приложением и пользовательских скриптов. Основные пакеты проекта приведены в таблице 3.2.

Таблица 3.2 – Основные пакеты проекта

Пакет	Назначение	Основные классы
common	математические и вспомогательные классы	Vec2, Vec3, Vec4, Mat4, Quaternion.

Продолжение таблицы 3.2

Пакет	Назначение	Основные классы
core	ядро компонентно-ориентированной модели	Entity, Component, System, EntityQuery, EntityObserver.
engine	управление приложением, сценами и ресурсами	Application, EngineContext, EngineConfig, Scene, SceneManager, AssetManager, Input, Time.
component	готовые компоненты объектов сцены	Transform, Camera, MeshRenderer, DirectionalLight, Script, Rigidbody, Collider, BoxCollider.
graphics	обертки над графическими ресурсами OpenGL	Window, Shader, Texture, MeshGpuResource.
system	системы обработки сцены	ScriptSystem, PhysicsSystem, RenderSystem, DebugRenderSystem.
script	переиспользуемые пользовательские скрипты	FreecamController.

Пакет core не зависит от OpenGL или LWJGL и содержит только общие понятия компонентной модели. Пакеты component и system строятся поверх ядра. Пакет engine объединяет инфраструктуру приложения, а пакет graphics содержит классы, непосредственно связанные с графическими ресурсами.

3.3.2 Основные элементы внутреннего устройства

Вместо проекта базы данных в данной работе рассматривается внутреннее устройство программной системы. Основные элементы системы приведены в таблице 3.3.

Таблица 3.3 – Основные элементы внутреннего устройства программной системы

Элемент	Роль в системе	Особенности реализации
Application	верхний управляющий объект приложения	создает окно, контекст выполнения, менеджеры ресурсов и сцен, запускает основной цикл.
EngineContext	контейнер общих сервисов	объединяет окно, ввод, время, менеджер ресурсов, менеджер сцен и конфигурацию.
Scene	центральный контейнер выполнения	хранит сущности, системы, запросы, индекс компонентов и очередь отложенных изменений.
Entity	объект сцены	представляет именованный контейнер компонентов, сам по себе не задает поведение объекта.
Component	признак или часть поведения сущности	имеет ссылку на сущность и методы жизненного цикла: присоединение, запуск, остановку и уничтожение.
System	обработчик сущностей сцены	выполняется в заданном порядке и получает доступ к сущностям через запросы.
EntityQuery	механизм выборки сущностей	позволяет системе работать только с сущностями, имеющими требуемый набор компонентов.
AssetManager	менеджер графических ресурсов	кэширует шейдеры и текстуры, освобождает созданные ресурсы при завершении работы.

Ключевыми элементами внутреннего устройства являются Scene, Entity, Component и System. Именно они определяют способ построения интерактивных приложений на базе разработанной системы. Диаграмма классов ядра компонентной модели представлена на рисунке 3.2.

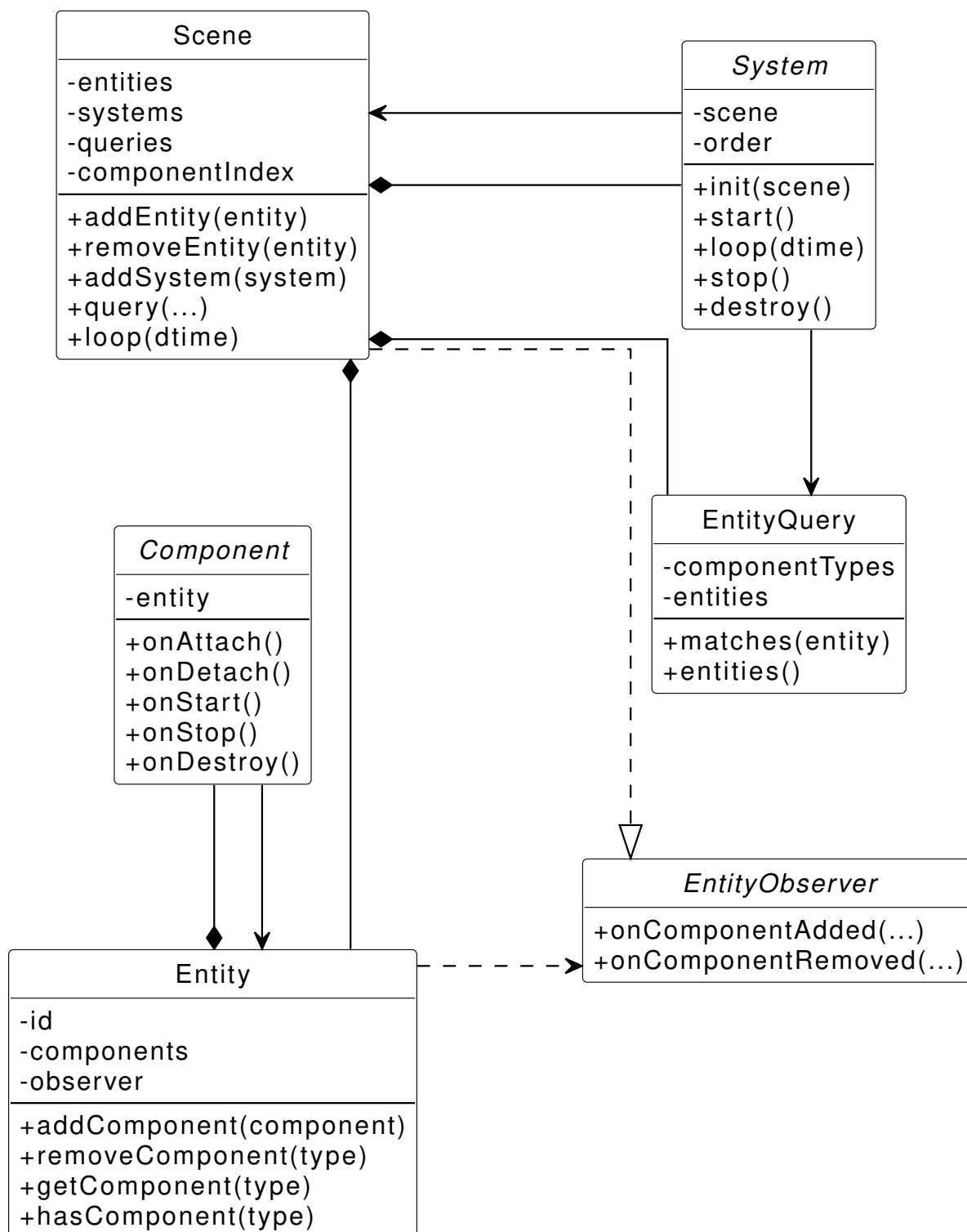


Рисунок 3.2 – Диаграмма классов ядра компонентной модели

3.3.3 Устройство приложения и контекста выполнения

Класс `Application` отвечает за создание и выполнение приложения. При создании экземпляра приложения формируется окно, создается объект `EngineContext`, инициализируются менеджер ресурсов, менеджер сцен, объект времени и состояние ввода. После этого разработчик загружает сцену через `SceneManager` и вызывает метод запуска приложения.

Контекст выполнения используется для того, чтобы основные подсистемы могли обращаться к общим сервисам. Например, пользовательский скрипт может получить текущее состояние ввода через фасад `Input`, а графическая подсистема — обратиться к менеджеру ресурсов. Такое решение уменьшает количество ссылок, которые пришлось бы вручную передавать между скриптами, системами и сценами.

Конфигурация приложения вынесена в отдельный объект `EngineConfig`. В ней задаются параметры, влияющие на выполнение приложения: цвет очистки окна, фиксированный шаг времени, масштаб времени, автоматическое обновление соотношения сторон камер и режим проверки ошибок `OpenGL`. Благодаря этому настройки не смешиваются с логикой сцены.

3.3.4 Устройство сцены

Сцена является центральной частью выполнения интерактивного приложения. Внутри нее хранятся сущности, системы обработки, запросы сущностей, индекс компонентов и очередь отложенных структурных изменений. Такое устройство необходимо потому, что во время выполнения кадра состав сцены может меняться: скрипт может создать новый объект, удалить существующий, добавить компонент или запросить изменение активной сцены.

Сцена хранит сущности в отображении по строковому идентификатору. Это позволяет быстро получать объект по имени и не допускать добавления двух сущностей с одинаковым идентификатором. Системы обработки хранятся отдельно и сортируются по числовому порядку выполнения. Благодаря этому можно явно задавать, что система скриптов должна обновляться

раньше физики, физика — раньше рендеринга, а отладочная отрисовка — после основной отрисовки сцены.

Для ускорения поиска объектов сцена поддерживает индекс компонентов. При добавлении сущности каждый ее компонент заносится в индекс. При удалении сущности или компонента индекс обновляется. Запрос EntityQuery содержит набор требуемых компонентов и хранит множество сущностей, удовлетворяющих этому набору. Например, графическая система создает запрос по компонентам Transform и MeshRenderer, а физическая система — запросы по Transform, Rigidbody и наследникам Collider.

Отдельное значение имеет механизм отложенных структурных изменений. Если сцена в данный момент выполняет системы, немедленное добавление или удаление объекта может нарушить обход коллекций. Поэтому при обновлении сцены такие изменения помещаются в очередь. После завершения текущего прохода систем очередь применяется в безопасный момент. В результате пользовательские скрипты могут менять состав сцены, не создавая ошибок обхода и не нарушая согласованное состояние запросов.

При запуске сцена инициализирует сущности, строит индексы, перестраивает запросы и вызывает метод init у систем. Затем запускаются компоненты сущностей и системы обработки. При остановке сцены системы останавливаются, после чего останавливаются компоненты. При уничтожении сцены освобождаются системы, сущности, индексы и запросы.

3.3.5 Сущности, компоненты и системы обработки

При проектировании ядра компонентной модели учитывались как классические подходы объектно-ориентированного проектирования, так и практические материалы по компонентным объектным системам [3, 28, 29].

Сущность представляет собой именованный контейнер компонентов. Она не содержит собственной прикладной логики и не наследуется от конкретных классов объектов. Смысл сущности определяется тем, какие компоненты к ней добавлены. Компонент Transform задает пространственное положение, Camera делает объект камерой, MeshRenderer позволяет отрисовать объ-

ект, Rigidbody включает объект в физическое моделирование, а наследник Script добавляет пользовательское поведение.

Базовый класс Component имеет жизненный цикл. При добавлении к сущности компонент получает ссылку на нее и может выполнить логику присоединения. При запуске сцены вызываются методы запуска, при остановке — методы остановки, при удалении — методы уничтожения. Это важно для компонентов, которые используют внутренние ресурсы или должны корректно реагировать на изменение состояния сцены.

Система обработки является объектом, который выполняется каждый кадр и работает с выбранными сущностями. В отличие от компонента, система обычно не принадлежит одной конкретной сущности. Она рассматривает сцену целиком и выбирает объекты по составу компонентов. Например, RenderSystem не хранится в каждом визуальном объекте, а обходит все сущности с компонентами Transform и MeshRenderer. Такой подход уменьшает связанность между объектами и централизует обработку однотипной логики.

3.3.6 Жизненный цикл приложения и сцены

Жизненный цикл одного кадра строится вокруг активной сцены. На каждом кадре приложение очищает окно, обновляет время, вызывает обновление активной сцены, применяет отложенные изменения, обновляет параметры камер, состояние ввода и окно. Последовательность выполнения одного кадра представлена на рисунке 3.3.

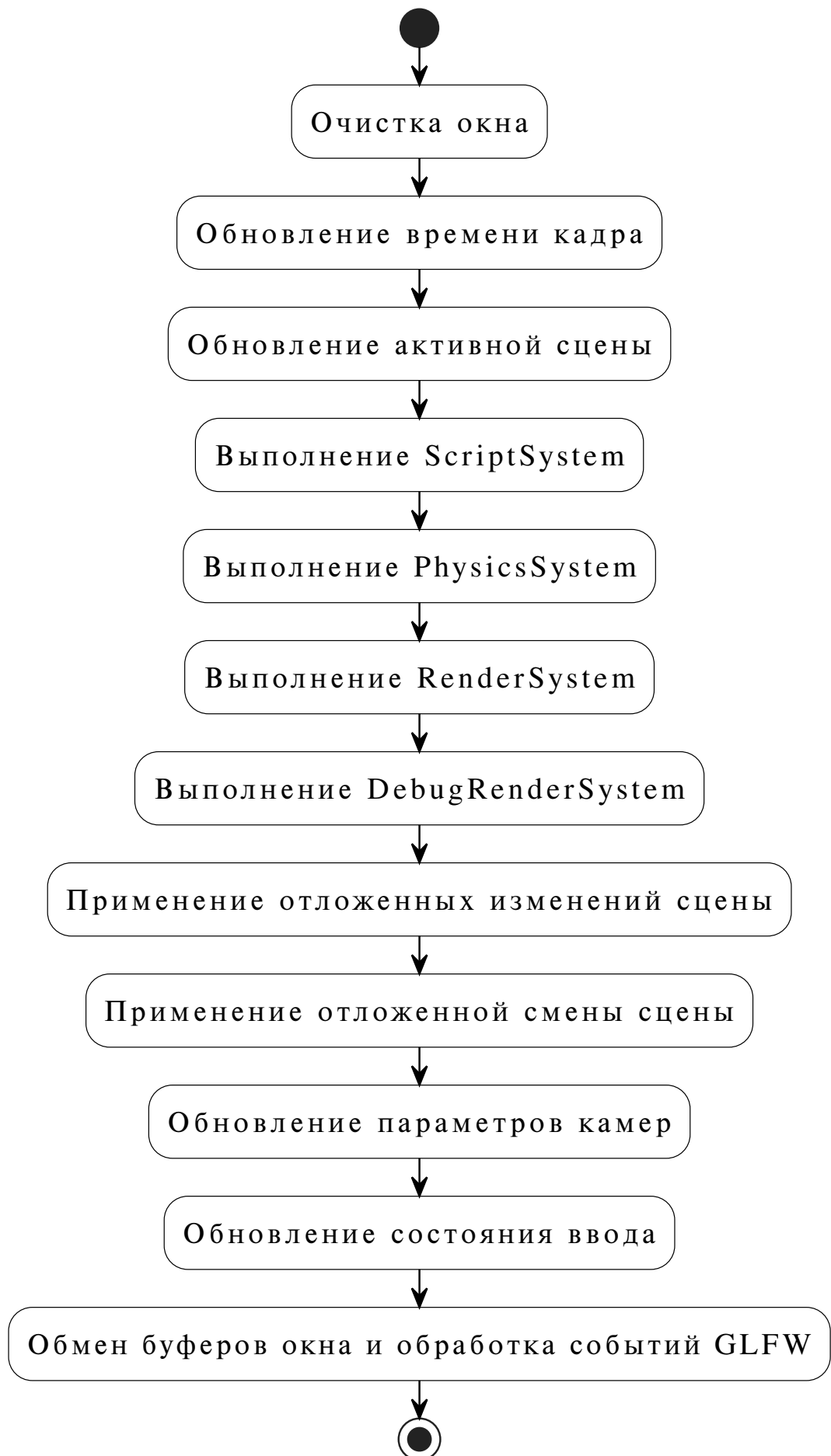


Рисунок 3.3 – Последовательность выполнения одного кадра приложения

Внутри обновления сцены выполняются зарегистрированные системы. Порядок выполнения задается числовым приоритетом системы. Для интерактивных сцен используется последовательность, при которой сначала выполняется ScriptSystem, затем PhysicsSystem, далее RenderSystem и DebugRenderSystem. Такой порядок выбран не случайно: скрипт может изменить управление объектом, физика обновляет положение тел, а рендеринг отображает уже обновленное состояние сцены.

3.4 Проектирование основных подсистем

3.4.1 Подсистема управления сценами

Управление сценами выполняет SceneManager. Он хранит активную сцену и поддерживает операции загрузки, замены, отложенной загрузки и отложенной замены. При обычной загрузке старая сцена останавливается, а новая инициализируется и запускается. При замене старая сцена дополнительно уничтожается.

Отложенная замена нужна для случаев, когда команда смены сцены вызывается из системы или пользовательского скрипта во время кадра. Например, обработчик перезапуска демонстрационной сцены может запросить создание новой сцены. Немедленное уничтожение текущей сцены в середине обновления было бы опасным, поэтому менеджер сцен применяет такое изменение после завершения текущего кадра.

3.4.2 Графическая подсистема

Графическая подсистема разрабатывается как слой между компонентами сцены и низкоуровневым графическим API. При проектировании учитывались подходы к рендерингу в реальном времени и практические руководства по OpenGL [16, 15, 18, 53, 54].

Графическая подсистема отвечает за отображение сцены средствами OpenGL. Ее центральным классом является RenderSystem. При инициализации система создает три запроса: к отрисовываемым объектам, к камерам

и к направленным источникам света. Отрисовываемыми считаются сущности, имеющие компоненты Transform и MeshRenderer. Камерами считаются сущности с Transform и Camera. Направленное освещение задается компонентом DirectionalLight.

На каждом кадре RenderSystem сначала выбирает активную камеру. Если в сцене присутствует несколько камер, выбирается включенная камера с наибольшим приоритетом. Затем выбирается активный направленный источник света. После этого строится очередь команд отрисовки: система обходит все сущности с Transform и MeshRenderer, проверяет наличие меша и материала, активность рендерера, соответствие слоя объекта маске камеры и видимость объекта относительно текущей камеры.

Видимость объекта проверяется по ограничивающей сфере. Радиус этой сферы хранится в MeshRenderer. Объект переводится в пространство камеры, после чего отбрасывается, если находится за ближней или дальней плоскостью отсечения либо выходит за границы области видимости камеры с учетом радиуса. Для ортогографической и перспективной камеры используются разные расчеты границ, но общий смысл одинаков: объект, не попадающий в область обзора, не добавляется в очередь отрисовки.

После построения очередь сортируется. В системе используется сортировка по очереди рендеринга, идентификатору шейдера, идентификатору текстуры и объекту меша. Это уменьшает количество переключений графического состояния и делает отрисовку более предсказуемой. Данная оптимизация не является полноценным пакетированием графических команд, но она уже снижает число лишних смен шейдера, текстуры и вершинных данных.

Для каждой команды отрисовки система получает GPU-представление меша. Если меш используется впервые, создается MeshGpuResource, включающий VAO, VBO и EBO. Если ресурс уже создан и версия меша не изменилась, он переиспользуется. Если версия меша изменилась, старый GPU-ресурс уничтожается и создается новый. Благодаря этому геометрия не загружается в видеопамять на каждом кадре.

Перед вызовом OpenGL-команды отрисовки материал активирует шейдер и текстуру. Затем в шейдер передаются матрицы модели, вида и проекции, параметры направленного света, ambient-цвет и флаг наличия освещения. После этого привязывается VAO и вызывается glDrawElements с режимом GL_TRIANGLES. Таким образом, каждый визуальный объект проходит путь: сущность сцены, компоненты Transform и MeshRenderer, команда рендеринга, GPU-ресурс меша, шейдер и команда OpenGL.

Основные классы графической подсистемы приведены в таблице 3.4.

Таблица 3.4 – Основные классы графической подсистемы

Класс	Назначение	Особенности реализации
RenderSystem	отрисовка объектов сцены	выбирает камеру и свет, строит и сортирует очередь рендеринга, вызывает OpenGL-команды.
MeshRenderer	визуальное представление сущности	связывает объект сцены с мешем, материалом, слоем, очередью рендеринга и радиусом видимости.
Camera	описание камеры	задает перспективную или ортографическую проекцию, приоритет, маску слоев, ближнюю и дальнюю плоскости отсечения.
Material	параметры отрисовки	содержит шейдер, цвет, текстуру и дополнительные uniform-параметры.
Mesh	геометрия объекта	хранит вершины, индексы и версию данных для контроля актуальности GPU-ресурса.
MeshGpuResource	GPU-представление меша	содержит идентификаторы VAO, VBO, EBO и количество индексов.

Продолжение таблицы 3.4

Класс	Назначение	Особенности реализации
Shader	шейдерная программа	компилирует вершинный и фрагментный шейдеры и передает uniform-переменные.
Texture	текстура OpenGL	загружает изображение через STB или создает отладочную текстуру.

3.4.3 Подсистема ресурсов

Подсистема ресурсов представлена классом `AssetManager`. Он хранит загруженные шейдеры и текстуры в отображениях по строковому ключу. При повторном запросе ресурса с тем же ключом возвращается уже созданный объект. Это исключает повторную компиляцию одного и того же шейдера и повторную загрузку одной и той же текстуры.

Менеджер ресурсов также предоставляет базовый шейдер и отладочную текстуру. Базовый шейдер используется как стандартный вариант для материалов, которым не задана отдельная шейдерная программа. Отладочная текстура позволяет показать объект даже в тех случаях, когда разработчик не назначил собственную текстуру. При завершении работы приложения `AssetManager` освобождает все созданные шейдеры и текстуры.

3.4.4 Подсистема пользовательского ввода

Подсистема ввода получает события от `GLFW` и сохраняет их в состоянии ввода. Поддерживаются клавиатура, кнопки мыши, положение курсора, смещение мыши и прокрутка. Прикладной код обращается к этим данным через фасад `Input`, поэтому пользовательские скрипты не зависят от конкретной реализации `callback`-функций окна.

Примером использования подсистемы ввода является `FreecamController`. Он перемещает камеру по клавишам `W`, `A`, `S`, `D`, `SPACE` и `LEFT_SHIFT`, а поворот

камеры определяет по смещению мыши. Такой скрипт одновременно проверяет работу ввода, трансформации и камеры.

3.4.5 Подсистема пользовательских скриптов

Пользовательские сценарии поведения реализуются через наследование от класса Script. Так как Script является компонентом, он добавляется к сущности так же, как Camera, MeshRenderer или Rigidbody. Система ScriptSystem находит компоненты-наследники Script и вызывает их методы жизненного цикла: init, start, loop, stop, destroy.

Скрипт может получать доступ к другим компонентам своей сущности и изменять их состояние. Например, скрипт движущейся платформы получает Rigidbody и задает ему скорость, а скрипт управления сценой отслеживает нажатие клавиши и запрашивает перезапуск демонстрационной сцены через SceneManager.

3.4.6 Подсистема отладочной визуализации

Подсистема отладочной визуализации предназначена для вывода вспомогательной геометрии поверх основной сцены. Она включает DebugRenderer и DebugRenderSystem. Разработчик или другая подсистема может добавить линии в очередь отладочной отрисовки, после чего DebugRenderSystem выводит их средствами OpenGL.

В физической демонстрационной сцене отладочная визуализация используется для сетки и границ коллайдеров. Это помогает проверять соответствие визуальной геометрии и физической формы объекта. Подсистема не заменяет основную графику, а служит инструментом разработки и проверки поведения сцены.

3.5 Проектирование физической подсистемы

Проектирование физической подсистемы основано на упрощенной модели твердых тел, обнаружении пересечений коробочных коллайдеров и импульсном разрешении контактов. Базовые алгоритмы таких подсистем рас-

смаатриваются в литературе по физическому моделированию и обнаружению столкновений [21, 22, 24, 25, 26, 55].

3.5.1 Общая структура физической подсистемы

Физическая подсистема проектируется как самостоятельная часть системы, без использования готового физического движка. Она предназначена для базовой демонстрации движения твердых тел и обработки столкновений в интерактивной сцене. В текущей версии системы поддерживаются динамические, статические и кинематические тела, гравитация, силы, моменты сил, линейная и угловая скорость, демпфирование, упругость, трение, слои столкновений, маски столкновений и коллайдер типа BoxCollider.

Название BoxCollider в тексте работы следует понимать как коллайдер формы параллелепипеда, то есть ориентированный ограничивающий параллелепипед. Его положение, ориентация и масштаб вычисляются на основе матрицы модели компонента Transform. В отличие от осево-ориентированной границы, такой коллайдер может поворачиваться вместе с объектом сцены.

Основные классы физической подсистемы приведены в таблице 3.5.

Таблица 3.5 – Основные классы физической подсистемы

Класс	Назначение	Особенности реализации
PhysicsSystem	расчет физики и столкновений	использует фиксированный шаг, аккумулятор времени, итерации решения скоростей и позиций.
Rigidbody	физическое тело	хранит массу, обратную массу, скорость, угловую скорость, силу, момент силы, тип тела и параметры демпфирования.
Collider	базовый класс области столкновения	задает смещение, слой, маску столкновений, трение, упругость, флаг триггера и активность.

Продолжение таблицы 3.5

Класс	Назначение	Особенности реализации
BoxCollider	коллайдер формы параллелепипеда	хранит половинные размеры по трем осям и используется для построения коллайдера формы параллелепипеда.
PhysicsBodyType	тип физического тела	разделяет тела на динамические, статические и кинематические.

Физическая система при инициализации создает два запроса: один для сущностей с Transform и Rigidbody, второй для сущностей с Transform и наследниками Collider. На каждом кадре она накапливает прошедшее время и выполняет один или несколько физических подшагов фиксированной длины. Число подшагов ограничивается параметром `maxSubSteps`, чтобы длинный кадр не приводил к бесконечной попытке “догнать” время моделирования.

3.5.2 Математическая модель движения

Поступательное движение динамического тела описывается классическими дифференциальными уравнениями движения материальной точки:

$$\frac{d\mathbf{x}}{dt} = \mathbf{v}, \quad \frac{d\mathbf{v}}{dt} = \mathbf{a}, \quad \mathbf{a} = \frac{\mathbf{F}}{m} + s_g \mathbf{g},$$

где $\mathbf{x}$ – положение тела, $\mathbf{v}$ – линейная скорость, $\mathbf{a}$ – ускорение, $\mathbf{F}$ – сумма внешних сил, m – масса тела, $\mathbf{g}$ – вектор гравитации, s_g – коэффициент влияния гравитации для конкретного тела.

Для вращательного движения используется угловая скорость $\boldsymbol{\omega}$ и момент силы $\boldsymbol{\tau}$:

$$\frac{d\boldsymbol{\omega}}{dt} = \mathbf{I}^{-1} \boldsymbol{\tau},$$

где $\mathbf{I}$ – тензор инерции тела. Для тела с параллелепипедным коллайдером в коде используется приближение инерции прямоугольного параллелепипеда. Если ширина, высота и глубина равны w , h и d , то главные моменты

инерции вычисляются как:

$$I_x = \frac{m}{12}(h^2 + d^2), \quad I_y = \frac{m}{12}(w^2 + d^2), \quad I_z = \frac{m}{12}(w^2 + h^2).$$

В реализации хранится не сам тензор инерции, а применяется обратная инерция по локальным осям параллелепипеда. Это нужно при расчете углового изменения скорости от импульса, приложенного не в центре масс.

3.5.3 Метод численного интегрирования

Физика обновляется с фиксированным шагом Δt . В проектном решении для демонстрационных сцен используется шаг 1/60 секунды. Внутри одного физического шага сначала обновляются скорости по силам, затем по обновленным скоростям изменяются положение и поворот объекта. Такой порядок соответствует полувявному методу Эйлера:

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \mathbf{a}_n \Delta t, \quad \mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{v}_{n+1} \Delta t.$$

Для угловой скорости используется аналогичная схема:

$$\boldsymbol{\omega}_{n+1} = \boldsymbol{\omega}_n + \mathbf{I}^{-1} \boldsymbol{\tau}_n \Delta t.$$

После этого из угловой скорости вычисляется малый поворот за текущий шаг:

$$\Delta\varphi = |\boldsymbol{\omega}_{n+1}| \Delta t.$$

Ось поворота определяется нормированным вектором угловой скорости, после чего создается кватернион дополнительного поворота. Новый поворот объекта получается умножением текущего кватерниона на кватернион приращения с последующей нормализацией. Такой способ удобен для трехмерной сцены, так как позволяет избежать проблем с накоплением углов Эйлера.

Перед применением скорости используются линейное и угловое демпфирование:

$$\mathbf{v}_{n+1} = \mathbf{v}_{n+1} \max(0, 1 - k_l \Delta t), \quad \omega_{n+1} = \omega_{n+1} \max(0, 1 - k_a \Delta t),$$

где k_l – коэффициент линейного демпфирования, а k_a – коэффициент углового демпфирования. Также в системе ограничиваются максимальная линейная и максимальная угловая скорости. Это уменьшает вероятность неустойчивого поведения при больших силах или ошибочно заданных параметрах сцены.

3.5.4 Обнаружение столкновений

Для проверки пересечений выпуклых коробочных форм применяется подход, близкий к методу разделяющих осей; иерархии и ограничивающие объемы широко применяются для ускорения таких проверок [27].

Обнаружение столкновений состоит из двух этапов. На первом этапе выполняется быстрая проверка пересечения осево-ориентированных ограничивающих объемов (AABB). Если эти объемы не пересекаются, более точная проверка не выполняется. Такой этап нужен для раннего отбрасывания пар объектов, которые находятся далеко друг от друга.

На втором этапе для двух коллайдеров формы параллелепипеда используется проверка по теореме разделяющих осей. Каждый BoxCollider преобразуется в коллайдер формы параллелепипеда: вычисляются центр, три локальные оси в мировых координатах и половинные размеры с учетом масштаба объекта. Далее проверяются оси первого параллелепипеда, оси второго параллелепипеда и попарные векторные произведения этих осей. Всего для пары трехмерных параллелепипедов рассматривается до пятнадцати осей.

Для каждой оси вычисляются радиусы проекции двух параллелепипедов и расстояние между их центрами вдоль этой оси. Если сумма радиусов меньше расстояния между центрами, между объектами существует разделяющая ось, и столкновения нет. Если пересечение найдено по всем осям, пара

считается сталкивающейся. Ось с минимальным перекрытием используется как нормаль столкновения, а величина перекрытия – как глубина проникновения.

После определения нормали и глубины проникновения формируется набор контактных точек. Для этого вершины, близкие к контактной плоскости, проецируются на эту плоскость и проверяются на принадлежность второму параллелепипеду. Если точек получается слишком много, они сокращаются до ограниченного количества, чтобы решатель столкновений не выполнял лишнюю работу.

3.5.5 Разрешение столкновений

Разрешение столкновений выполняется итерационно. Сначала несколько раз обрабатываются скорости, затем несколько раз выполняется позиционная коррекция. Такой подход не дает идеального аналитического решения для всех контактов сразу, но хорошо подходит для интерактивной демонстрационной сцены.

Для контактной точки вычисляется относительная скорость:

$$\mathbf{v}_{rel} = (\mathbf{v}_b + \boldsymbol{\omega}_b \times \mathbf{r}_b) - (\mathbf{v}_a + \boldsymbol{\omega}_a \times \mathbf{r}_a),$$

где $\mathbf{r}_a$ и $\mathbf{r}_b$ – радиус-векторы от центра масс тел до контактной точки. Проекция этой скорости на нормаль контакта показывает, сближаются ли тела вдоль нормали.

Нормальный импульс вычисляется по формуле:

$$j_n = \frac{b - \mathbf{v}_{rel} \cdot \mathbf{n}}{m_a^{-1} + m_b^{-1} + D_a + D_b},$$

где $\mathbf{n}$ – нормаль контакта, b – скоростная поправка, учитывающая упругость и баумгартовскую коррекцию проникновения, D_a и D_b – угловые добавки, возникающие из-за приложения импульса не в центре масс. Если знаменатель равен нулю или тела не должны двигаться, импульс не применяется.

После нормального импульса рассчитывается касательное направление и импульс трения. Его величина ограничивается максимальным значением, пропорциональным нормальному импульсу и коэффициенту трения. Благодаря этому объект может не только отскакивать от поверхности, но и постепенно терять касательную скорость при контакте.

Позиционная коррекция устраняет остаточное взаимное проникновение тел. В системе используется небольшая допустимая глубина s , после которой часть проникновения распределяется между телами пропорционально обратным массам:

$$C = \min(0,05, \max(0, p - s) \cdot 0,2),$$

где p – глубина проникновения. Если одно тело статическое, вся коррекция фактически приходится на динамическое тело. Такой способ не делает физику абсолютно точной, но предотвращает заметное накопление проникновений между объектами.

3.5.6 Архитектурные ограничения физической подсистемы

Физическая подсистема предназначена для базовой обработки твердых тел и столкновений в демонстрационных интерактивных сценах. На текущем этапе поддерживается параллелепипедный тип коллайдера, динамические, статические и кинематические тела, импульсное разрешение столкновений, трение, упругость, ограничение скорости и режим сна для устойчиво покоящихся тел.

При этом система не ставит целью заменить промышленный физический движок. В текущую область разработки не входят составные соединения тел, шарнирные ограничители, непрерывное обнаружение столкновений для очень быстрых объектов, коллайдеры произвольной сетки и сложные материалы. Такое ограничение соответствует задачам ВКР: необходимо показать собственную архитектуру физической подсистемы и ее совместную работу со сценой, компонентами, скриптами и рендерингом.

3.6 Проектирование демонстрационных сцен

Для проверки применимости архитектуры к разным типам интерактивных приложений в проекте предусматривается не одна, а несколько демонстрационных сцен. Каждая сцена строится программно средствами Java API и использует общий набор элементов: сущности, компоненты, системы обработки, ресурсы, пользовательские скрипты и жизненный цикл сцены. Отличие сцен состоит в том, какие подсистемы они делают основными и какие сценарии взаимодействия должны быть проверены после реализации.

На стадии технического проектирования демонстрационные сцены рассматриваются как проектные сценарии применения движка. Они не заменяют системное тестирование: скриншоты, пошаговые проверки и фактические результаты работы сцен должны быть приведены в рабочем проекте. В данном разделе фиксируется назначение сцен и их роль в подтверждении выбранной архитектуры.

Состав демонстрационных сцен приведен в таблице 3.6.

Таблица 3.6 – Проектное назначение демонстрационных сцен

Сцена	Основные подсистемы	Проектное назначение
ShowcaseScene	сцена, трансформации, камера, освещение, рендеринг, отладочная визуализация	демонстрация базовой трехмерной сцены, иерархии объектов, свободной камеры, направленного света, слоев видимости, направляющих и ограничивающих объемов.
Interactive2DScene	ортографическая камера, ввод мыши, пользовательские скрипты, 2D-представление объектов	демонстрация интерактивного сценария с обработкой положения курсора, нажатий мыши, переключателей, изменения цвета, вращения, пульсации и слайдера.

Продолжение таблицы 3.6

Сцена	Основные подсистемы	Проектное назначение
TexturedLightingScene	материалы, текстуры, шейдеры, направленное освещение, скрипты анимации	демонстрация работы материалов, процедурно созданных и загружаемых текстур, передачи uniform-параметров в шейдеры и изменения источника света во времени.
PhysicsScene	физика, коллайдеры, динамические и кинематические тела, рендеринг, отладочная визуализация	демонстрация совместной работы физической подсистемы, визуального отображения, скриптов управления, перезапуска сцены и отладочного отображения границ коллайдеров.

Набор сцен выбран так, чтобы не сводить проверку программной системы к одному частному примеру. ShowcaseScene подтверждает базовую пригодность архитектуры для трехмерной сцены. Interactive2DScene показывает, что та же компонентная модель применима для интерактивных двумерных объектов и обработки пользовательского ввода. TexturedLightingScene выделяет графическую часть, связанную с материалами, текстурами и освещением. PhysicsScene используется как наиболее комплексный сценарий, в котором одновременно задействованы сцена, скрипты, рендеринг, ввод, физика и отладочная визуализация.

При проектировании демонстрационных сцен приняты следующие общие решения: сцены создаются программно без визуального редактора; каждая сцена регистрирует собственный набор систем обработки; объекты сцены формируются из сущностей и компонентов; повторяющиеся элементы поведения выносятся в пользовательские скрипты; переключение или переза-

пуск сцены выполняется через SceneManager с учетом механизма отложенных структурных изменений.

3.7 Выводы по разделу

В рамках технического проекта определено внутреннее устройство разрабатываемой компонентно-ориентированной программной системы. Обоснован выбор Java 25, LWJGL, OpenGL, GLSL, GLFW и STB как технологической основы проекта. Вместо проекта базы данных описаны элементы устройства движка: приложение, контекст выполнения, сцена, сущности, компоненты, системы обработки, запросы сущностей, ресурсы, графические классы и физические компоненты.

Подробно рассмотрено устройство сцены, включая хранение сущностей и систем, индексацию компонентов, запросы сущностей, жизненный цикл и механизм отложенных структурных изменений. Описана графическая подсистема: выбор активной камеры, построение очереди рендеринга, проверка видимости, сортировка команд, кэширование GPU-ресурсов мешей и передача данных в шейдеры. Отдельно рассмотрена физическая подсистема, проектируемая без использования готового физического движка: приведены уравнения движения, метод полуявного интегрирования Эйлера, обнаружение столкновений коллайдеров формы параллелепипеда и импульсное разрешение контактов.

В качестве проектных сценариев применения системы определены четыре демонстрационные сцены: ShowcaseScene, Interactive2DScene, TexturedLightingScene и PhysicsScene. Их фактическая реализация, спецификация задействованных классов и результаты системного тестирования должны быть раскрыты на стадии рабочего проекта.

4 Рабочий проект

Рабочий проект отражает стадию реализации компонентно-ориентированной программной системы. В данном разделе приводится спецификация реализованных классов, описание демонстрационных сцен, тестовые наборы для модульного и регрессионного тестирования, а также результаты системного тестирования программной основы для построения интерактивных приложений на языке Java.

В рабочем проекте рассматриваются программные компоненты, из которых состоит разработанная система: ядро ECS, классы управления приложением и сценами, компоненты объектов, графические и ресурсные классы, системы обработки, физическая подсистема и демонстрационные сцены.

4.1 Спецификация компонентов программной системы

Исходный код программной системы разделен на пакеты по функциональному назначению. Такое разделение соответствует практикам организации Java-кода и проектирования программных интерфейсов [6, 7], а также позволяет отдельно развивать ядро компонентной модели, прикладные компоненты сцены, графическую подсистему, физическую подсистему, средства ввода и демонстрационные примеры.

Основные группы классов программной системы приведены в таблице 4.1.

Таблица 4.1 – Группы классов программной системы

Группа классов	Пакеты	Назначение
Ядро компонентной модели	qchromatic.jecse.core	Реализует сущности, компоненты, системы обработки, запросы сущностей и общий жизненный цикл объектов ECS.

Продолжение таблицы 4.1

Группа классов	Пакеты	Назначение
Управление приложением и сценами	qchromatic.jecse.engine	Обеспечивает создание приложения, хранение контекста выполнения, загрузку сцен, учет времени, ввод и управление ресурсами.
Компоненты сцены	qchromatic.jecse.component	Описывает прикладные свойства сущностей: трансформацию, камеру, визуальное представление, свет, скрипты, физические тела и коллайдеры.
Графическая подсистема	qchromatic.jecse.graphics, qchromatic.jecse.system	Создает окно, OpenGL-контекст, шейдеры, текстуры, GPU-ресурсы мешей, а также выполняет отрисовку сцены и отладочной геометрии.
Физическая подсистема	qchromatic.jecse.component, qchromatic.jecse.system	Реализует физические тела, коробочные коллайдеры, обнаружение столкновений и импульсное разрешение контактов.
Демонстрационные сцены	common.showcase	Содержит Java-классы, показывающие применение программной системы в 3D-сцене, 2D-интерактивной сцене, текстурированной сцене с освещением и физической сцене.

4.1.1 Вспомогательные математические классы

Вспомогательные математические классы используются в большинстве подсистем программной системы и реализуют базовые операции, необходимые для работы с координатами, поворотами и матрицами преобразований [17, 23]. Через них задаются координаты объектов, направления, цвета,

повороты, матрицы модели, вида и проекции. Эти классы не относятся к отдельной демонстрационной сцене, а образуют общий прикладной слой для трансформаций, рендеринга, камеры, ввода и физической подсистемы.

Спецификация вспомогательных математических классов приведена в таблице 4.2.

Таблица 4.2 – Спецификация вспомогательных математических классов

Класс	Основные поля и методы	Назначение
Vec2	x, y, add, mul, length, normalized, dot	Описывает двумерные величины. Используется для координат мыши, смещения курсора, прокрутки и расчетов в двумерных демонстрационных сценах.
Vec3	x, y, z, add, mul, length, normalized, cross, dot	Описывает трехмерные векторы. Применяется для положения, масштаба, направления движения, скорости, сил, нормалей столкновений и направления источника света.
Vec4	x, y, z, w, add, mul, length, normalized, dot	Хранит четырехкомпонентные значения. Используется для цветов материалов и света, а также для передачи uniform-параметров в графическую подсистему.
Mat4	identity, mul, transformPoint, translation, rotation, scale, frustum, ortho	Реализует матрицу 4x4. Используется для построения матриц модели, вида и проекции, а также для преобразования точек между пространствами сцены.

Продолжение таблицы 4.2

Класс	Основные поля и методы	Назначение
Quaternion	identity, axisAngle, euler, mul, normalize, rotate, forward, right, up, toMat4	Описывает поворот объекта без накопления лишних угловых ошибок, формирует направления камеры и объектов, а также преобразуется в матрицу поворота.

4.1.2 Классы ядра компонентной модели

Ядро компонентной модели реализует базовые элементы ECS-архитектуры. Сущность не содержит прикладной логики сама по себе, а получает поведение через набор компонентов и обработку системами. Компоненты хранят состояние объекта, системы выполняют обработку сущностей сцены, а запросы сущностей позволяют получать только те объекты, которые имеют необходимый набор компонентов.

Спецификация основных классов ядра компонентной модели приведена в таблице 4.3.

Таблица 4.3 – Спецификация классов ядра компонентной модели

Класс	Основные поля и методы	Назначение
Entity	id, addComponent, getComponent, removeComponent, hasComponent, components	Хранит уникальный идентификатор сущности и набор компонентов. Обеспечивает добавление, получение и удаление компонентов. При изменении состава компонентов уведомляет наблюдателя сцены.
Component	entity, started, destroyed, onAttach, onStart, onStop, onDestroy	Является базовым классом для всех компонентов. Хранит ссылку на сущность и управляет жизненным циклом компонента: присоединение, запуск, остановка, уничтожение.

Продолжение таблицы 4.3

Класс	Основные поля и методы	Назначение
System	init, start, loop, stop, destroy, order, scene	Является базовым классом для систем обработки. Позволяет выполнять код на разных стадиях жизненного цикла сцены и задавать порядок выполнения систем.
EntityQuery	matches, update, contains, snapshot, iterator	Хранит выборку сущностей, соответствующих набору обязательных компонентов. Используется системами для обхода только подходящих объектов сцены.
EntityObserver	onComponentAdded, onComponentRemoved	Определяет интерфейс уведомлений о добавлении и удалении компонентов. В реализации сцены используется для обновления индексов и запросов сущностей.
Disposable	destroy	Определяет единый интерфейс освобождения ресурсов. Используется для компонентов и графических объектов, которые требуют явного уничтожения.

Класс Entity хранит компоненты в отображении по точному классу компонента. Благодаря этому получение компонента по типу выполняется напрямую, а сама сущность не может иметь два компонента одного и того же конкретного класса. Для компонентов-наследников предусмотрены методы, учитывающие иерархию классов, например `getComponentsInheritance` и `hasComponentInheritance`. Это используется в скриптовой подсистеме, где пользовательские скрипты являются наследниками базового класса `Script`.

4.1.3 Классы управления приложением и сценами

Классы управления приложением связывают ECS-ядро с окном, пользовательским вводом, ресурсами и временем. Основной цикл приложения на-

ходится в классе Application. Сцена хранит сущности и системы обработки, а менеджер сцен отвечает за загрузку, замену и уничтожение активной сцены.

Спецификация основных классов управления приложением приведена в таблице 4.4.

Таблица 4.4 – Спецификация классов управления приложением и сценами

Класс	Основные поля и методы	Назначение
Application	run, context, getWindow	Создает контекст программной системы, запускает основной цикл, обновляет время, активную сцену, ввод, окно и освобождает ресурсы после завершения работы.
EngineConfig	clearColor, fixedDeltaTime, timeScale, autoUpdateCameraAspect, debugOpenGLErrors	Хранит параметры запуска программной системы: цвет очистки экрана, фиксированный шаг времени, масштаб времени, автоматическое обновление соотношения сторон камеры и режим проверки ошибок OpenGL.
EngineContext	use, current, input, assets, sceneManager, time, window, destroy	Хранит текущие экземпляры подсистем приложения и делает их доступными из статических фасадов. Освобождает активную сцену, ресурсы и окно при завершении приложения.
Scene	init, start, loop, stop, dispose, addEntity, removeEntity, addSystem, query	Хранит сущности, системы, индексы компонентов и запросы сущностей. Управляет жизненным циклом компонентов и систем, а также выполняет отложенные структурные изменения.
SceneManager	load, replace, requestLoad, requestReplace, applyPendingChanges, disposeActiveScene	Управляет активной сценой. Поддерживает немедленную загрузку и замену сцены, а также отложенную замену, безопасную для вызова из пользовательских скриптов.

Продолжение таблицы 4.4

Класс	Основные поля и методы	Назначение
Time	reset, update, deltaTime, unscaledDeltaTime, time, frame	Хранит время приложения, длительность текущего кадра, не масштабированное время, номер кадра и параметры масштаба времени.
InputState	setKeyState, setMousePosition, setScrollOffset, getKey, getKeyDown, getMouseDelta, update	Хранит состояние клавиатуры, мыши, прокрутки и смещение курсора между кадрами. Позволяет отличать удержание клавиши от однократного нажатия.
Input	use, getKey, getKeyDown, getMousePosition, getMouseDelta, getMouseScroll	Статический фасад для доступа к текущему состоянию ввода из пользовательских скриптов и систем.
KeyCode	A--Z, ALPHA_0--ALPHA_9, F1--F12, UP, DOWN, LEFT, RIGHT, MB_1--MB_8	Предоставляет именованные константы клавиатуры и мыши, связанные с кодами GLFW. Используется в пользовательских скриптах для читаемой обработки ввода.

Особое значение имеет механизм отложенных структурных изменений в классе Scene. Если сущность, компонент или система добавляются либо удаляются во время обхода систем, действие помещается в очередь и выполняется после завершения небезопасного участка обновления. Такой подход предотвращает ошибки, связанные с изменением коллекций во время их обхода, и позволяет вызывать изменения структуры сцены из пользовательских скриптов.

4.1.4 Компоненты сцены

Компоненты сцены описывают данные, которые могут быть добавлены к сущностям. Именно набор компонентов определяет, будет ли сущность

отображаться, участвовать в физике, выполнять пользовательский скрипт или использоваться как камера.

Спецификация основных компонентов сцены приведена в таблице 4.5.

Таблица 4.5 – Спецификация компонентов сцены

Компонент	Основные поля и методы	Назначение
Transform	position, rotation, scale, parent, children, worldPosition, getModelMatrix	Хранит положение, поворот, масштаб и иерархию трансформаций. Формирует локальную и мировую матрицы объекта.
Camera	ortho, fov, orthoSize, aspectRatio, near, far, priority, cullingMask, getViewMatrix, getProjectionMatrix	Описывает перспективную или ортографическую камеру. Формирует матрицы вида и проекции, задает приоритет камеры и маску видимости слоев.
MeshRenderer	mesh, material, enabled, layer, renderQueue, boundsRadius	Связывает сущность с геометрией и материалом. Используется системой рендеринга для построения очереди отображения.
DirectionalLight	direction, color, intensity, enabled	Описывает направленный источник света, применяемый при отрисовке материалов с освещением.
Script	init, start, loop, stop, destroy	Базовый класс пользовательского поведения. Подключается к сущности как компонент и обрабатывается ScriptSystem.
Rigidbody	type, velocity, angularVelocity, addForce, addImpulse, mass, useGravity, sleep, wakeUp	Описывает физическое тело. Хранит тип тела, скорость, угловую скорость, массу, силы, импульсы, демпфирование и состояние сна.
Collider	enabled, trigger, offset, layer, mask, restitution, friction	Базовый класс коллайдера. Задает активность, смещение, слой столкновений, маску взаимодействия, упругость и трение.

Продолжение таблицы 4.5

Компонент	Основные поля и методы	Назначение
BoxCollider	halfExtents, size	Реализует коробочный коллайдер. Задает половинные размеры или полный размер параллелепипеда, участвующего в физическом расчете.

В компонентах используется цепочечный стиль настройки, при котором метод изменения параметра возвращает сам объект. Благодаря этому сущность сцены может создаваться компактно: сначала создается Entity, затем последовательно добавляются Transform, MeshRenderer, Rigidbody, BoxCollider и другие компоненты.

4.1.5 Графические и ресурсные классы

Графическая подсистема реализует создание окна, управление OpenGL-ресурсами, шейдерами, текстурами и материалами. Часть классов работает напрямую с нативными ресурсами и поэтому реализует явное освобождение памяти и объектов OpenGL.

Спецификация основных графических и ресурсных классов приведена в таблице 4.6.

Таблица 4.6 – Спецификация графических и ресурсных классов

Класс	Основные поля и методы	Назначение
Window	show, clear, update, destroy, shouldClose, aspectRatio, setCursorDisabled, addResizeListener	Создает GLFW-окно, OpenGL-контекст, регистрирует обработчики ввода и обновляет состояние окна в основном цикле.

Продолжение таблицы 4.6

Класс	Основные поля и методы	Назначение
Shader	use, setUniform, getUniformLocation, destroy	Загружает и компилирует GLSL-шейдеры, линкует программу OpenGL, кэширует расположения uniform-переменных и передает значения в шейдер.
Texture	load, debugTexture, setPixel, uploadToGPU, bind, destroy	Загружает текстуры из файлов через STB или создает процедурные изображения, передает данные в OpenGL и освобождает объект текстуры.
Mesh	vertices, triangles, vertexStride, vertexCount, indexCount, version	Хранит массив вершин, индексы треугольников, шаг вершины и номер версии геометрии для обновления GPU-ресурса при изменении меша.
Meshes	triangle, quad, cube	Предоставляет фабричные методы для создания базовой геометрии: треугольника, прямоугольника и куба.
Material	use, shader, texture, color, set, setTexture	Хранит шейдер, набор uniform-параметров и текстуры. Перед отрисовкой активирует шейдер, привязывает текстуры и передает параметры материала.
AssetManager	shader, basicShader, debugTexture, texture, destroy	Кэширует шейдеры и текстуры по строковым ключам, предотвращает повторную загрузку одинаковых ресурсов и освобождает их при завершении приложения.
Assets	use, basicShader, debugTexture, texture	Статический фасад для доступа к текущему менеджеру ресурсов из разных частей программы.

Продолжение таблицы 4.6

Класс	Основные поля и методы	Назначение
MeshGpuResource	vaoId, vboId, eboId, meshVersion, indexCount, destroy	Хранит идентификаторы VAO, VBO и EBO, связанные с конкретной версией меша, и удаляет соответствующие OpenGL-объекты.
DebugRenderer	line, axis, bounds, consumeLines, clear	Накапливает отладочные линии на кадр. Используется для отображения осей, границ объектов, коллайдеров и вспомогательной сетки.
Shaders	basic	Предоставляет краткий доступ к стандартному шейдеру программной системы через фасад ресурсов. Упрощает создание базовых материалов в демонстрационных сценах.
WindowResizeListener	onResize	Определяет интерфейс обратного вызова при изменении размеров окна. Используется для реакции подсистем на изменение ширины, высоты и соотношения сторон.

Для типового использования разработчик не должен напрямую создавать VAO, VBO и EBO. Эти операции скрыты внутри системы рендеринга и класса MeshGpuResource. При изменении версии меша старый GPU-ресурс уничтожается и создается новый.

4.1.6 Системы обработки

Системы обработки реализуют поведение сцены на каждом кадре. Каждая система получает доступ к сцене, формирует запросы сущностей и выполняет обработку только подходящих объектов. Порядок выполнения задается числовым значением order базового класса System.

Спецификация систем обработки приведена в таблице 4.7.

Таблица 4.7 – Спецификация систем обработки

Система	Используемые компоненты	Выполняемые функции
ScriptSystem	Script и его наследники	Находит пользовательские скрипты, вызывает методы init, start, loop, stop, destroy, реагирует на добавление и удаление скриптов во время работы сцены.
PhysicsSystem	Transform, Rigidbody, Collider, BoxCollider	Выполняет фиксированный физический шаг, применяет силы и гравитацию, интегрирует скорости и положения, обнаруживает столкновения коробочных коллайдеров, разрешает контакты и отображает коллайдеры в режиме отладки.
RenderSystem	Transform, MeshRenderer, Camera, DirectionalLight	Выбирает активную камеру и источник света, строит очередь команд отрисовки, проверяет слои и видимость, сортирует команды и выполняет вызовы OpenGL для отображения мешей.
DebugRenderSystem	Transform, Camera; данные DebugRenderer	Получает накопленные отладочные линии, формирует динамический буфер вершин и рисует вспомогательную геометрию поверх основной сцены.

Для реализации рендеринга и физики используются внутренние служебные структуры. Они не входят в публичный API, но важны для понимания рабочего проекта, так как показывают, каким образом системы обработки подготавливают данные кадра и результаты столкновений. Основные внутренние структуры систем обработки приведены в таблице 4.8.

Таблица 4.8 – Внутренние структуры систем обработки

Структура	Система	Назначение
RenderCommand	RenderSystem	Хранит меш, материал, трансформацию и номер очереди отрисовки. На основе этих структур формируется упорядоченный список объектов, которые должны быть отображены в текущем кадре.
LightState	RenderSystem	Хранит подготовленные параметры направленного света: направление, цвет и признак активности. Передается в материал перед выполнением команд отрисовки.
ColliderEntry	PhysicsSystem	Связывает сущность, трансформацию, физическое тело, коллайдер и вычисленную физическую форму. Используется при поиске пар объектов для проверки столкновений.
BoxShape	PhysicsSystem	Описывает коробочную форму в мировых координатах: центр, локальные оси, половинные размеры и граничные значения. Нужна для SAT-проверки и отладочной визуализации.
SatResult	PhysicsSystem	Хранит результат проверки пересечения коробок методом разделяющих осей: минимальную глубину проникновения и нормаль столкновения.
Contact	PhysicsSystem	Описывает контакт двух коллайдеров: участвующие объекты, нормаль, глубину проникновения и набор точек контакта. Используется при позиционной коррекции и импульсном разрешении столкновения.

В демонстрационных сценах используется следующий типовой порядок выполнения систем: сначала ScriptSystem, затем PhysicsSystem, затем RenderSystem и DebugRenderSystem. Благодаря этому пользовательские скрипты успевают обработать ввод, физическая подсистема обновляет состояние тел, после чего графическая подсистема отображает уже измененную сцену.

4.1.7 Демонстрационные сцены

Демонстрационные сцены реализованы как самостоятельные Java-классы с методом main. Каждая сцена создает приложение, настраивает окно, формирует объект Scene, добавляет в него системы обработки, сущности и компоненты, после чего запускает основной цикл приложения. Сцены не используют визуальный редактор и показывают, что программная система может применяться через Java API.

Спецификация демонстрационных сцен приведена в таблице 4.9.

Таблица 4.9 – Спецификация демонстрационных сцен

Сцена	Состав сцены	Назначение реализации
ShowcaseScene	Перспективная камера, свободное управление камерой, направленный свет, корневой и дочерний кубы, пол, объект на отдельном слое, скрытый объект, отладочные направляющие.	Демонстрирует базовый трехмерный рендеринг, иерархию Transform, работу камеры, слои видимости, отсечение объектов, вращение объектов через скрипты и отладочную геометрию.
Interactive2DScene	Ортогографическая камера, фон, интерактивная кнопка изменения цвета, переключатель вращения, пульсирующий объект, слайдер и скрипт отслеживания координат мыши.	Демонстрирует применение той же компонентной архитектуры для двумерной интерактивной сцены с обработкой курсора, кликов мыши и изменением состояния объектов.

Продолжение таблицы 4.9

Сцена	Состав сцены	Назначение реализации
TexturedLightingScene	Перспективная камера, свободное управление камерой, анимированный направленный свет, процедурно созданные текстуры травы, земли и камня, набор текстурированных кубов.	Демонстрирует работу материалов, текстур, uniform-параметров, направленного освещения и пользовательского скрипта изменения параметров света во времени.
PhysicsScene	Перспективная камера, свободное управление камерой, направленный свет, пол, узкая опора, движущаяся кинематическая платформа, статические стены, динамические кубы, отладочная сетка и визуализация коллайдеров.	Демонстрирует совместную работу скриптов, физики, рендеринга, ввода, перезапуска сцены и отладочной визуализации физической подсистемы.

Наиболее комплексной демонстрационной сценой является PhysicsScene, так как она одновременно использует большинство реализованных подсистем. Структура основных объектов данной сцены приведена в таблице 4.10.

Таблица 4.10 – Основные объекты сцены PhysicsScene

Объект сцены	Компоненты	Назначение
sun	DirectionalLight	Источник направленного света, влияющий на освещение видимых объектов.

Продолжение таблицы 4.10

Объект сцены	Компоненты	Назначение
camera	Transform, Camera, FreecamController	Камера наблюдателя со свободным перемещением и поворотом по клавиатуре и мыши.
kinematic-platform	Transform, Rigidbody, BoxCollider, MeshRenderer, PlatformBounceScript	Движущаяся кинематическая платформа, которая меняет направление движения в заданных границах и взаимодействует с динамическими объектами.
physics-grid	PhysicsGridScript	Скрипт вывода отладочной сетки на плоскости сцены для визуальной оценки положения объектов.
scene-controls	PhysicsSceneControlScript	Скрипт перезапуска сцены по нажатию клавиши R через отложенную замену сцены в SceneManager.
floor	Transform, BoxCollider, MeshRenderer	Статическая опорная поверхность для падения динамических тел.
narrow-support	Transform, BoxCollider, MeshRenderer	Узкая статическая опора, позволяющая проверить падение тел с края и возникновение вращения при смещенном контакте.
Статические стены	Transform, BoxCollider, MeshRenderer	Ограничивают область сцены и используются для проверки столкновений со стенами.
Динамические кубы	Transform, Rigidbody, BoxCollider, MeshRenderer	Физические тела, падающие под действием гравитации и взаимодействующие с полом, опорами, стенами и кинематической платформой.

4.2 Модульное и регрессионное тестирование программной системы

Тестирование программной системы выполнялось по наборам проверок, для которых фиксировались входные данные, ожидаемый результат и фактическое поведение. Такой подход соответствует общим принципам систематического тестирования программ [13].

Тестирование выполнено для подсистем, которые могут быть проверены без создания графического OpenGL-контекста: ядра компонентной модели, подсистемы ввода, менеджера сцен и физической подсистемы. Подсистемы, зависящие от окна, OpenGL-ресурсов и действий пользователя, дополнительно проверяются системным тестированием демонстрационных сцен.

Автоматизированные тесты реализованы как Java-классы с методом main. При нарушении ожидаемого результата тест выбрасывает AssertionError. Для ядра компонентной модели дополнительно приведены функциональные тестовые наборы, которые фиксируют проверяемые сценарии работы сущностей, компонентов и запросов. Такой формат позволяет воспроизводимо проверять критичные сценарии работы подсистем и не требует создания графического окна для проверяемых модулей.

4.2.1 Тестирование ядра компонентной модели

Так как основой разработанной программной системы является компонентная модель, отдельно проверялись сценарии работы сущностей, компонентов, запросов и жизненного цикла сцены. Данные проверки не требуют графического окна и показывают, что пользователь может строить интерактивное приложение из самостоятельных объектов, компонентов и систем обработки. Тестовые наборы ядра компонентной модели приведены в таблице 4.11.

Таблица 4.11 – Тестовые наборы ядра компонентной модели

Описание теста	Входные данные	Эталонный результат
Добавление компонента к сущности	Создается сущность Entity, после чего к ней добавляется компонент Transform.	Сущность содержит добавленный компонент, а компонент получает ссылку на свою сущность.
Получение компонента по типу	После добавления Transform вызывается получение компонента по его классу.	Возвращается тот же экземпляр компонента, который был добавлен к сущности.
Проверка наличия компонента	Для сущности с Transform выполняется проверка наличия Transform и MeshRenderer.	Проверка для Transform возвращает положительный результат, проверка для отсутствующего компонента возвращает отрицательный результат.
Удаление компонента	Из сущности удаляется ранее добавленный компонент Transform.	Компонент больше не возвращается при запросе по типу, а сущность исключается из запросов, которым требуется Transform.
Обновление запроса сущностей	В сцене создается запрос на сущности с компонентами Transform и MeshRenderer; затем к сущности последовательно добавляются эти компоненты.	Сущность попадает в выборку только после появления полного набора компонентов.
Отложенное добавление сущности	Во время выполнения цикла сцены система или скрипт запрашивает добавление новой сущности.	Структура сцены не изменяется во время небезопасного обхода, новая сущность появляется после применения отложенных изменений.

Продолжение таблицы 4.11

Описание теста	Входные данные	Эталонный результат
Отложенное удаление сущности	Во время выполнения цикла сцены система или скрипт запрашивает удаление существующей сущности.	Сущность удаляется после завершения текущего обхода, связанные компоненты проходят завершающие этапы жизненного цикла.
Вызов жизненного цикла компонента	К сущности добавляется компонент, затем сцена запускается, обновляется и уничтожается.	Для компонента последовательно вызываются методы присоединения, запуска, остановки и уничтожения в соответствии с состоянием сцены.

4.2.2 Тестирование подсистемы ввода

Тестирование подсистемы ввода выполняется классом `InputStateTest`. Проверяется накопление смещения мыши, сохранение позиции курсора, сброс покадровых данных и накопление прокрутки. Тестовые наборы приведены в таблице 4.12.

Таблица 4.12 – Тестовые наборы подсистемы ввода

Описание теста	Входные данные	Эталонный результат
Первичная установка позиции курсора	Вызов <code>setMousePosition(100, 50)</code> для нового объекта <code>InputState</code> .	Позиция курсора равна (100, 50), смещение мыши равно (0, 0), так как предыдущая позиция еще неизвестна.
Накопление смещения курсора в пределах кадра	Последовательные вызовы <code>setMousePosition(110, 45)</code> и <code>setMousePosition(125, 40)</code> .	Смещение мыши за кадр равно (25, -10).

Продолжение таблицы 4.12

Описание теста	Входные данные	Эталонный результат
Сброс покадрового смещения после обновления	Вызов <code>update()</code> после накопления смещения.	Смещение мыши становится (0, 0), текущая позиция остается (125, 40).
Сохранение отслеживания позиции после обновления	После <code>update()</code> выполняется <code>setMousePosition(130, 35)</code> .	Смещение мыши равно (5, -5), так как предыдущая позиция была сохранена.
Сброс отслеживания позиции при повторном входе курсора	Вызовы <code>resetMousePositionTracking()</code> и <code>setMousePosition(400, 300)</code> .	Смещение мыши равно (0, 0), новая позиция считается первой после сброса.
Накопление прокрутки мыши	Вызовы <code>setScrollOffset(0, 1)</code> и <code>setScrollOffset(0, 2)</code> .	Значение прокрутки равно (0, 3).
Сброс прокрутки после обновления	Вызов <code>update()</code> после накопления прокрутки.	Значение прокрутки сбрасывается в (0, 0).

4.2.3 Тестирование менеджера сцен

Тестирование менеджера сцен выполняется классом `SceneManagerTest`. В тесте используется вспомогательная система `ProbeSystem`, которая фиксирует факт запуска и уничтожения сцены. Тестовые наборы приведены в таблице 4.13.

Таблица 4.13 – Тестовые наборы менеджера сцен

Описание теста	Входные данные	Эталонный результат
Загрузка первой сцены	Создаются две сцены. В менеджер загружается первая сцена методом <code>load</code> .	Активной сценой становится первая сцена, ее система переходит в состояние запуска.

Продолжение таблицы 4.13

Описание теста	Входные данные	Эталонный результат
Запрос отложенной замены сцены	После загрузки первой сцены вызывается <code>requestReplace(second)</code> .	Активная сцена не меняется до вызова <code>applyPendingChanges</code> .
Применение отложенной замены сцены	После запроса замены вызывается <code>applyPendingChanges</code> .	Активной становится вторая сцена, предыдущая сцена уничтожается, система новой сцены запускается.
Освобождение активной сцены	После проверки вызывается <code>disposeActiveScene</code> .	Активная сцена корректно уничтожается, связанные системы получают вызовы завершения жизненного цикла.

4.2.4 Тестирование физической подсистемы

Тестирование физической подсистемы выполняется классом `PhysicsSystemTest`. Тесты создают сцены без графического окна, добавляют в них `PhysicsSystem`, сущности с `Transform`, `Rigidbody` и `BoxCollider`, после чего многократно вызывают `scene.loop` с фиксированным шагом времени.

Тестовые наборы физической подсистемы приведены в таблице 4.14.

Таблица 4.14 – Тестовые наборы физической подсистемы

Описание теста	Входные данные	Эталонный результат
Падение и остановка куба на полу	Динамический куб расположен над статическим полом, гравитация равна $(0, -10, 0)$, выполняется 240 физических кадров.	Куб находится на высоте, близкой к половине размера, вертикальная скорость близка к нулю, тело переходит в состояние сна.

Продолжение таблицы 4.14

Описание теста	Входные данные	Эталонный результат
Пробуждение тела после удаления опоры	После перехода куба в сон из сцены удаляется статический пол.	Тело выходит из состояния сна и начинает падать вниз.
Учет масштаба коллайдера	Статический пол создан через масштабированный Transform и единственный BoxCollider.	Динамический куб корректно останавливается на масштабированной поверхности.
Возникновение вращения при смещенном контакте	Куб расположен частично над узкой опорой, контакт происходит не под центром массы.	У тела появляется заметный поворот, что подтверждает учет момента силы при контакте.
Падение с края узкой опоры	Куб размещается так, что значительная часть тела не имеет поддержки.	Тело не засыпает на краю и падает вниз с опоры.
Затухание вращения куба на полу	Куб имеет начальную угловую скорость и находится на статическом полу.	Через заданное число шагов угловая скорость уменьшается, объект не уходит далеко от начальной области.
Отсутствие искусственного вращения у покоящегося куба	Куб находится на плоском полу без начальной угловой скорости.	Тело не получает значительной угловой скорости и не смещается по плоскости.
Падение нависающего куба на пол	Куб расположен на краю узкой опоры, под опорой находится общий пол.	Куб не остается приклеенным к опоре и начинает падать на нижний пол.
Затухание вращения вокруг вертикальной оси	Куб имеет начальную угловую скорость вокруг оси Y.	Угловая скорость постепенно уменьшается, а смещение по плоскости остается ограниченным.

Продолжение таблицы 4.14

Описание теста	Входные данные	Эталонный результат
Контакт со стеной без прилипания	Куб соприкасается со статической вертикальной стеной и падает под действием гравитации.	Куб не прилипает к стене и продолжает движение вниз.
Стабилизация вращающегося куба в углу	Куб находится рядом с полом и двумя стенами, имеет начальную угловую скорость.	Угловая скорость уменьшается, объект остается внутри ограниченной области сцены.

Результаты модульного, функционального и регрессионного тестирования показывают, что проверенные подсистемы выполняют ожидаемые функции: ядро компонентной модели корректно связывает сущности, компоненты и запросы, состояние ввода обновляется по кадрам, менеджер сцен безопасно выполняет отложенную замену, а физическая подсистема обрабатывает типовые и граничные сценарии столкновений коробочных коллайдеров.

4.3 Системное тестирование программной системы

Системное тестирование выполнялось путем запуска четырех демонстрационных сцен, реализованных в пакете `common.showcase`. В отличие от модульных тестов, системное тестирование проверяет совместную работу приложения, окна, ввода, сцен, компонентов, систем обработки, графики, ресурсов и пользовательских скриптов.

Для каждой сцены проверяются запуск приложения, отображение объектов, реакция на пользовательский ввод и выполнение функций, которые данная сцена должна демонстрировать. В таблицах приведены ожидаемые и фактически наблюдаемые результаты проверки.

4.3.1 Проверка сцены ShowcaseScene

Сцена ShowcaseScene предназначена для проверки базового трехмерного рендеринга и основных возможностей сцены: работы камеры, иерархии трансформаций, направленного света, слоев отображения и отладочной геометрии. Место для скриншота сцены приведено на рисунке 4.1.

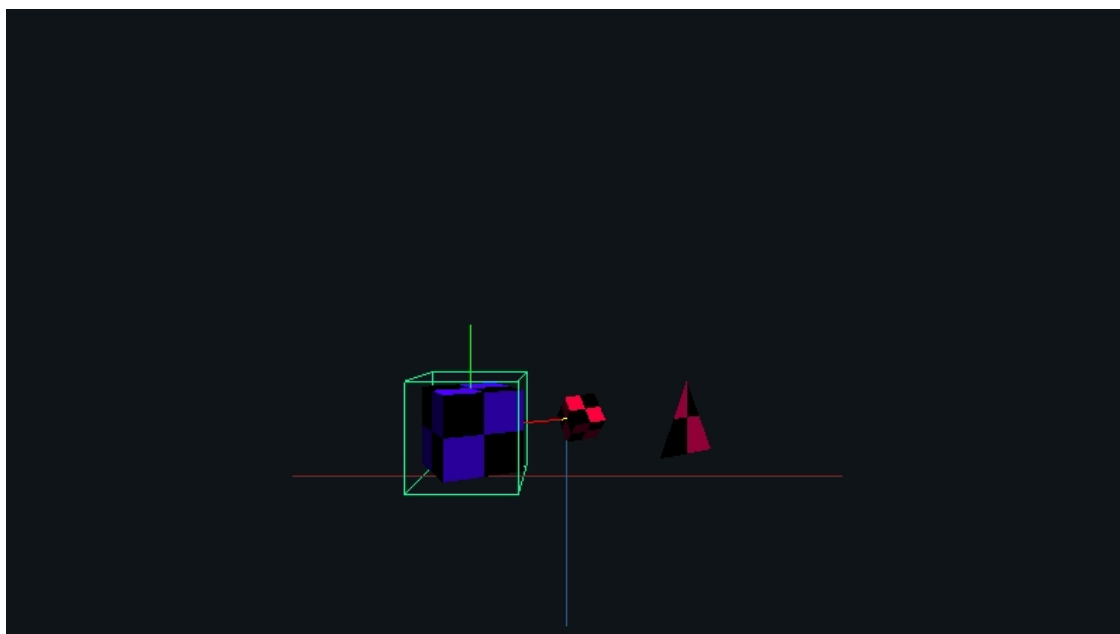


Рисунок 4.1 – Демонстрационная сцена ShowcaseScene

Системные тесты сцены ShowcaseScene приведены в таблице 4.15.

Таблица 4.15 – Системное тестирование сцены ShowcaseScene

Сценарий	Ожидаемый результат	Фактический результат
Запуск класса common.showcase.ShowcaseScene.	Создается окно приложения, загружается сцена, отображаются трехмерные объекты.	Окно приложения создано, сцена загружена, на экране отображаются кубы, пол и отладочные направляющие.

Продолжение таблицы 4.15

Сценарий	Ожидаемый результат	Фактический результат
Осмотр сцены свободной камерой.	При использовании клавиш перемещения и мыши камера меняет положение и направление обзора.	Камера перемещается по сцене и поворачивается вслед за движением мыши, обзор объектов меняется без перезапуска сцены.
Проверка иерархии трансформаций.	Дочерний куб движется относительно родительского объекта, сохраняя связь с ним.	Дочерний куб сохраняет положение относительно родительского объекта и изменяет мировое положение вместе с ним.
Проверка отладочной визуализации.	В сцене отображаются направляющие линии и вспомогательная геометрия, формируемая DebugRenderer.	Отладочные линии отображаются поверх основной сцены и обновляются при изменении положения объектов.
Проверка слоев и отсечения объектов.	Объекты, исключенные маской камеры или находящиеся за дальнейюю плоскость отсечения, не отображаются в основной сцене.	Объекты с неподходящим слоем и объекты за пределами области видимости камеры не попадают в отображаемую часть сцены.

4.3.2 Проверка сцены **Interactive2DScene**

Сцена **Interactive2DScene** предназначена для проверки ортографической камеры, преобразования координат курсора в пространство сцены и работы пользовательских скриптов интерактивных объектов. Место для скриншота сцены приведено на рисунке 4.2.

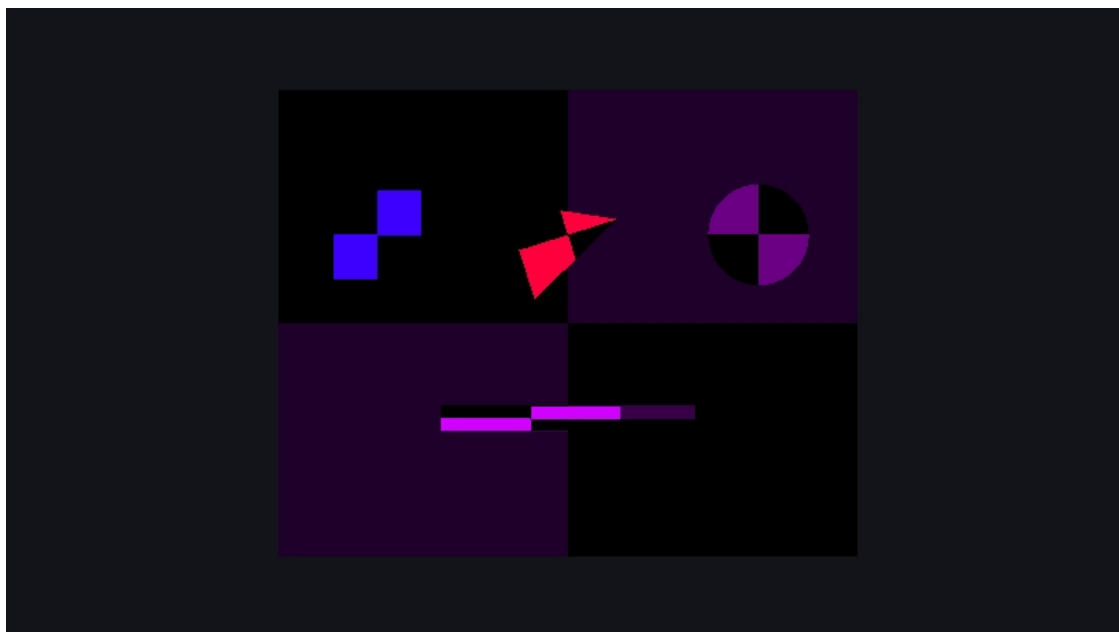


Рисунок 4.2 – Демонстрационная сцена Interactive2DScene

Системные тесты сцены Interactive2DScene приведены в таблице 4.16.

Таблица 4.16 – Системное тестирование сцены Interactive2DScene

Сценарий	Ожидаемый результат	Фактический результат
Запуск класса common.showcase.Interactive2DScene	Создается окно приложения, отображается двумерная сцена с ортографической камерой и интерактивными объектами.	Окно создано, отображаются фон, кнопка, переключатель, слайдер и остальные объекты двумерной сцены.
Перемещение курсора над объектами сцены.	Скрипт отслеживания мыши корректно получает положение курсора в координатах сцены.	При перемещении курсора скрипт получает обновленные координаты в пространстве сцены, что используется для проверки наведения.
Нажатие на кнопку изменения цвета.	Визуальное состояние соответствующего объекта изменяется после клика мышью.	После клика объект меняет цвет, изменение сохраняется до следующего действия пользователя.

Продолжение таблицы 4.16

Сценарий	Ожидаемый результат	Фактический результат
Нажатие на переключатель вращения.	Объект-переключатель изменяет режим поведения, вращение включается или отключается.	После нажатия переключатель меняет состояние, а связанный объект начинает вращаться или останавливается.
Взаимодействие со слайдером.	Заполненная часть слайдера меняет масштаб в зависимости от положения курсора и действия пользователя.	При перетаскивании указателя заполненная часть слайдера изменяет размер пропорционально положению курсора.

4.3.3 Проверка сцены `TexturedLightingScene`

Сцена `TexturedLightingScene` предназначена для проверки материалов, процедурно созданных текстур, передачи текстурных данных в OpenGL и работы анимированного источника направленного света. Место для скриншота сцены приведено на рисунке 4.3.

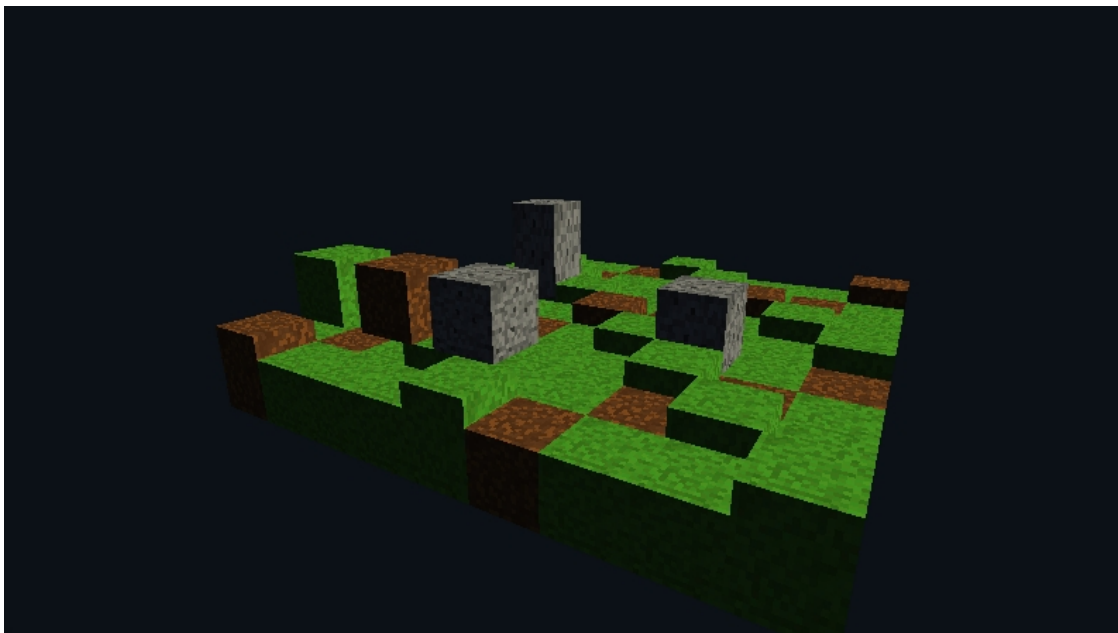


Рисунок 4.3 – Демонстрационная сцена TexturedLightingScene

Системные тесты сцены TexturedLightingScene приведены в таблице 4.17.

Таблица 4.17 – Системное тестирование сцены TexturedLightingScene

Сценарий	Ожидаемый результат	Фактический результат
Запуск класса common.showcase. TexturedLightingScene.	Создается окно приложения, отображается сцена с несколькими текстурированными объектами.	Окно создано, сцена загружена, на объектах видны разные текстуры и освещенные грани.
Проверка процедурных текстур.	На объектах отображаются разные визуальные текстуры, созданные программно.	Объекты получают отличающиеся процедурные изображения, повторной загрузки файлов текстур не требуется.
Проверка направленного освещения.	Освещение влияет на видимые грани объектов, сцена не выглядит плоской.	Грани объектов имеют разную яркость в зависимости от направления света и нормалей поверхности.

Продолжение таблицы 4.17

Сценарий	Ожидаемый результат	Фактический результат
Проверка анимации источника света.	Скрипт OrbitingLightScript изменяет параметры света во времени, что визуально влияет на освещение сцены.	В процессе работы сцены направление света изменяется, освещенные и затененные стороны объектов плавно меняются.
Проверка освобождения текстур.	При завершении сцены скрипт очистки и контекст приложения освобождают созданные ресурсы.	При закрытии сцены выполняется очистка процедурных текстур и связанных OpenGL-ресурсов без аварийного завершения.

4.3.4 Проверка сцены PhysicsScene

Сцена PhysicsScene предназначена для комплексной проверки физической подсистемы и ее взаимодействия с рендерингом, вводом, скриптами, камерой и отладочной визуализацией. В сцене присутствуют динамические тела, статическое окружение, узкая опора, кинематическая платформа, отладочная сетка и перезапуск по клавише R.

Место для основного скриншота сцены приведено на рисунке 4.4.

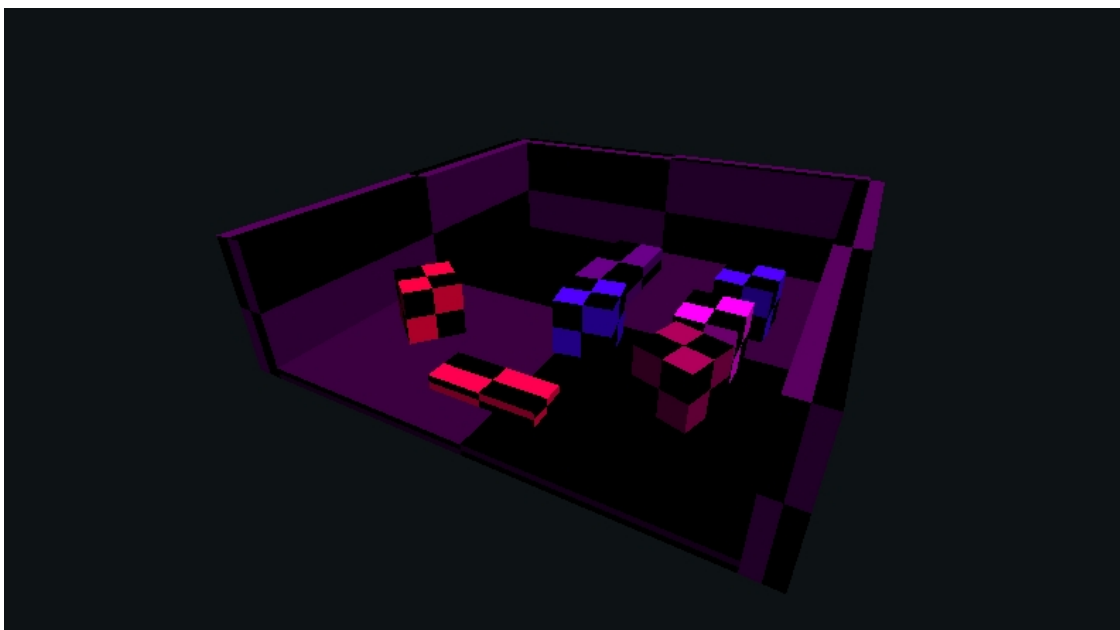


Рисунок 4.4 – Демонстрационная сцена PhysicsScene

Место для скриншота отладочной визуализации физической сцены приведено на рисунке 4.5.

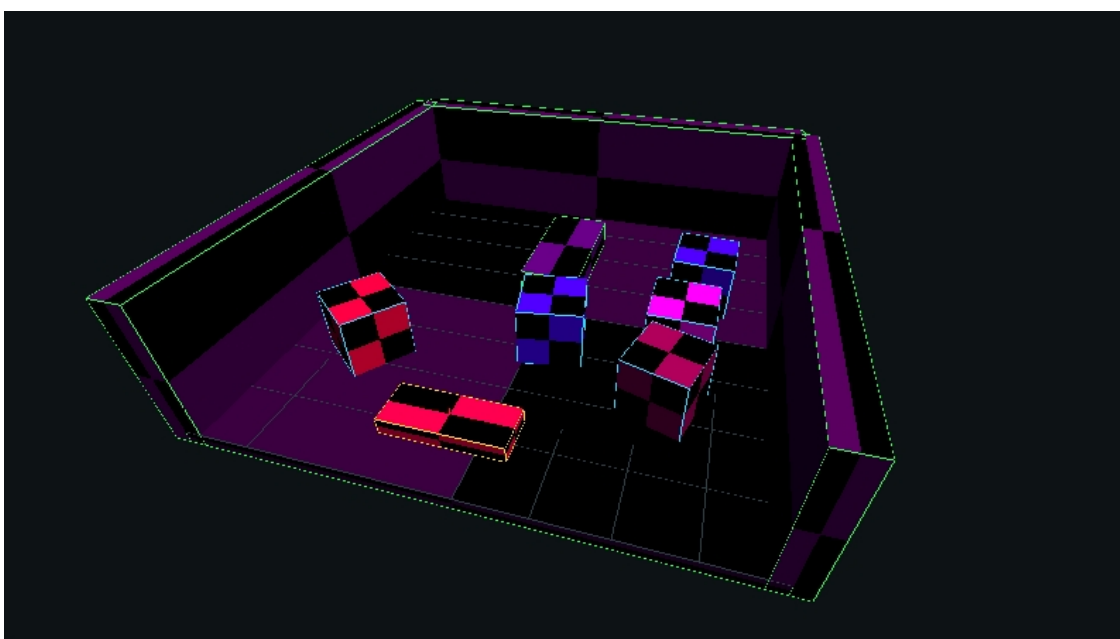


Рисунок 4.5 – Отладочная визуализация коллайдеров в сцене PhysicsScene

Системные тесты сцены PhysicsScene приведены в таблице 4.18.

Таблица 4.18 – Системное тестирование сцены PhysicsScene

Сценарий	Ожидаемый результат	Фактический результат
Запуск класса common.showcase.PhysicsScene.	Создается окно приложения, загружается физическая сцена, отображаются пол, стены, платформа и динамические кубы.	Окно приложения создано, физическая сцена загружена, видны статические объекты, платформа и набор динамических кубов.
Наблюдение за падением динамических кубов.	Кубы падают под действием гравитации и останавливаются на полу, опоре или платформе.	Динамические кубы перемещаются вниз, сталкиваются с опорными поверхностями и после затухания движения переходят в состояние покоя.
Проверка движущейся кинематической платформы.	Платформа перемещается между заданными границами, меняет направление и взаимодействует с динамическими объектами.	Платформа периодически меняет направление движения, а находящиеся рядом динамические объекты реагируют на контакт с ней.
Проверка столкновения со статическими объектами.	Динамические объекты сталкиваются с полом, стенами и узкой опорой, не проходят сквозь них и получают реалистичное изменение движения.	Кубы не проходят сквозь пол, стены и узкую опору; при смещенном контакте появляется падение или вращение.
Проверка отладочной сетки и визуализации коллайдеров.	В сцене отображается вспомогательная сетка и границы коллайдеров, позволяющие визуально сопоставить физические формы с графическими объектами.	Отладочная сетка и границы коллайдеров отображаются в сцене и визуально совпадают с положением физических объектов.

Продолжение таблицы 4.18

Сценарий	Ожидаемый результат	Фактический результат
Перезапуск сцены по клавише R.	При нажатии клавиши выполняется запрос замены активной сцены, объекты возвращаются в исходные положения.	После нажатия клавиши R активная сцена заменяется новой, динамические объекты возвращаются в начальные положения.

ЗАКЛЮЧЕНИЕ

В процессе выполнения выпускной квалификационной работы была разработана компонентно-ориентированная программная система для построения интерактивных приложений на языке Java. Разработанная система предоставляет программную основу, которая позволяет создавать сцены, добавлять в них сущности и компоненты, подключать системы обработки, выполнять пользовательские сценарии поведения, обрабатывать ввод, отображать графические объекты и использовать базовые физические взаимодействия.

При выполнении работы основной акцент был сделан на том, чтобы система не сводилась к набору отдельных примеров, а имела общую архитектуру, пригодную для расширения. Объекты сцены представлены как сущности, поведение и состояние которых задаются через независимые компоненты. Обработка таких объектов выполняется специализированными системами. Это позволило разделить разные части логики приложения: управление сценой, рендеринг, обработку ввода, выполнение пользовательских скриптов, отладочную визуализацию и физическую обработку.

Основные результаты работы:

1. Проведен анализ предметной области. Рассмотрены интерактивные приложения как класс программных систем, особенности их жизненного цикла, задачи обработки пользовательского ввода, обновления состояния объектов и вывода кадра. Изучены существующие программные средства построения интерактивных приложений, включая низкоуровневые библиотеки, фреймворки и более крупные готовые решения. На основе анализа обоснована целесообразность разработки промежуточной программной основы, ориентированной на Java-разработчика.

2. Сформулированы требования к компонентно-ориентированной программной системе. Определены назначение разработки, требования к архитектурной модели, управлению приложением и сценами, графической подсистеме, пользовательскому вводу, системе пользовательских скриптов, ба-

зовой физической подсистеме, демонстрационным сценам и программному интерфейсу. Также описан порядок использования системы разработчиком при создании собственной интерактивной сцены.

3. Спроектирована архитектура программной системы. В качестве технологической основы выбраны Java, LWJGL, OpenGL, GLSL, GLFW и STB. Определено внутреннее устройство приложения, контекста выполнения, сцены, сущностей, компонентов, систем обработки, запросов сущностей и ресурсов. Отдельно спроектированы графическая подсистема, подсистема пользовательского ввода, подсистема пользовательских скриптов, отладочная визуализация и физическая подсистема.

4. Реализованы основные классы и компоненты программной системы. В составе реализации представлены ядро компонентной модели, классы управления приложением и сценами, математические классы, компоненты трансформации, камеры, графического представления, освещения, пользовательских скриптов, физических тел и коллайдеров. Реализованы системы обработки, отвечающие за выполнение скриптов, отрисовку сцены, отладочную визуализацию и базовую физическую обработку.

5. Реализованы демонстрационные сцены, показывающие разные варианты использования разработанной системы. Сцена ShowcaseScene демонстрирует базовую трехмерную сцену, камеру, освещение и отладочные элементы. Interactive2DScene показывает обработку пользовательского ввода и работу интерактивных объектов. TexturedLightingScene демонстрирует использование материалов, текстур и направленного освещения. PhysicsScene проверяет совместную работу физической подсистемы, рендеринга, скриптов, ввода и отладочной визуализации.

6. Проведено модульное, регрессионное и системное тестирование разработанной программной системы. Были проверены ядро компонентной модели, подсистема ввода, менеджер сцен и физическая подсистема. Системное тестирование демонстрационных сцен подтвердило корректность создания окна приложения, загрузки сцен, обработки пользовательских действий,

отображения объектов, работы освещения, текстур, физических тел и перезапуска сцены.

Поставленная во введении цель достигнута. Разработанная программная система позволяет создавать простые интерактивные приложения на Java без повторной реализации базовых механизмов управления сценой, обработки ввода, организации компонентов, рендеринга и физической обработки. Задачи, сформулированные в начале работы, выполнены в полном объеме.

Практическая ценность результата заключается в том, что разработанная система может использоваться как основа для учебных, демонстрационных и исследовательских интерактивных приложений. Она также может быть полезна для изучения внутреннего устройства компонентной архитектуры, так как основные механизмы работы сцены, компонентов, систем обработки, графики и физики реализованы явно и доступны для анализа и модификации.

Перспективами дальнейшего развития являются расширение графической подсистемы, добавление новых типов материалов и источников света, поддержка анимации, развитие физической модели, сохранение и загрузка описания сцен, расширение набора готовых компонентов и создание более подробной пользовательской документации. В дальнейшем на основе разработанного ядра может быть создана более полнофункциональная программная платформа для построения интерактивных приложений и игровых прототипов на языке Java.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Грегори, Дж. Игровой движок. Программирование и внутреннее устройство / Дж. Грегори. – 3-е изд. – Санкт-Петербург : Питер, 2021. – 1136 с. – ISBN 978-5-4461-1134-3. – Текст : непосредственный.
2. Нистром, Р. Паттерны программирования игр / Р. Нистром. – Москва : Эксмо, 2021. – 432 с. – ISBN 978-5-04-102290-7. – Текст : непосредственный.
3. Гамма, Э. Приемы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес. – Санкт-Петербург : Питер, 2020. – 366 с. – ISBN 978-5-4461-1595-0. – Текст : непосредственный.
4. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. – Санкт-Петербург : Питер, 2018. – Текст : непосредственный.
5. Фаулер, М. Рефакторинг. Улучшение проекта существующего кода / М. Фаулер. – Санкт-Петербург : Диалектика, 2019. – Текст : непосредственный.
6. Блох, Дж. Java. Эффективное программирование / Дж. Блох. – Санкт-Петербург : Диалектика, 2019. – Текст : непосредственный.
7. Хорстманн, К. Java. Библиотека профессионала. Том 1. Основы / К. Хорстманн. – Санкт-Петербург : Диалектика, 2022. – Текст : непосредственный.
8. Шилдт, Г. Java. Полное руководство / Г. Шилдт. – Санкт-Петербург : Диалектика, 2023. – Текст : непосредственный.
9. Ларман, К. Применение UML 2.0 и шаблонов проектирования / К. Ларман. – Москва : Вильямс, 2006. – Текст : непосредственный.
10. Буч, Г. Язык UML. Руководство пользователя / Г. Буч, Дж. Рамбо, И. Якобсон. – 2-е изд. – Москва : ДМК Пресс, 2007. – Текст : непосредственный.

11. Фаулер, М. UML. Основы / М. Фаулер. – 3-е изд. – Санкт-Петербург : Символ-Плюс, 2004. – Текст : непосредственный.
12. Соммервилл, И. Инженерия программного обеспечения / И. Соммервилл. – 10-е изд. – Москва : Вильямс, 2018. – Текст : непосредственный.
13. Майерс, Г. Искусство тестирования программ / Г. Майерс, К. Сандлер, Т. Баджетт. – 3-е изд. – Москва : Диалектика, 2012. – Текст : непосредственный.
14. Прессман, Р. Программная инженерия. Практический подход / Р. Прессман. – 8-е изд. – Москва : Вильямс, 2016. – Текст : непосредственный.
15. Eberly, D. H. 3D Game Engine Design : A Practical Approach to Real-Time Computer Graphics / D. H. Eberly. – 2nd ed. – Boca Raton : CRC Press, 2006. – 1040 p. – ISBN 978-0-12-229063-3. – Текст : непосредственный.
16. Akenine-Moller, T. Real-Time Rendering / T. Akenine-Moller, E. Haines, N. Hoffman. – 4th ed. – Boca Raton : CRC Press, 2018. – 1198 p. – ISBN 978-1-138-62700-0. – Текст : непосредственный.
17. Lengyel, E. Mathematics for 3D Game Programming and Computer Graphics / E. Lengyel. – 3rd ed. – Boston : Course Technology, 2012. – 576 p. – ISBN 978-1-4354-5886-4. – Текст : непосредственный.
18. Lengyel, E. Foundations of Game Engine Development. Volume 2 : Rendering / E. Lengyel. – Lincoln : Terathon Software, 2019. – Текст : непосредственный.
19. Shirley, P. Fundamentals of Computer Graphics / P. Shirley, S. Marschner. – 4th ed. – Boca Raton : CRC Press, 2016. – 784 p. – ISBN 978-1-4822-2939-4. – Текст : непосредственный.
20. Hughes, J. F. Computer Graphics : Principles and Practice / J. F. Hughes, A. van Dam, M. McGuire, D. F. Sklar, J. D. Foley, S. K. Feiner, K. Akeley. – 3rd ed. – Boston : Addison-Wesley, 2013. – Текст : непосредственный.
21. Ericson, C. Real-Time Collision Detection / C. Ericson. – San Francisco : Morgan Kaufmann, 2005. – 632 p. – ISBN 978-1-55860-732-3. – Текст : непосредственный.

22. Millington, I. Game Physics Engine Development / I. Millington. – 2nd ed. – Boca Raton : CRC Press, 2010. – 542 p. – ISBN 978-0-12-381976-5. – Текст : непосредственный.
23. Dunn, F. 3D Math Primer for Graphics and Game Development / F. Dunn, I. Parberry. – 2nd ed. – Boca Raton : CRC Press, 2011. – 846 p. – ISBN 978-1-56881-723-1. – Текст : непосредственный.
24. van den Bergen, G. Collision Detection in Interactive 3D Environments / G. van den Bergen. – San Francisco : Morgan Kaufmann, 2004. – 278 p. – ISBN 978-1-55860-801-6. – Текст : непосредственный.
25. Mirtich, B. Impulse-based Dynamic Simulation of Rigid Body Systems : диссертация / B. Mirtich. – Berkeley : University of California, 1996. – Текст : непосредственный.
26. Catto, E. Iterative Dynamics with Temporal Coherence : доклад / E. Catto. – Game Developers Conference, 2005. – Текст : непосредственный.
27. Gottschalk, S. OBBTree : A Hierarchical Structure for Rapid Interference Detection / S. Gottschalk, M. C. Lin, D. Manocha // Proceedings of SIGGRAPH. – 1996. – Текст : непосредственный.
28. Bilas, S. A Data-Driven Game Object System : доклад / S. Bilas. – Game Developers Conference, 2002. – Текст : непосредственный.
29. West, M. Evolve Your Hierarchy : сайт / M. West. – URL: <https://cowboyprogramming.com/2007/01/05/evolve-your-heirachy/> (дата обращения: 01.06.2026).
30. Artemis-ODB Documentation : сайт. – URL: <https://github.com/junkdog/artemis-odb/wiki> (дата обращения: 01.06.2026).
31. Ashley Entity System Documentation : сайт. – URL: <https://github.com/libgdx/ashley/wiki> (дата обращения: 01.06.2026).
32. EnTT Documentation : сайт. – URL: <https://skypjack.github.io/entt/> (дата обращения: 01.06.2026).
33. Flecs Documentation : сайт. – URL: <https://www.flecs.dev/flecs/> (дата обращения: 01.06.2026).

34. Java Platform, Standard Edition and Java Development Kit. Version 25 API Specification : сайт. – URL: <https://docs.oracle.com/en/java/javase/25/docs/api/index.html> (дата обращения: 17.05.2026).
35. The Java Language Specification. Java SE 25 Edition : сайт. – URL: <https://docs.oracle.com/javase/specs/jls/se25/html/index.html> (дата обращения: 17.05.2026).
36. The Java Virtual Machine Specification. Java SE 25 Edition : сайт. – URL: <https://docs.oracle.com/javase/specs/jvms/se25/html/index.html> (дата обращения: 17.05.2026).
37. LWJGL – Lightweight Java Game Library : сайт. – URL: <https://www.lwjgl.org/> (дата обращения: 17.05.2026).
38. GLFW Documentation : сайт. – URL: <https://www.glfw.org/docs/> (дата обращения: 17.05.2026).
39. OpenGL – The Industry’s Foundation for High Performance Graphics : сайт. – URL: <https://www.khronos.org/opengl/> (дата обращения: 17.05.2026).
40. The OpenGL Graphics System. Version 3.3 (Core Profile) : сайт. – URL: <https://registry.khronos.org/OpenGL/specs/gl/glspec33.core.pdf> (дата обращения: 01.06.2026).
41. OpenGL Wiki : сайт. – URL: <https://wikis.khronos.org/opengl/> (дата обращения: 01.06.2026).
42. Kessenich, J. The OpenGL Shading Language. Language Version 3.30 / J. Kessenich, D. Baldwin, R. Rost : сайт. – URL: <https://registry.khronos.org/OpenGL/specs/gl/GLSLangSpec.3.30.pdf> (дата обращения: 17.05.2026).
43. stb single-file public domain libraries for C/C++ : сайт. – URL: <https://github.com/nothings/stb> (дата обращения: 17.05.2026).
44. libGDX – Cross-platform Java game development framework : сайт. – URL: <https://libgdx.com/> (дата обращения: 17.05.2026).
45. jMonkeyEngine : сайт. – URL: <https://jmonkeyengine.org/> (дата обращения: 17.05.2026).

46. Processing : сайт. – URL: <https://processing.org/> (дата обращения: 17.05.2026).
47. Unity Manual. GameObjects and components : сайт. – URL: <https://docs.unity3d.com/Manual/GameObject.html> (дата обращения: 01.06.2026).
48. Unity Entity Component System : сайт. – URL: <https://unity.com/ecs> (дата обращения: 17.05.2026).
49. PlantUML Documentation : сайт. – URL: <https://plantuml.com/> (дата обращения: 17.05.2026).
50. Unified Modeling Language 2.5.1 Specification : сайт. – URL: <https://www.omg.org/spec/UML/2.5.1/> (дата обращения: 17.05.2026).
51. Bejarano, A. H. 3D Game Development with LWJGL 3 / A. H. Bejarano. – Berkeley : Apress, 2017. – 344 p. – ISBN 978-1-4842-3236-1. – Текст : непосредственный.
52. Gordon, V. S. Computer Graphics Programming in OpenGL with Java / V. S. Gordon, J. L. Clevenger. – Dulles : Mercury Learning and Information, 2018. – Текст : непосредственный.
53. Shreiner, D. OpenGL Programming Guide : The Official Guide to Learning OpenGL / D. Shreiner, G. Sellers, J. Kessenich, B. Licea-Kane. – 8th ed. – Boston : Addison-Wesley, 2013. – 984 p. – ISBN 978-0-321-77303-6. – Текст : непосредственный.
54. Wright, R. S. OpenGL SuperBible : Comprehensive Tutorial and Reference / R. S. Wright, N. Haemel, G. Sellers, B. Lipchak. – 5th ed. – Boston : Addison-Wesley, 2010. – Текст : непосредственный.
55. Box2D Documentation : сайт. – URL: <https://box2d.org/documentation/> (дата обращения: 01.06.2026).

ПРИЛОЖЕНИЕ А

Представление графического материала

Графический материал, выполненный на отдельных листах, изображен на рисунках А.1–А.6.

Сведения о ВКРБ

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА ПО ПРОГРАММЕ БАКАЛАВРИАТА

«Разработка компонентно-ориентированной программной
системы для построения интерактивных приложений на Java»

Руководитель ВКРБ
к.т.н., доцент
Рубанов Алексей Фёдорович

Автор ВКРБ
студент группы ПО-226
Алешин Кирилл Романович

			ВКРБ 2206044.09.03.04.26.002		
Автор работы Руководитель Пересмотрен	Фамилия И. О.	Подпись/Дата	Сведения о ВКРБ		
	Алешин К.Р.				
	Рубанов А.Ф.				
	Алешин К.Р.				
			Выпускная квалификационная работа бакалавра		
			03/У ПО-226		

Рисунок А.1 – Сведения о ВКРБ

Цель и задачи разработки

Целью разработки является создание компонентно ориентированной программной системы, позволяющей Java-разработчику строить простые интерактивные приложения и учебные игровые прототипы без самостоятельной реализации полного набора низкоуровневых механизмов.

Для достижения поставленной цели необходимо решить следующие задачи:

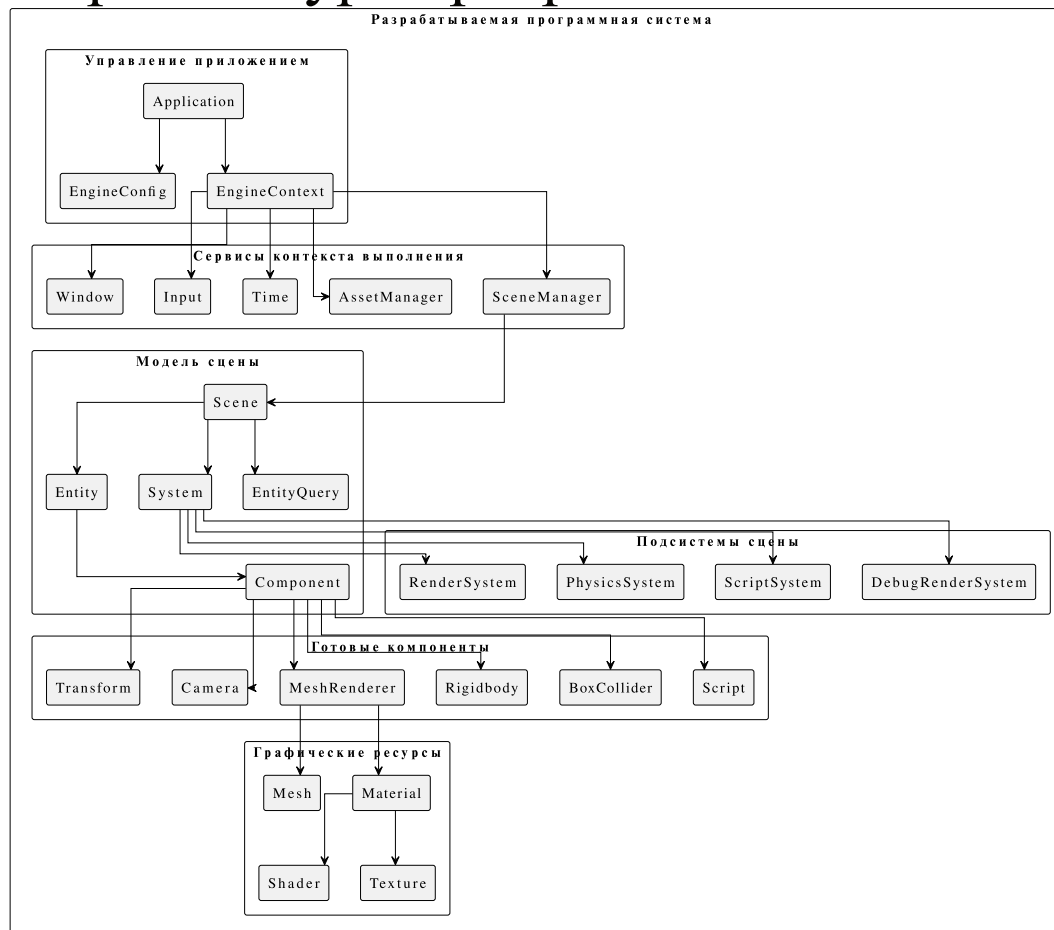
- разработать ядро компонентно-ориентированной модели, включающее сущности, компоненты системы обработки, запросы сущностей и жизненный цикл объектов сцены;
- спроектировать и реализовать подсистему управления приложением и сценами, обеспечивающую создание окна, выполнение основного цикла, обработку времени кадра и безопасное изменение состава сцены во время работы программы;
- реализовать графическую подсистему на основе OpenGL, обеспечивающую отображение объектов сцены с использованием мешей, материалов, текстур, камер и направленного освещения;
- реализовать подсистемы пользовательского ввода, скриптов и базовой физики, позволяющие обрабатывать действия пользователя, задавать поведение объектов средствами Java-кода и демонстрировать взаимодействие физических тел и коллайдеров;
- подготовить набор демонстрационных сцен и описание порядка использования программной системы с примерами кода, показывающими со30 здание приложения, сцены, сущностей, компонентов, систем обработки и пользовательских скриптов;

ВКРБ 2206044.09.03.04.26.002			
Страница 11.11	Листов 11.11	Сведения о ВКРБ	
Адрес работы	Адрес ВУ	Доп. 2	Доп. 3
Регистрация	Регистр ВУ	Выпускная квалификационная работа ВУ	
Пересмотр	Число ВУ	03/У ПО-226	

Рисунок А.2 – Цель и задачи разработки

Архитектура программной системы

Разрабатываемая программная система



ВКРБ 2206044.09.03.04.26.002			
Автор работы	Степанов И. В.	Рецензент	Иванов И. И.
Руководитель	Аликин В. В.	Дата	Лист 3
Перепроверка	Степанов И. В.	Дата	Лист 6
Выпускная квалификационная работа		03/У ПО-226	

Рисунок А.3 – Архитектура программной системы



Рисунок А.4 – Компонентная модель программной системы

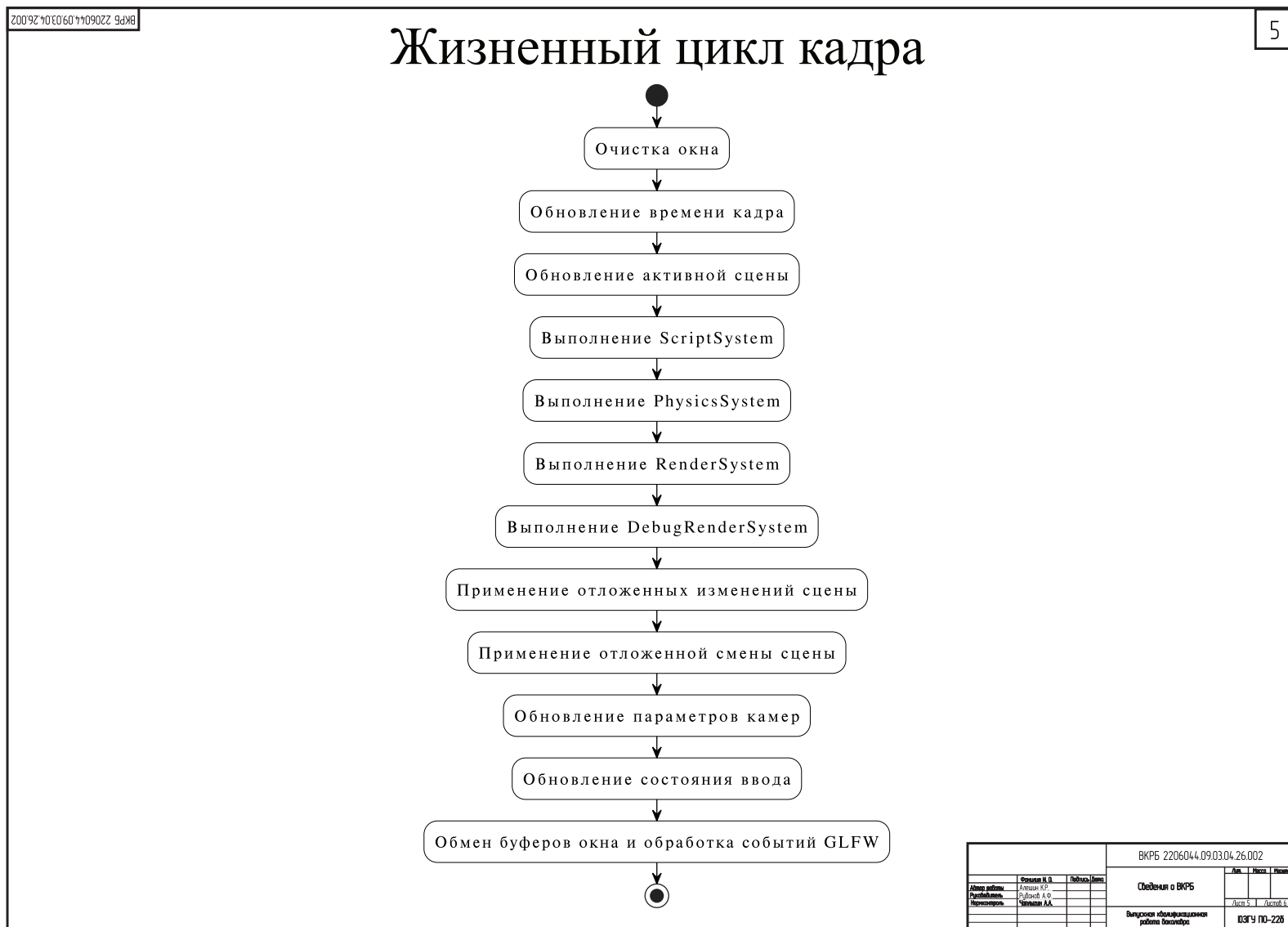


Рисунок А.5 – Жизненный цикл кадра приложения

Заключение

В процессе выполнения выпускной квалификационной работы была разработана компонентно-ориентированная программная система для построения интерактивных приложений на языке Java. Разработанная система предоставляет программную основу, которая позволяет создавать сцены, добавлять в них сущности и компоненты, подключать системы обработки, выполнять пользовательские сценарии поведения, обрабатывать ввод, отображать графические объекты и использовать базовые физические взаимодействия.

Основные результаты работы:

1. Проведен анализ предметной области. Рассмотрены интерактивные приложения как класс программных систем, особенности их жизненного цикла, задачи обработки пользовательского ввода, обновления состояния объектов и вывода кадра.
2. Сформулированы требования к компонентно-ориентированной программной системе.
3. Спроектирована архитектура программной системы. В качестве технологической основы выбраны Java, LWJGL, OpenGL, GLSL, GLFW и STB.
4. Реализованы основные классы и компоненты программной системы. В составе реализации представлены ядро компонентной модели, классы управления приложением и сценами, математические классы, компоненты трансформации, камеры, графического представления, освещения, пользовательских скриптов, физических тел и коллайдеров.
5. Реализованы демонстрационные сцены, показывающие разные варианты использования разработанной системы.
6. Проведено модульное, регрессионное и системное тестирование разработанной программной системы.

ВКРБ 2206044.09.03.04.26.002			
Сведения о ВКРБ		Акт	Исход. I. Исход. II
Автор работы	Алешин К.В.		
Руководитель	Сидоров А.В.	Акт 6	Акт 6
Пересмотр	Сидоров А.А.		
Выпускная квалификационная работа		03/У ПО-226	

Рисунок А.6 – Результаты разработки и тестирования

ПРИЛОЖЕНИЕ Б

Фрагменты исходного кода программы

В приложении приведены основные фрагменты исходного кода разработанной программной системы. Листинги выбраны таким образом, чтобы показать реализацию компонентной модели, жизненного цикла приложения и сцены, графической подсистемы, подсистемы ввода, скриптов, физической подсистемы, демонстрационного использования и тестирования.

Листинг Б.1 – базовый класс компонента Component.java

```
1 package qchromatic.jecse.core;
2
3 public abstract class Component {
4     protected Entity entity;
5     private boolean _started;
6     private boolean _destroyed;
7
8     public Entity entity () { return entity; }
9
10    public boolean started () { return _started; }
11
12    public boolean destroyed () { return _destroyed; }
13
14    public void onAttach () { }
15
16    public void onDetach () { }
17
18    public void onStart () { }
19
20    public void onStop () { }
21
22    public void onDestroy () { }
23
24    final void attach (Entity entity) {
25        if (this.entity == entity)
26            return;
27
28        if (this.entity != null && this.entity != entity)
29            throw new IllegalStateException("Component is already attached to
30                another entity");
31
32        this.entity = entity;
33        _destroyed = false;
34        onAttach();
35    }
36
37    final void detach (Entity entity) {
38        if (this.entity == entity) {
39            stopInternal();
40            onDetach();
41            this.entity = null;
42        }
43    }
44
45    final void startInternal () {
46        if (_started || _destroyed) return;
47
48        _started = true;
49        onStart();
50    }
51
52    final void stopInternal () {
53        if (!_started) return;
```

```
53
54     _started = false;
55     onStop();
56 }
57
58 final void destroyInternal () {
59     if (_destroyed) return;
60
61     stopInternal();
62     _destroyed = true;
63     onDestroy();
64
65     if (this instanceof Disposable disposable)
66         disposable.destroy();
67 }
68 }
```

Листинг Б.2 – базовый класс системы обработки System.java

```
1 package qchromatic.jecse.core;
2
3 import qchromatic.jecse.engine.Scene;
4
5 public abstract class System {
6     protected Scene scene;
7     private int _order;
8
9     public System () { this(0); }
10
11     protected System (int order) {
12         _order = order;
13     }
14
15     public void init () { }
16     public void start () { }
17     public void loop (float dtime) { }
18     public void stop () { }
19
20     public void destroy () { }
21
22     public void onEntityAdded (Entity entity) { }
23     public void onEntityRemoved (Entity entity) { }
24
25     public void onComponentAdded (Entity entity, Component component) { }
26     public void onComponentRemoved (Entity entity, Component component) { }
27
28     public int order () { return _order; }
29
30     public System order (int order) {
31         _order = order;
32         return this;
33     }
34
35     public System scene (Scene scene) {
36         if (scene == null) return this;
37
38         this.scene = scene;
39         return this;
40     }
41 }
```

Листинг Б.3 – сущность и управление компонентами Entity.java

```
1 package qchromatic.jecse.core;
2
3 import java.util.*;
4 import java.util.concurrent.atomic.AtomicLong;
5
6 public class Entity {
7     private static final AtomicLong NEXT_ID = new AtomicLong(1);
8
9     private final String _id;
10    private final Map<Class<? extends Component>, Component> _components;
11
12    private EntityObserver _observer;
13
14    public Entity () { this("entity-" + NEXT_ID.getAndIncrement()); }
15    public Entity (String id) {
16        if (id == null || id.isBlank())
17            throw new IllegalArgumentException("Entity id cannot be null or blank")
18            ;
19
20        _id = id;
21        _components = new LinkedHashMap<>();
22    }
23
24    public String id () { return _id; }
25
26    public void observer (EntityObserver observer) { _observer = observer; }
27
28    public <T extends Component> Entity addComponent (T component) {
29        if (component == null)
30            throw new IllegalArgumentException("Component cannot be null");
31
32        if (component.entity != null && component.entity != this)
33            throw new IllegalStateException("Component is already attached to
34            another entity");
35
36        if (_components.containsKey(component.getClass()))
37            throw new IllegalStateException("Entity already has component: " +
38            component.getClass().getName());
39
40        component.attach(this);
41        _components.put(component.getClass(), component);
42
43        if (_observer != null)
44            _observer.onComponentAdded(this, component);
45
46        return this;
47    }
48
49    public Component[] getAllComponents () {
50        return _components.values().toArray(new Component[0]);
51    }
52
53    public Collection<Component> components () {
```

```

51     return Collections.unmodifiableCollection(_components.values());
52 }
53
54 public void detachAllComponents () {
55     for (Component component : _components.values())
56         component.detach(this);
57 }
58
59 public void attachAllComponents () {
60     for (Component component : _components.values())
61         component.attach(this);
62 }
63
64 public void startComponents () {
65     for (Component component : _components.values())
66         component.startInternal();
67 }
68
69 public void stopComponents () {
70     for (Component component : _components.values())
71         component.stopInternal();
72 }
73
74 public void destroyComponents () {
75     for (Component component : _components.values())
76         component.destroyInternal();
77 }
78
79 public void startComponent (Component component) {
80     if (component != null && _components.containsKey(component))
81         component.startInternal();
82 }
83
84 public <T extends Component> T getComponent (Class<T> componentClass) {
85     if (componentClass == null) return null;
86     return componentClass.cast(_components.get(componentClass));
87 }
88
89 public <T extends Component> List<T> getComponentsInheritance(Class<T>
90     componentClass) {
91     List<T> result = new ArrayList<>();
92     for (Component component : _components.values()) {
93         if (componentClass.isAssignableFrom(component.getClass())) {
94             result.add(componentClass.cast(component));
95         }
96     }
97
98     return result;
99 }
100
101 public boolean hasComponent (Class<? extends Component> componentClass) {
102     if (componentClass == null) return false;
103     return _components.containsKey(componentClass);
104 }

```

```

104
105 public boolean hasComponentInheritance(Class<? extends Component>
    componentClass) {
106     if (componentClass == null) return false;
107
108     for (Component component : _components.values()) {
109         if (componentClass.isAssignableFrom(component.getClass())) {
110             return true;
111         }
112     }
113     return false;
114 }
115
116 public void removeComponent (Class<? extends Component> componentClass) {
117     if (componentClass == null) return;
118
119     Component component = _components.remove(componentClass);
120
121     if (component == null) return;
122
123     if (_observer != null)
124         _observer.onComponentRemoved(this, component);
125     else
126         finishComponentRemoval(component);
127 }
128
129 public void finishComponentRemoval (Component component) {
130     if (component == null) return;
131     component.destroyInternal();
132     component.detach(this);
133 }
134 }

```

Листинг Б.4 – фрагменты управления сценой Scene.java

```
1 public final class Scene {
2     public void init () {
3         ensureNotDisposed();
4         initializeIfNeeded();
5         start();
6     }
7
8     public void start () {
9         ensureNotDisposed();
10        initializeIfNeeded();
11
12        if (_running) return;
13
14        _running = true;
15        for (Entity entity : entitySnapshot())
16            entity.startComponents();
17
18        for (System system : systemSnapshot())
19            system.start();
20
21        flushStructuralChanges();
22    }
23
24    public void loop (float dtime) {
25        if (!_running) return;
26
27        flushStructuralChanges();
28
29        _updating = true;
30        try {
31            for (System system : systemSnapshot())
32                system.loop(dtime);
33        } finally {
34            _updating = false;
35        }
36
37        flushStructuralChanges();
38    }
39
40    public void stop () {
41        if (!_running) return;
42
43        flushStructuralChanges();
44        _running = false;
45
46        for (System system : systemSnapshot())
47            system.stop();
48
49        for (Entity entity : entitySnapshot())
50            entity.stopComponents();
51    }
52
53    public void dispose () {
```

```

54     if (_disposed) return;
55
56     stop();
57
58     for (System system : systemSnapshot())
59         system.destroy();
60
61     for (Entity entity : entitySnapshot())
62         disposeEntity(entity);
63
64     _pendingStructuralChanges.clear();
65     _componentIndex.clear();
66     _queries.clear();
67     _systems.clear();
68     _systemMap.clear();
69     _entities.clear();
70
71     _initialized = false;
72     _disposed = true;
73 }
74
75 public void addEntity (Entity entity) {
76     ensureNotDisposed();
77     if (entity == null)
78         throw new IllegalArgumentException("Entity cannot be null");
79
80     if (_updating || _flushingStructuralChanges) {
81         _pendingStructuralChanges.add(() -> addEntityNow(entity));
82         return;
83     }
84
85     addEntityNow(entity);
86 }
87
88 public void removeEntity (String id) {
89     ensureNotDisposed();
90
91     if (_updating || _flushingStructuralChanges) {
92         _pendingStructuralChanges.add(() -> removeEntityNow(id));
93         return;
94     }
95
96     removeEntityNow(id);
97 }
98
99 public void addSystem (System system) {
100     ensureNotDisposed();
101     if (system == null)
102         throw new IllegalArgumentException("System cannot be null");
103
104     if (_updating || _flushingStructuralChanges) {
105         _pendingStructuralChanges.add(() -> addSystemNow(system));
106         return;
107     }

```



```

108
109     addSystemNow(system);
110 }
111
112 public void removeSystem (Class<? extends System> systemClass) {
113     ensureNotDisposed();
114
115     if (_updating || _flushingStructuralChanges) {
116         _pendingStructuralChanges.add(() -> removeSystemNow(systemClass));
117         return;
118     }
119
120     removeSystemNow(systemClass);
121 }
122
123 public final EntityQuery query (Class<? extends Component>...
124     componentClasses) {
125     return createQuery(false, Arrays.asList(componentClasses));
126 }
127
128 public final EntityQuery queryInheritance (Class<? extends Component
129     >... componentClasses) {
130     return createQuery(true, Arrays.asList(componentClasses));
131 }
132
133 private void flushStructuralChanges () {
134     if (_flushingStructuralChanges || _pendingStructuralChanges.isEmpty())
135         return;
136
137     _flushingStructuralChanges = true;
138     try {
139         while (!_pendingStructuralChanges.isEmpty()) {
140             List<Runnable> changes = List.copyOf(_pendingStructuralChanges);
141             _pendingStructuralChanges.clear();
142
143             for (Runnable change : changes)
144                 change.run();
145         }
146     } finally {
147         _flushingStructuralChanges = false;
148     }
149 }

```

Листинг Б.5 – главный цикл приложения Application.java

```
1  public void run (Scene startScene) {
2      _context.use();
3
4      try {
5          if (startScene != null)
6              _context.sceneManager().load(startScene);
7          else if (!_context.sceneManager().hasActiveScene())
8              throw new RuntimeException("No start scene");
9
10         updateCameraAspect();
11         _window.show();
12
13         _context.time().reset((float) glfwGetTime());
14         while (!_window.shouldClose()) {
15             _window.clear(_context.config().clearColor());
16
17             float currentTime = (float) glfwGetTime();
18             _context.time().update(currentTime);
19             _context.sceneManager().activeScene().loop(_context.time().deltaTime
20                 ());
21             _context.sceneManager().applyPendingChanges();
22             updateCameraAspect();
23             _context.input().update();
24             _window.update();
25         }
26     } finally {
27         _context.destroy();
28     }
```

Листинг Б.6 – управление сменой сцен SceneManager.java

```
1 package qchromatic.jecse.engine;
2
3 public final class SceneManager {
4     private static final SceneManager DEFAULT = new SceneManager();
5
6     private Scene _activeScene;
7     private Scene _pendingScene;
8     private boolean _pendingReplace;
9
10    public Scene activeScene () {
11        if (_activeScene == null) throw new RuntimeException("No active scene");
12        return _activeScene;
13    }
14
15    public boolean hasActiveScene () {
16        return _activeScene != null;
17    }
18
19    public void load (Scene scene) {
20        if (scene == null) return;
21        clearPendingChange();
22
23        if (_activeScene == scene) {
24            _activeScene.start();
25            return;
26        }
27
28        if (_activeScene != null)
29            _activeScene.stop();
30
31        _activeScene = scene;
32        _activeScene.start();
33    }
34
35    public void replace (Scene scene) {
36        if (scene == null) return;
37        clearPendingChange();
38
39        if (_activeScene == scene) {
40            _activeScene.start();
41            return;
42        }
43
44        if (_activeScene != null) {
45            Scene previousScene = _activeScene;
46            _activeScene = null;
47            previousScene.dispose();
48        }
49
50        _activeScene = scene;
51        _activeScene.start();
52    }
53 }
```

```

54 public void requestLoad (Scene scene) {
55     if (scene == null) return;
56
57     _pendingScene = scene;
58     _pendingReplace = false;
59 }
60
61 public void requestReplace (Scene scene) {
62     if (scene == null) return;
63
64     _pendingScene = scene;
65     _pendingReplace = true;
66 }
67
68 public void applyPendingChanges () {
69     if (_pendingScene == null) return;
70
71     Scene scene = _pendingScene;
72     boolean replace = _pendingReplace;
73     clearPendingChange();
74
75     if (replace)
76         replace(scene);
77     else
78         load(scene);
79 }
80
81 public void unload () {
82     if (_activeScene == null) return;
83     clearPendingChange();
84
85     _activeScene.stop();
86     _activeScene = null;
87 }
88
89 public void disposeActiveScene () {
90     if (_activeScene == null) return;
91     clearPendingChange();
92
93     Scene scene = _activeScene;
94     _activeScene = null;
95     scene.dispose();
96 }
97
98 public static Scene getActiveScene () {
99     return currentManager().activeScene();
100 }
101
102 public static boolean hasScene () {
103     return currentManager().hasActiveScene();
104 }
105
106 public static void loadScene (Scene scene) {
107     currentManager().load(scene);

```

```

108     }
109
110     public static void replaceScene (Scene scene) {
111         currentManager().replace(scene);
112     }
113
114     public static void requestLoadScene (Scene scene) {
115         currentManager().requestLoad(scene);
116     }
117
118     public static void requestReplaceScene (Scene scene) {
119         currentManager().requestReplace(scene);
120     }
121
122     public static void unloadScene () {
123         currentManager().unload();
124     }
125
126     public static void disposeScene () {
127         currentManager().disposeActiveScene();
128     }
129
130     private static SceneManager currentManager () {
131         EngineContext context = EngineContext.currentOrNull();
132         return context == null ? DEFAULT : context.sceneManager();
133     }
134
135     private void clearPendingChange () {
136         _pendingScene = null;
137         _pendingReplace = false;
138     }
139 }

```

Листинг Б.7 – фрагменты графической подсистемы RenderSystem.java

```
1 public final class RenderSystem {
2     public void init () {
3         _renderables = scene.query(Transform.class, MeshRenderer.class);
4         _cameras = scene.query(Transform.class, Camera.class);
5         _lights = scene.query(DirectionalLight.class);
6     }
7
8     public void loop (float dtime) {
9         Entity cameraEntity = selectCamera();
10        if (cameraEntity == null) return;
11
12        Camera camera = cameraEntity.getComponent(Camera.class);
13        LightState light = selectLight();
14
15        buildRenderQueue(camera);
16        sortRenderQueue();
17
18        for (RenderCommand command : _renderQueue)
19            renderCommand(command, camera, light);
20
21        checkOpenGLErrors();
22    }
23
24    private Entity selectCamera () {
25        Entity selected = null;
26        int selectedPriority = Integer.MIN_VALUE;
27
28        for (Entity entity : _cameras) {
29            Camera camera = entity.getComponent(Camera.class);
30            if (camera == null || !camera.enabled()) continue;
31
32            if (selected == null || camera.priority() > selectedPriority) {
33                selected = entity;
34                selectedPriority = camera.priority();
35            }
36        }
37
38        return selected;
39    }
40
41    private LightState selectLight () {
42        for (Entity entity : _lights) {
43            DirectionalLight light = entity.getComponent(DirectionalLight.class);
44            if (light != null && light.enabled())
45                return new LightState(light.direction(), light.color().multiplied(
46                    light.intensity()), true);
47        }
48
49        return new LightState(new Vec3(0f, -1f, 0f), new Vec4(1f), false);
50    }
51
52    private void buildRenderQueue (Camera camera) {
```

```

52     _renderQueue.clear();
53
54     for (Entity entity : _renderables) {
55         MeshRenderer meshRenderer = entity.getComponent(MeshRenderer.class)
56         ;
57         Transform transform = entity.getComponent(Transform.class);
58
59         if (meshRenderer == null || transform == null || !meshRenderer.
60             enabled()) continue;
61         if ((camera.cullingMask() & meshRenderer.layer()) == 0) continue;
62
63         Mesh mesh = meshRenderer.mesh();
64         Material material = meshRenderer.material();
65
66         if (mesh == null || material == null) continue;
67         if (!isVisible(camera, transform, meshRenderer)) continue;
68
69         _renderQueue.add(new RenderCommand(mesh, material, transform,
70             meshRenderer.renderQueue()));
71     }
72
73     private void sortRenderQueue () {
74         _renderQueue.sort(Comparator
75             .comparingInt((RenderCommand command) -> command.renderQueue)
76             .thenComparingInt(command -> command.material.shader().handler())
77             .thenComparingInt(command -> command.material.texture().id())
78             .thenComparingInt(command -> java.lang.System.identityHashCode(
79                 command.mesh)));
80     }
81
82     private void renderCommand (RenderCommand command, Camera camera,
83         LightState light) {
84         MeshGpuResource resource = getMeshGpuResource(command.mesh);
85         Material material = command.material;
86
87         material.use();
88
89         Shader shader = material.shader();
90         shader.setUniform("u_model", command.transform.getModelMatrix().
91             transposed());
92         shader.setUniform("u_view", camera.getViewMatrix().transposed());
93         shader.setUniform("u_projection", camera.getProjectionMatrix().
94             transposed());
95         shader.setUniform("u_lightDirection", light.direction);
96         shader.setUniform("u_lightColor", light.color);
97         shader.setUniform("u_ambientColor", _ambientColor);
98         shader.setUniform("u_lit", light.enabled);
99
100        glBindVertexArray(resource.vaoId());
101        glDrawElements(GL_TRIANGLES, resource.indexCount(), GL_UNSIGNED_INT,
102            0);
103        glBindVertexArray(0);
104    }

```


Листинг Б.8 – фрагменты физической подсистемы PhysicsSystem.java

```
1 public final class PhysicsSystem {
2     public void loop (float dtime) {
3         if (dtime <= 0f) return;
4
5         _accumulator += Math.min(dtime, _fixedTimeStep * _maxSubSteps);
6
7         int steps = 0;
8         while (_accumulator >= _fixedTimeStep && steps < _maxSubSteps) {
9             step(_fixedTimeStep);
10            _accumulator -= _fixedTimeStep;
11            steps++;
12        }
13
14        if (steps == _maxSubSteps && _accumulator >= _fixedTimeStep)
15            _accumulator = 0f;
16
17        if (_debugDrawColliders)
18            drawColliderBounds();
19    }
20
21    private void step (float dtime) {
22        integrateForces(dtime);
23        integrateTransforms(dtime);
24        solveVelocity();
25        solvePosition();
26        updateSleeping(dtime);
27    }
28
29    private void integrateForces (float dtime) {
30        for (Entity entity : _bodies) {
31            Transform transform = entity.getComponent(Transform.class);
32            Rigidbody body = entity.getComponent(Rigidbody.class);
33            if (transform == null || body == null || !body.enabled()) continue;
34
35            if (body.staticBody()) {
36                body.velocity(0f, 0f, 0f);
37                body.angularVelocity(0f, 0f, 0f);
38                body.clearForces();
39                continue;
40            }
41
42            if (body.sleeping()) {
43                body.clearForces();
44                continue;
45            }
46
47            if (body.dynamic()) {
48                Vec3 velocity = body.velocity();
49                Vec3 acceleration = body.force().multiplied(body.inverseMass());
50                if (body.useGravity())
51                    acceleration.add(_gravity.multiplied(body.gravityScale()));
52
53                velocity.add(acceleration.multiplied(dtime));
```

```

54         velocity.mul(Math.max(0f, 1f - body.linearDamping() * dttime));
55         body.velocity(velocity);
56
57         Vec3 angularVelocity = body.angularVelocity();
58         if (!body.lockRotation()) {
59             Vec3 angularAcceleration = applyInverseInertia(entity,
60                 transform, body, body.torque());
61             angularVelocity.add(angularAcceleration.multiplied(dttime));
62             angularVelocity.mul(Math.max(0f, 1f - body.angularDamping() *
63                 dttime));
64             body.angularVelocity(angularVelocity);
65         } else {
66             body.angularVelocity(0f, 0f, 0f);
67         }
68     }
69     clampBodySpeeds(body);
70     body.clearForces();
71 }
72
73 private void integrateTransforms (float dttime) {
74     for (Entity entity : _bodies) {
75         Transform transform = entity.getComponent(Transform.class);
76         Rigidbody body = entity.getComponent(Rigidbody.class);
77         if (transform == null || body == null || !body.enabled() || body.
78             staticBody() || body.sleeping()) continue;
79
80         transform.position(transform.position().added(body.velocity().
81             multiplied(dttime)));
82
83         Vec3 angularVelocity = body.angularVelocity();
84         if (!body.lockRotation() && angularVelocity.lengthSquared() > 0f) {
85             float angleRad = angularVelocity.length() * dttime;
86             Vec3 axis = angularVelocity.normalized();
87             Quaternion delta = Quaternion.axisAngle(axis, (float) Math.
88                 toDegrees(angleRad));
89             transform.rotation(delta.multiplied(transform.rotation()).
90                 normalized());
91         }
92     }
93 }
94
95 private List<Contact> contacts () {
96     List<ColliderEntry> entries = colliderEntries();
97     List<Contact> result = new ArrayList<>();
98
99     for (int i = 0; i < entries.size(); i++) {
100         for (int j = i + 1; j < entries.size(); j++) {
101             ColliderEntry a = entries.get(i);
102             ColliderEntry b = entries.get(j);
103
104             if (!canCollide(a.collider, b.collider)) continue;

```

```

102         Contact contact = contact(a, b);
103         if (contact != null)
104             result.add(contact);
105     }
106 }
107
108 return result;
109 }
110
111 private Contact contact (ColliderEntry a, ColliderEntry b) {
112     if (!aabbOverlap(a.shape, b.shape)) return null;
113
114     SatResult result = new SatResult();
115     Vec3 centerDelta = subtract(b.shape.center, a.shape.center);
116
117     for (int i = 0; i < 3; i++) {
118         if (!testAxis(a.shape.axes[i], centerDelta, a.shape, b.shape,
119             result)) return null;
120         if (!testAxis(b.shape.axes[i], centerDelta, a.shape, b.shape,
121             result)) return null;
122     }
123
124     for (int i = 0; i < 3; i++) {
125         for (int j = 0; j < 3; j++) {
126             Vec3 axis = a.shape.axes[i].crossed(b.shape.axes[j]);
127             if (axis.lengthSquared() > EPSILON && !testAxis(axis, centerDelta,
128                 a.shape, b.shape, result))
129                 return null;
130         }
131     }
132
133     if (result.normal.lengthSquared() <= 0f)
134         return null;
135
136     return new Contact(a, b, result.normal, result.penetration,
137         contactPoints(a.shape, b.shape, result.normal, result.penetration)
138     );
139 }
140
141 private boolean testAxis (Vec3 axis, Vec3 centerDelta, BoxShape a,
142     BoxShape b, SatResult result) {
143     Vec3 normal = axis.normalized();
144     float radiusA = projectionRadius(a, normal);
145     float radiusB = projectionRadius(b, normal);
146     float distance = centerDelta.dot(normal);
147     float overlap = radiusA + radiusB - Math.abs(distance);
148
149     if (overlap <= 0f)
150         return false;
151
152     if (overlap < result.penetration) {
153         result.penetration = overlap;
154         result.normal = distance >= 0f ? normal : normal.multiplied(-1f);
155     }
156 }

```

```

150
151     return true;
152 }
153
154 private void resolveVelocityAtPoint (Contact contact, Vec3 point) {
155     float invMassA = inverseMass(contact.a.body);
156     float invMassB = inverseMass(contact.b.body);
157     Vec3 centerA = worldCenterOfMass(contact.a);
158     Vec3 centerB = worldCenterOfMass(contact.b);
159     Vec3 rA = subtract(point, centerA);
160     Vec3 rB = subtract(point, centerB);
161
162     Vec3 relativeVelocity = subtract(pointVelocity(contact.b, rB),
163                                     pointVelocity(contact.a, rA));
164     float velocityAlongNormal = relativeVelocity.dot(contact.normal);
165     float velocityBias = contactVelocityBias(contact, velocityAlongNormal);
166     if (velocityAlongNormal >= velocityBias)
167         return;
168
169     float denominator = invMassA + invMassB
170                       + angularDenominator(contact.a, rA, contact.normal)
171                       + angularDenominator(contact.b, rB, contact.normal);
172     if (denominator <= 0f)
173         return;
174
175     float impulseMagnitude = (velocityBias - velocityAlongNormal) /
176                             denominator;
177     applyImpulse(contact.a, contact.normal.multiplied(-impulseMagnitude),
178                 point);
179     applyImpulse(contact.b, contact.normal.multiplied(impulseMagnitude),
180                 point);
181
182     relativeVelocity = subtract(pointVelocity(contact.b, rB),
183                                 pointVelocity(contact.a, rA));
184     Vec3 tangent = subtract(relativeVelocity, contact.normal.multiplied(
185                             relativeVelocity.dot(contact.normal)));
186     if (tangent.lengthSquared() > EPSILON) {
187         tangent.normalize();
188         float tangentDenominator = invMassA + invMassB
189                                   + angularDenominator(contact.a, rA, tangent)
190                                   + angularDenominator(contact.b, rB, tangent);
191         if (tangentDenominator > 0f) {
192             float tangentImpulseMagnitude = -relativeVelocity.dot(tangent) /
193                                             tangentDenominator;
194             float friction = (float) Math.sqrt(contact.a.collider.friction()
195                                                 * contact.b.collider.friction());
196             float maxFriction = Math.abs(impulseMagnitude) * friction;
197             tangentImpulseMagnitude = clamp(tangentImpulseMagnitude, -
198                                             maxFriction, maxFriction);
199
200             applyImpulse(contact.a, tangent.multiplied(-
201                                     tangentImpulseMagnitude), point);

```

```

192         applyImpulse(contact.b, tangent.multiplied(
193             tangentImpulseMagnitude), point);
194     }
195 }
196
197 private void resolvePosition (Contact contact) {
198     float invMassA = inverseMass(contact.a.body);
199     float invMassB = inverseMass(contact.b.body);
200     float invMassSum = invMassA + invMassB;
201     if (invMassSum <= 0f) return;
202
203     float percent = 0.2f;
204     float correction = Math.min(0.05f, Math.max(0f, contact.penetration -
205         POSITION_SLOP) * percent);
206     if (correction <= 0f) return;
207
208     if (invMassA > 0f)
209         move(contact.a.transform, contact.normal.multiplied(-correction *
210             invMassA / invMassSum));
211     if (invMassB > 0f)
212         move(contact.b.transform, contact.normal.multiplied(correction *
213             invMassB / invMassSum));
214 }
215
216 private void updateSleeping (float dtime) {
217     Map<Rigidbody, Boolean> supported = new IdentityHashMap<>();
218     Map<Rigidbody, Boolean> touchedByActiveBody = new IdentityHashMap<>()
219     ;
220     for (Contact contact : contacts()) {
221         if (contact.a.collider.trigger() || contact.b.collider.trigger())
222             continue;
223         if (isSupported(contact, contact.a, contact.normal.multiplied(-1f))
224             && isStableSupport(contact.b.body))
225             supported.put(contact.a.body, true);
226         if (isSupported(contact, contact.b, contact.normal) &&
227             isStableSupport(contact.a.body))
228             supported.put(contact.b.body, true);
229         if (isActiveBody(contact.b.body) && contact.a.body != null)
230             touchedByActiveBody.put(contact.a.body, true);
231         if (isActiveBody(contact.a.body) && contact.b.body != null)
232             touchedByActiveBody.put(contact.b.body, true);
233     }
234
235     for (Entity entity : _bodies) {
236         Rigidbody body = entity.getComponent(Rigidbody.class);
237         if (body == null || !body.enabled() || !body.dynamic()) continue;
238
239         if (!supported.containsKey(body)) {
240             _sleepTimers.remove(body);
241             if (body.sleeping())
242                 body.wakeUp();
243             continue;
244         }
245     }

```

```

238
239     if (touchedByActiveBody.containsKey(body)) {
240         _sleepTimers.remove(body);
241         if (body.sleeping())
242             body.wakeUp();
243         continue;
244     }
245
246     boolean slow = body.velocity().lengthSquared() <=
247         _sleepLinearThreshold * _sleepLinearThreshold
248         && body.angularVelocity().lengthSquared() <=
249             _sleepAngularThreshold * _sleepAngularThreshold;
250     if (!slow) {
251         _sleepTimers.remove(body);
252         continue;
253     }
254
255     float sleepTime = _sleepTimers.getOrDefault(body, 0f) + dtime;
256     _sleepTimers.put(body, sleepTime);
257     if (sleepTime >= _sleepDelay)
258         body.sleep();
259 }

```

Листинг Б.9 – выполнение пользовательских скриптов ScriptSystem.java

```
1 package qchromatic.jecse.system;
2
3 import qchromatic.jecse.component.Script;
4 import qchromatic.jecse.core.Component;
5 import qchromatic.jecse.core.Entity;
6 import qchromatic.jecse.core.EntityQuery;
7 import qchromatic.jecse.core.System;
8
9 import java.util.*;
10
11 public class ScriptSystem extends System {
12     private final List<Script> _scripts;
13     private final Set<Script> _initializedScripts;
14     private final Set<Script> _startedScripts;
15     private final Map<Entity, List<Script>> _scriptsByEntity;
16     private EntityQuery _scriptEntities;
17     private boolean _running;
18
19     public ScriptSystem () {
20         super(0);
21         _scripts = new ArrayList<>();
22         _initializedScripts = new LinkedHashSet<>();
23         _startedScripts = new LinkedHashSet<>();
24         _scriptsByEntity = new LinkedHashMap<>();
25         _running = false;
26     }
27
28     @Override
29     public void init () {
30         _scriptEntities = scene.queryInheritance(Script.class);
31         _scripts.clear();
32         _initializedScripts.clear();
33         _startedScripts.clear();
34         _scriptsByEntity.clear();
35
36         for (Entity entity : _scriptEntities)
37             registerEntity(entity);
38     }
39
40     @Override
41     public void start () {
42         _running = true;
43         for (Script script : List.copyOf(_scripts))
44             startScript(script);
45     }
46
47     @Override
48     public void loop (float dtime) {
49         for (Script script : List.copyOf(_scripts))
50             script.loop(dtime);
51     }
52
53     @Override
```

```

54 public void stop () {
55     _running = false;
56     for (Script script : List.copyOf(_startedScripts))
57         stopScript(script);
58 }
59
60 @Override
61 public void destroy () {
62     for (Script script : List.copyOf(_scripts))
63         destroyScript(script);
64
65     _scripts.clear();
66     _initializedScripts.clear();
67     _startedScripts.clear();
68     _scriptsByEntity.clear();
69 }
70
71 @Override
72 public void onEntityAdded (Entity entity) {
73     registerEntity(entity);
74 }
75
76 @Override
77 public void onEntityRemoved (Entity entity) {
78     unregisterEntity(entity, true);
79 }
80
81 @Override
82 public void onComponentAdded(Entity entity, Component component) {
83     if (component instanceof Script script)
84         registerScript(entity, script);
85 }
86
87 @Override
88 public void onComponentRemoved(Entity entity, Component component) {
89     if (component instanceof Script script)
90         unregisterScript(entity, script, true);
91 }
92
93 private void registerEntity (Entity entity) {
94     for (Script script : entity.getComponentsInheritance(Script.class))
95         registerScript(entity, script);
96 }
97
98 private void registerScript (Entity entity, Script script) {
99     List<Script> entityScripts = _scriptsByEntity.computeIfAbsent(entity,
100         ignored -> new ArrayList<>());
101     if (entityScripts.contains(script)) return;
102
103     entityScripts.add(script);
104     _scripts.add(script);
105     initScript(script);
106
107     if (_running)

```



```

107     startScript(script);
108 }
109
110 private void unregisterEntity (Entity entity, boolean destroy) {
111     List<Script> entityScripts = _scriptsByEntity.remove(entity);
112     if (entityScripts == null) return;
113
114     for (Script script : List.copyOf(entityScripts))
115         unregisterScript(entity, script, destroy);
116 }
117
118 private void unregisterScript (Entity entity, Script script, boolean
119     destroy) {
120     List<Script> entityScripts = _scriptsByEntity.get(entity);
121     if (entityScripts != null) {
122         entityScripts.remove(script);
123         if (entityScripts.isEmpty())
124             _scriptsByEntity.remove(entity);
125     }
126
127     _scripts.remove(script);
128
129     if (destroy)
130         destroyScript(script);
131 }
132
133 private void initScript (Script script) {
134     if (_initializedScripts.add(script))
135         script.init();
136 }
137
138 private void startScript (Script script) {
139     initScript(script);
140     if (_startedScripts.add(script))
141         script.start();
142 }
143
144 private void stopScript (Script script) {
145     if (_startedScripts.remove(script))
146         script.stop();
147 }
148
149 private void destroyScript (Script script) {
150     stopScript(script);
151
152     if (_initializedScripts.remove(script))
153         script.destroy();
154 }

```

Листинг Б.10 – обработка пользовательского ввода InputState.java

```
1 package qchromatic.jecse.engine;
2
3 import qchromatic.jecse.common.Vec2;
4
5 public final class InputState {
6     private static final int MAX_KEYS = 512;
7     private static final int MAX_MOUSE_BUTTONS = 16;
8
9     private final boolean[] _keys;
10    private final boolean[] _keysPrev;
11    private final boolean[] _mouseButtons;
12    private final boolean[] _mouseButtonsPrev;
13
14    private float _mouseX;
15    private float _mouseY;
16    private float _mouseDX;
17    private float _mouseDY;
18    private boolean _hasMousePosition;
19    private float _scrollX;
20    private float _scrollY;
21
22    public InputState () {
23        _keys = new boolean[MAX_KEYS];
24        _keysPrev = new boolean[MAX_KEYS];
25        _mouseButtons = new boolean[MAX_MOUSE_BUTTONS];
26        _mouseButtonsPrev = new boolean[MAX_MOUSE_BUTTONS];
27    }
28
29    public void update() {
30        java.lang.System.arraycopy(_keys, 0, _keysPrev, 0, MAX_KEYS);
31        java.lang.System.arraycopy(_mouseButtons, 0, _mouseButtonsPrev, 0,
32            MAX_MOUSE_BUTTONS);
33        _mouseDX = 0f;
34        _mouseDY = 0f;
35        _scrollX = 0f;
36        _scrollY = 0f;
37    }
38
39    public void reset () {
40        for (int i = 0; i < MAX_KEYS; i++) {
41            _keys[i] = false;
42            _keysPrev[i] = false;
43        }
44
45        for (int i = 0; i < MAX_MOUSE_BUTTONS; i++) {
46            _mouseButtons[i] = false;
47            _mouseButtonsPrev[i] = false;
48        }
49
50        _mouseDX = 0f;
51        _mouseDY = 0f;
52        _mouseX = 0f;
53        _mouseY = 0f;
```

```

53     _hasMousePosition = false;
54     _scrollX = 0f;
55     _scrollY = 0f;
56 }
57
58 public void setKeyState(int key, boolean pressed) {
59     if (key >= 0 && key < MAX_KEYS) _keys[key] = pressed;
60 }
61
62 public void setMouseButtonState(int button, boolean pressed) {
63     if (button >= 0 && button < MAX_MOUSE_BUTTONS) _mouseButtons[button] =
        pressed;
64 }
65
66 public void setMousePosition(float x, float y) {
67     if (!_hasMousePosition) {
68         _mouseX = x;
69         _mouseY = y;
70         _mouseDX = 0f;
71         _mouseDY = 0f;
72         _hasMousePosition = true;
73         return;
74     }
75
76     _mouseDX += x - _mouseX;
77     _mouseDY += y - _mouseY;
78     _mouseX = x;
79     _mouseY = y;
80 }
81
82 public void setScrollOffset(float xoffset, float yoffset) {
83     _scrollX += xoffset;
84     _scrollY += yoffset;
85 }
86
87 public void resetMousePositionTracking () {
88     _hasMousePosition = false;
89     _mouseDX = 0f;
90     _mouseDY = 0f;
91 }
92
93 public boolean getKey (int key) {
94     return key >= 0 && key < MAX_KEYS && _keys[key];
95 }
96
97 public boolean getKeyDown (int key) {
98     return key >= 0 && key < MAX_KEYS && _keys[key] && !_keysPrev[key];
99 }
100
101 public boolean getKeyUp (int key) {
102     return key >= 0 && key < MAX_KEYS && !_keys[key] && _keysPrev[key];
103 }
104
105 public boolean getMouseButton (int button) {

```

```

106     return button >= 0 && button < MAX_MOUSE_BUTTONS && _mouseButtons[button
107         ];
108 }
109 public boolean getMouseButtonDown (int button) {
110     return button >= 0 && button < MAX_MOUSE_BUTTONS && _mouseButtons[button]
111         && !_mouseButtonsPrev[button];
112 }
113 public boolean getMouseButtonUp (int button) {
114     return button >= 0 && button < MAX_MOUSE_BUTTONS && !_mouseButtons[button
115         ] && _mouseButtonsPrev[button];
116 }
117 public Vec2 getMousePosition () {
118     return new Vec2(_mouseX, _mouseY);
119 }
120
121 public Vec2 getMouseDelta () {
122     return new Vec2(_mouseDX, _mouseDY);
123 }
124
125 public Vec2 getMouseScroll () {
126     return new Vec2(_scrollX, _scrollY);
127 }
128 }

```

Листинг Б.11 – фрагменты демонстрационной физической сцены

PhysicsScene.java

```
1 public final class PhysicsScene {
2     private static Scene createScene () {
3         Scene scene = new Scene();
4
5         scene.addSystem(new ScriptSystem());
6
7         scene.addSystem(new PhysicsSystem()
8             .gravity(0f, -9.81f, 0f)
9             .fixedTimeStep(1f / 60f)
10            .maxSubSteps(5)
11            .velocityIterations(12)
12            .positionIterations(10)
13            .maxSpeeds(16f, 8f)
14            .sleepThresholds(0.08f, 0.12f, 0.5f)
15            .debugDrawColliders(true));
16
17         scene.addSystem(new RenderSystem()
18             .ambientColor(new Vec4(0.18f, 0.20f, 0.23f, 1f)));
19
20         scene.addSystem(new DebugRenderSystem());
21
22         scene.addEntity(new Entity("sun")
23             .addComponent(new DirectionalLight()
24                 .direction(-0.4f, -1f, -0.25f)
25                 .color(new Vec4(1f, 0.94f, 0.82f, 1f))
26                 .intensity(1.2f)));
27
28         scene.addEntity(new Entity("camera")
29             .addComponent(new Transform()
30                 .position(0f, 4f, 9f)
31                 .rotation(Quaternion.euler(-22f, 0f, 0f)))
32             .addComponent(new Camera()
33                 .fov(60f)
34                 .near(0.1f)
35                 .far(80f)
36                 .cullingMask(LAYER_WORLD | LAYER_PHYSICS)
37                 .priority(10))
38             .addComponent(new FreecamController()
39                 .speed(7f)
40                 .sensitivity(18f)));
41
42         addFloor(scene);
43         addWall(scene, "left-wall", -4f, 1f, 0f, 0.25f, 1.2f, 4f);
44         addWall(scene, "right-wall", 4f, 1f, 0f, 0.25f, 1.2f, 4f);
45         addWall(scene, "back-wall", 0f, 1f, -4f, 4f, 1.2f, 0.25f);
46
47         addNarrowSupport(scene);
48         addDynamicBox(scene, "off-center-box", 0.75f, 3.2f, -1.2f, new Vec4
49             (0.25f, 0.68f, 1f, 1f), new Vec3(0f, 0f, 0f));
```

```

50 addDynamicBox(scene, "falling-box-a", -1.2f, 4.5f, 0f, new Vec4(0.25f
    , 0.68f, 1f, 1f), new Vec3(1.6f, 0f, 0f));
51 addDynamicBox(scene, "falling-box-b", 0.1f, 7f, 0.2f, new Vec4(1f,
    0.55f, 0.24f, 1f), new Vec3(-0.6f, 0f, 0.2f));
52 addDynamicBox(scene, "falling-box-c", 3f, 7f, 0.2f, new Vec4(1f, 0f,
    1f, 1f), new Vec3(-0.6f, 0f, 0.2f));
53 addDynamicBox(scene, "low-bounce-box", 1.4f, 5.8f, -0.5f, new Vec4
    (0.55f, 1f, 0.32f, 1f), new Vec3(-1.1f, 0f, 0.5f), 0.08f);
54
55 scene.addEntity(new Entity("kinematic-platform")
56     .addComponent(new Transform()
57         .position(0f, 0.35f, 3.1f)
58         .scale(1.8f, 0.25f, 0.7f))
59     .addComponent(new Rigidbody()
60         .type(PhysicsBodyType.KINEMATIC)
61         .velocity(0.9f, 0f, 0f)
62         .useGravity(false))
63     .addComponent(new BoxCollider()
64         .halfExtents(0.5f, 0.5f, 0.5f)
65         .friction(0.9f))
66     .addComponent(new MeshRenderer()
67         .mesh(Meshes.cube())
68         .material(new Material().color(new Vec4(1f, 0.9f, 0.28f, 1f))
69             )
70         .layer(LAYER_PHYSICS)
71         .boundsRadius(2f))
72     .addComponent(new PlatformBounceScript(-2.4f, 2.4f)));
73
74 scene.addEntity(new Entity("physics-grid")
75     .addComponent(new PhysicsGridScript()));
76
77 scene.addEntity(new Entity("scene-controls")
78     .addComponent(new PhysicsSceneControlScript()));
79
80 return scene;
81 }
82
83 private static void addDynamicBox (Scene scene, String id, float x,
84     float y, float z, Vec4 color, Vec3 velocity) {
85     addDynamicBox(scene, id, x, y, z, color, velocity, 0.1f);
86 }
87
88 private static final class PhysicsSceneControlScript extends Script {
89     @Override
90     public void loop (float dt) {
91         if (!Input.getKeyDown(KeyCode.R)) return;
92
93         EngineContext context = EngineContext.current();
94         context.window().setCursorDisabled();
95         context.sceneManager().requestReplace(createScene());
96     }
97 }
98
99 }

```

Листинг Б.12 – фрагменты тестирования физической подсистемы PhysicsSystemTest.java

```
1 public final class PhysicsSystemTest {
2     private static void testOverhangingBoxFallsToFloor () {
3         Scene scene = new Scene();
4         scene.addSystem(new PhysicsSystem()
5             .gravity(0f, -10f, 0f)
6             .fixedTimeStep(1f / 60f)
7             .velocityIterations(12)
8             .positionIterations(10)
9             .sleepThresholds(0.08f, 0.12f, 0.4f));
10
11         scene.addEntity(new Entity("floor")
12             .addComponent(new Transform()
13                 .position(0f, -0.1f, 0f)
14                 .scale(8f, 0.2f, 8f))
15             .addComponent(new BoxCollider()
16                 .halfExtents(0.5f, 0.5f, 0.5f)
17                 .friction(1f)));
18
19         scene.addEntity(new Entity("narrow-support")
20             .addComponent(new Transform()
21                 .position(0f, 0.25f, 0f))
22             .addComponent(new BoxCollider()
23                 .halfExtents(0.5f, 0.25f, 2f)
24                 .friction(0.9f)));
25
26         Rigidbody body = new Rigidbody()
27             .mass(1f)
28             .linearDamping(0.04f)
29             .angularDamping(0.08f);
30         Transform transform = new Transform()
31             .position(0.82f, 1f, 0f);
32         scene.addEntity(new Entity("overhanging-box")
33             .addComponent(transform)
34             .addComponent(body)
35             .addComponent(new BoxCollider()
36                 .halfExtents(0.5f, 0.5f, 0.5f)
37                 .friction(0.9f)));
38
39         scene.init();
40         for (int i = 0; i < 120; i++)
41             scene.loop(1f / 60f);
42
43         if (transform.position().y > 0.8f)
44             throw new AssertionError("overhanging box stayed glued to the
45                 support too long: position="
46                 + transform.position().x + "," + transform.position().y + "," +
47                 transform.position().z
48                 + " angular=" + body.angularVelocity().x + "," + body.
49                     angularVelocity().y + "," + body.angularVelocity().z
50                 + " sleeping=" + body.sleeping());
```

```

49     scene.dispose();
50 }
51
52 private static void testWallContactDoesNotGlueFallingBox () {
53     Scene scene = new Scene();
54     scene.addSystem(new PhysicsSystem()
55         .gravity(0f, -10f, 0f)
56         .fixedTimeStep(1f / 60f)
57         .velocityIterations(10)
58         .positionIterations(10)
59         .maxSpeeds(20f, 10f));
60
61     scene.addEntity(new Entity("wall")
62         .addComponent(new Transform()
63             .position(0f, 2f, 0f)
64             .scale(0.2f, 5f, 4f))
65         .addComponent(new BoxCollider()
66             .halfExtents(0.5f, 0.5f, 0.5f)
67             .friction(1f)));
68
69     Rigidbody body = new Rigidbody()
70         .mass(1f)
71         .velocity(-0.3f, 0f, 0f)
72         .linearDamping(0.02f)
73         .angularDamping(0.08f);
74     Transform transform = new Transform()
75         .position(0.55f, 3f, 0f);
76     scene.addEntity(new Entity("wall-touching-box")
77         .addComponent(transform)
78         .addComponent(body)
79         .addComponent(new BoxCollider()
80             .halfExtents(0.5f, 0.5f, 0.5f)
81             .friction(1f)));
82
83     scene.init();
84     for (int i = 0; i < 180; i++)
85         scene.loop(1f / 60f);
86
87     if (transform.position().y > 1.5f)
88         throw new AssertionError("box was glued to the wall instead of
89             falling: y=" + transform.position().y);
90
91     scene.dispose();
92 }
93
94 private static void testYawSpinSettlesOnFloor () {
95     Scene scene = new Scene();
96     scene.addSystem(new PhysicsSystem()
97         .gravity(0f, -10f, 0f)
98         .fixedTimeStep(1f / 60f)
99         .velocityIterations(12)
100        .positionIterations(10)
101        .maxSpeeds(20f, 10f)
102        .sleepThresholds(0.08f, 0.12f, 0.35f));

```



```

102
103     scene.addEntity(new Entity("ground")
104         .addComponent(new Transform()
105             .position(0f, -0.1f, 0f)
106             .scale(8f, 0.2f, 8f))
107         .addComponent(new BoxCollider()
108             .halfExtents(0.5f, 0.5f, 0.5f)
109             .friction(1f)));
110
111     Rigidbody body = new Rigidbody()
112         .mass(1f)
113         .angularVelocity(0f, 8f, 0f)
114         .linearDamping(0.05f)
115         .angularDamping(0.08f);
116     Transform transform = new Transform()
117         .position(0f, 0.55f, 0f);
118     scene.addEntity(new Entity("yaw-spinning-box")
119         .addComponent(transform)
120         .addComponent(body)
121         .addComponent(new BoxCollider()
122             .halfExtents(0.5f, 0.5f, 0.5f)
123             .friction(1f)));
124
125     scene.init();
126     for (int i = 0; i < 600; i++)
127         scene.loop(1f / 60f);
128
129     if (body.angularVelocity().length() > 0.25f)
130         throw new AssertionError("yaw spinning box did not lose angular
131             velocity");
132     if (Math.abs(transform.position().x) > 0.4f || Math.abs(transform.
133         position().z) > 0.4f)
134         throw new AssertionError("yaw spinning box drifted too far");
135
136     scene.dispose();
137 }
138
139 private static void expectNear (String label, float actual, float
140     expected, float tolerance) {
141     if (Math.abs(actual - expected) > tolerance)
142         throw new AssertionError(label + ": expected " + expected + " +/- "
143             + tolerance + ", got " + actual);
144 }
145 }

```

Автор ВКР

(подпись, дата)

К. Р. Алешин

Руководитель ВКР

(подпись, дата)

А. Ф. Рубанов

Нормоконтроль

(подпись, дата)

А. А. Чаплыгин

Место для диска